

---

**NED UNIVERSITY OF ENGINEERING & TECHNOLOGY**

Department of Computer Science & Information Technology

---

# OBJECT ORIENTED PROGRAMMING

( CT-260 )

*Lab Tasks & Complex Computing Problem (CCP) Report*



|                 |                          |
|-----------------|--------------------------|
| <b>Name</b>     | Areefa Samar             |
| <b>Roll No</b>  | CT-25062                 |
| <b>Section</b>  | B                        |
| <b>Program</b>  | BS Computer Science & IT |
| <b>Semester</b> | 2nd Semester             |

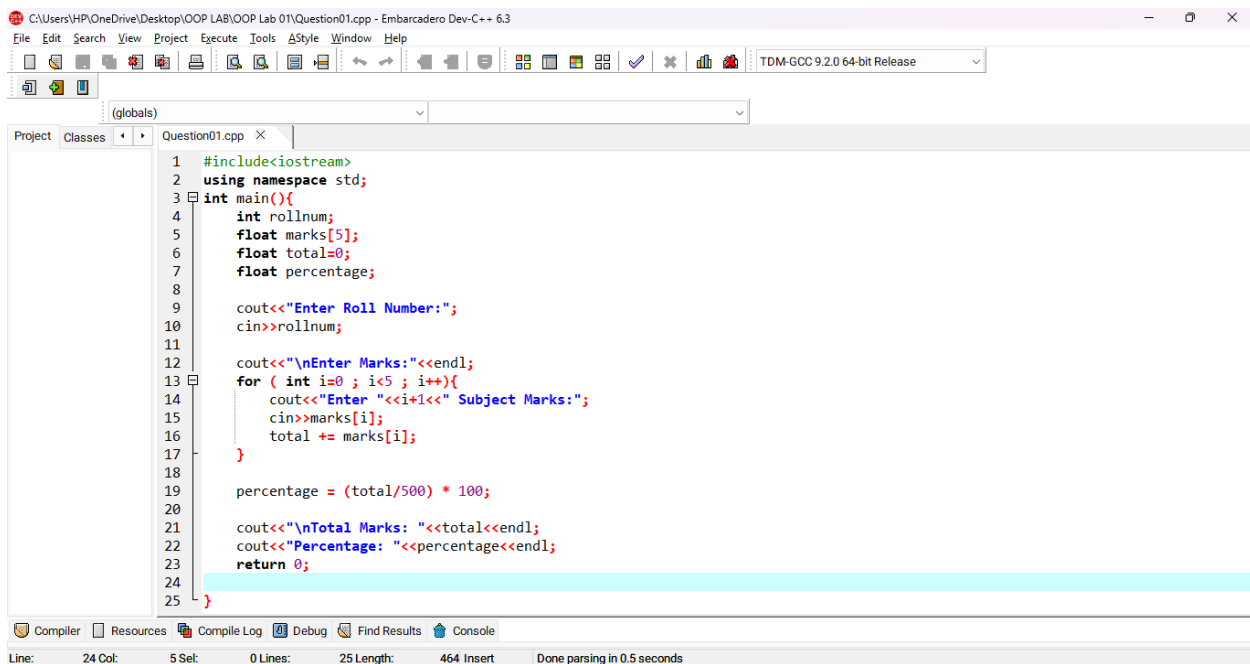
---

Academic Year 2025 - 2026

# **LAB 01**

**OBJECT ORIENTED PROGRAMMING (CT-260)**

**Question 01:** Write a program that take input of your roll number along with the marks obtained in five subjects and display the total marks obtained and the percentage.

**Source Code:**

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int rollnum;
5     float marks[5];
6     float total=0;
7     float percentage;
8
9     cout<<"Enter Roll Number:";
10    cin>>rollnum;
11
12    cout<<"\nEnter Marks:"<<endl;
13    for ( int i=0 ; i<5 ; i++){
14        cout<<"Enter "<<i+1<<" Subject Marks:";
15        cin>>marks[i];
16        total += marks[i];
17    }
18
19    percentage = (total/500) * 100;
20
21    cout<<"\nTotal Marks: "<<total<<endl;
22    cout<<"Percentage: "<<percentage<<endl;
23    return 0;
24 }
25
```

Line: 24 Col: 5 Sel: 0 Lines: 25 Length: 464 Insert Done parsing in 0.5 seconds

**Output:**

```
C:\Users\HP\OneDrive\Desktop >
Enter Roll Number:25062

Enter Marks:
Enter 1 Subject Marks:98
Enter 2 Subject Marks:99
Enter 3 Subject Marks:87
Enter 4 Subject Marks:98
Enter 5 Subject Marks:80

Total Marks: 462
Percentage: 92.4

-----
Process exited after 14.1 seconds with return value 0
Press any key to continue . . .
```

**Question 02:** Write a program to swap three numbers entered by a user using pointers.

**Source Code:**

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 01\Question02.cpp - [Executing] - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

(globals)
Project Classes Question01.cpp Question02.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int *num1, *num2, *num3;
5     int a, b, c, temp;
6
7     cout<<"Enter Number 01:";
8     cin>>a;
9
10    cout<<"Enter Number 02:";
11    cin>>b;
12
13    cout<<"Enter Number 03:";
14    cin>>c;
15
16    num1=&a;
17    num2=&b;
18    num3=&c;
19
20    temp = *num1;
21    *num1 = *num2;
22    *num2 = *num3;
23    *num3 = temp;
24
25    cout<<"\nSwapped Numbers:"<<endl;
26    cout<<"Number 01: "<<a<<endl;
27    cout<<"Number 02: "<<b<<endl;
28    cout<<"Number 03: "<<c<<endl;
29    return 0;
30 }
31

Compiler Resources Compile Log Debug Find Results Console
Line: 31 Col: 1 Sel: 0 Lines: 31 Length: 481 Insert Done parsing in 0.016 seconds
```

Output:

```
C:\Users\HP\OneDrive\Desktop >
Enter Number 01:34
Enter Number 02:56
Enter Number 03:78

Swapped Numbers:
Number 01: 56
Number 02: 78
Number 03: 34

-----
Process exited after 3.026 seconds with return value 0
Press any key to continue . . . |
```

**Question 03:** Write a program to convert temp from Fahrenheit to Celsius unit using equation  $C = (F - 32) / 1.8$ .

Source Code:

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 01\Question03.cpp - [Executing] - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

(globals)
Project Classes Question01.cpp Question02.cpp Question03.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     float fahrenheit, celsius;
5     cout << "Enter Temperature (in Fahrenheit): ";
6     cin >> fahrenheit;
7
8     celsius = (fahrenheit - 32) / 1.8;
9
10    cout<<"Temperature in Celsius: "<< celsius << endl;
11    return 0;
12 }
13
14
```

Compiler Resources Compile Log Debug Find Results Console  
Line: 14 Col: 1 Sel: 0 Lines: 14 Length: 269 Insert Done parsing in 0.016 seconds

**Output:**

```
C:\Users\HP\OneDrive\Desktop >
Enter Temperature (in Fahrenheit): 98.6
Temperature in Celsius: 37

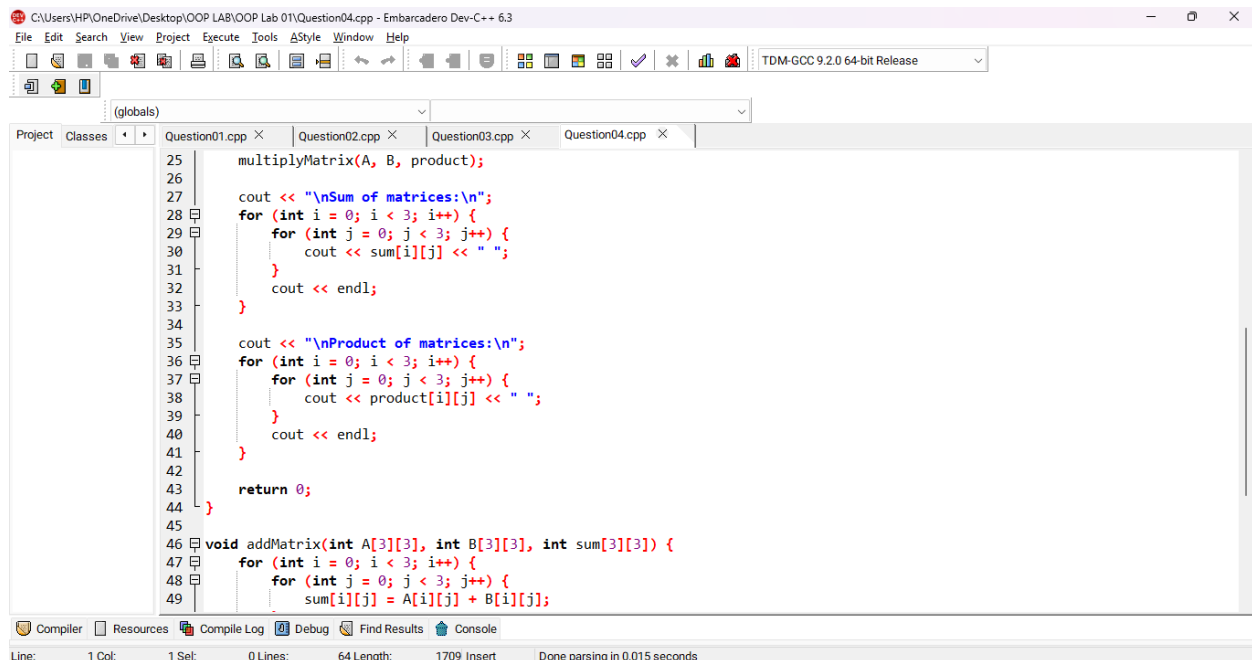
-----
Process exited after 31.71 seconds with return value 0
Press any key to continue . . . |
```

**Question 04:** Using 2-D arrays, write a program that allows the user to input two, 3x3 matrices. Write a function for adding two matrices. Write another function for multiplying the two matrices.

**Source Code:**

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 01\Question04.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

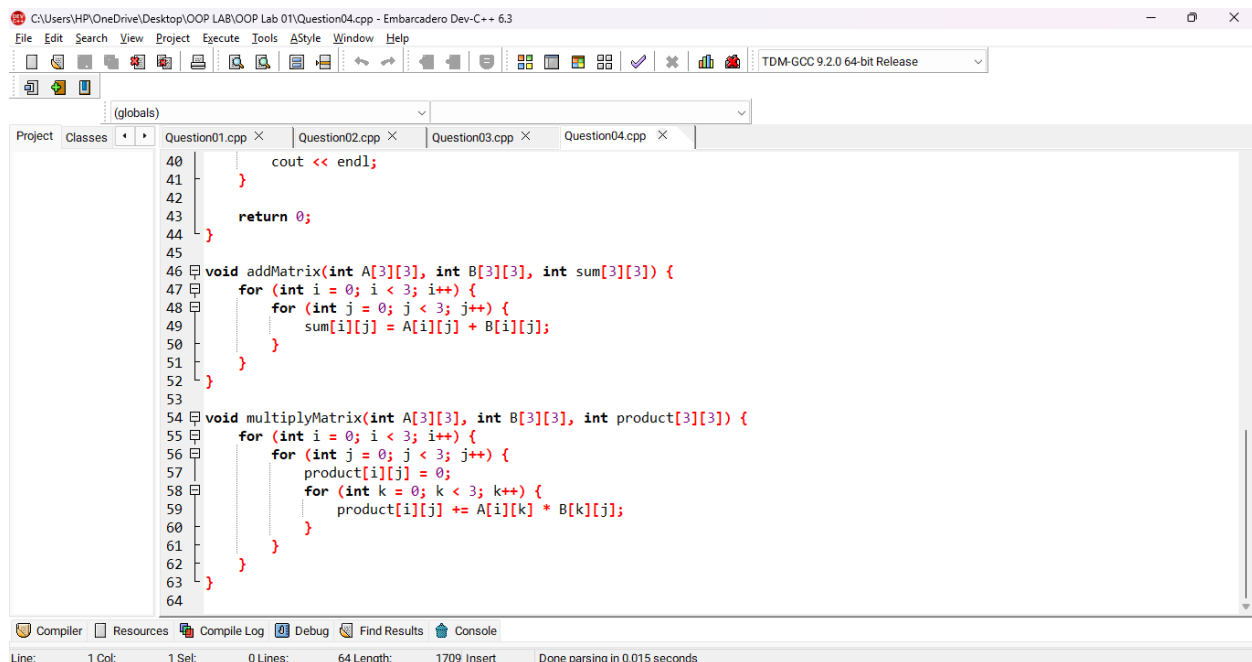
(globals)
Project Classes Question01.cpp Question02.cpp Question03.cpp Question04.cpp
1 #include <iostream>
2 using namespace std;
3 void addMatrix(int A[3][3], int B[3][3], int sum[3][3]);
4 void multiplyMatrix(int A[3][3], int B[3][3], int product[3][3]);
5 int main() {
6     int A[3][3], B[3][3], sum[3][3], product[3][3];
7
8     cout << "Enter elements of first 3x3 matrix:\n";
9     for (int i = 0; i < 3; i++) {
10         for (int j = 0; j < 3; j++) {
11             cout << "Enter element (" << i+1 << ", " << j+1 << "): " << endl;
12             cin >> A[i][j];
13         }
14     }
15
16     cout << "Enter elements of second 3x3 matrix:\n";
17     for (int i = 0; i < 3; i++) {
18         for (int j = 0; j < 3; j++) {
19             cout << "Enter element (" << i+1 << ", " << j+1 << "): " << endl;
20             cin >> B[i][j];
21         }
22     }
23
24     addMatrix(A, B, sum);
25     multiplyMatrix(A, B, product);
26 }
```



The screenshot shows a C++ IDE with the following code in `Question04.cpp`:

```
25 multiplyMatrix(A, B, product);
26
27 cout << "\nSum of matrices:\n";
28 for (int i = 0; i < 3; i++) {
29     for (int j = 0; j < 3; j++) {
30         cout << sum[i][j] << " ";
31     }
32     cout << endl;
33 }
34
35 cout << "\nProduct of matrices:\n";
36 for (int i = 0; i < 3; i++) {
37     for (int j = 0; j < 3; j++) {
38         cout << product[i][j] << " ";
39     }
40     cout << endl;
41 }
42
43 return 0;
44 }
45
46 void addMatrix(int A[3][3], int B[3][3], int sum[3][3]) {
47     for (int i = 0; i < 3; i++) {
48         for (int j = 0; j < 3; j++) {
49             sum[i][j] = A[i][j] + B[i][j];
50         }
51     }
52 }
```

The IDE status bar at the bottom indicates: Line: 1 Col: 1 Sel: 0 Lines: 64 Length: 1709 Insert Done parsing in 0.015 seconds.



The screenshot shows the same C++ IDE with the following code in `Question04.cpp`:

```
40     cout << endl;
41 }
42
43 return 0;
44 }
45
46 void addMatrix(int A[3][3], int B[3][3], int sum[3][3]) {
47     for (int i = 0; i < 3; i++) {
48         for (int j = 0; j < 3; j++) {
49             sum[i][j] = A[i][j] + B[i][j];
50         }
51     }
52 }
53
54 void multiplyMatrix(int A[3][3], int B[3][3], int product[3][3]) {
55     for (int i = 0; i < 3; i++) {
56         for (int j = 0; j < 3; j++) {
57             product[i][j] = 0;
58             for (int k = 0; k < 3; k++) {
59                 product[i][j] += A[i][k] * B[k][j];
60             }
61         }
62     }
63 }
64 }
```

The IDE status bar at the bottom indicates: Line: 1 Col: 1 Sel: 0 Lines: 64 Length: 1709 Insert Done parsing in 0.015 seconds.

Output:

```
C:\Users\HP\OneDrive\Deskt  X  +  v
Enter elements of first 3x3 matrix:
Enter element (1,1):
1
Enter element (1,2):
2
Enter element (1,3):
3
Enter element (2,1):
4
Enter element (2,2):
5
Enter element (2,3):
6
Enter element (3,1):
7
Enter element (3,2):
8
Enter element (3,3):
9
Enter elements of second 3x3 matrix:
Enter element (1,1):
10
Enter element (1,2):
11
Enter element (1,3):
12
Enter element (2,1):
13
Enter element (2,2):
14
```

```
C:\Users\HP\OneDrive\Deskt  X  +  v
Enter element (1,2):
11
Enter element (1,3):
12
Enter element (2,1):
13
Enter element (2,2):
14
Enter element (2,3):
15
Enter element (3,1):
16
Enter element (3,2):
17
Enter element (3,3):
18

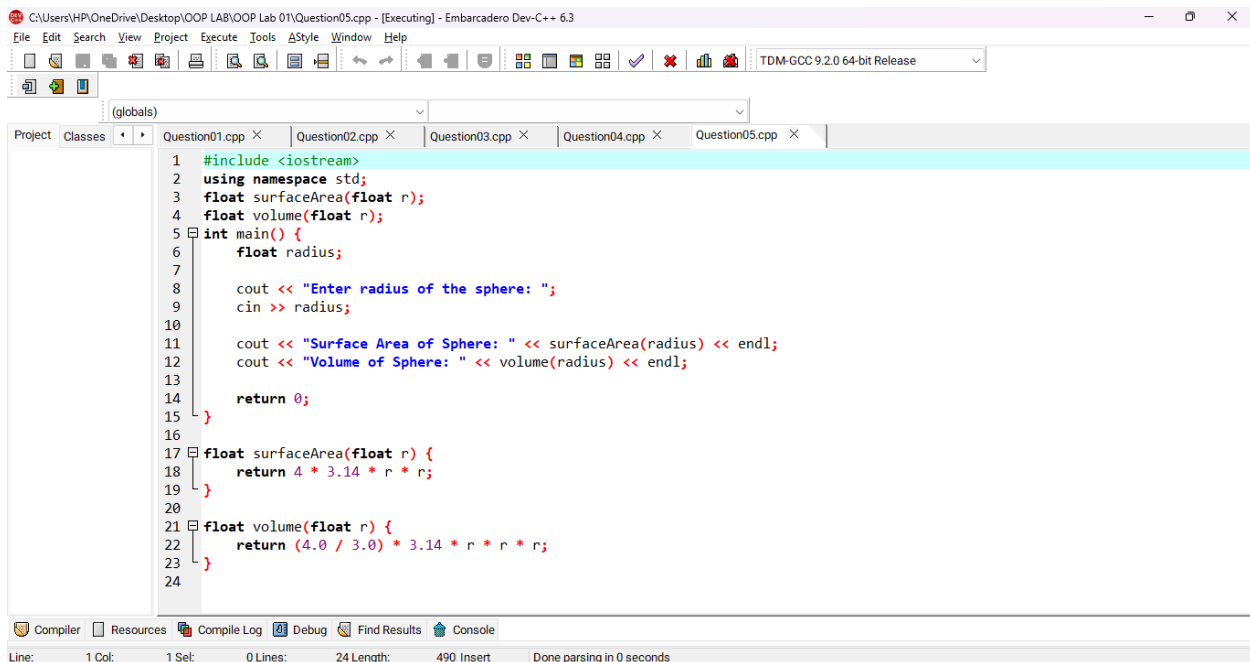
Sum of matrices:
11 13 15
17 19 21
23 25 27

Product of matrices:
84 90 96
201 216 231
318 342 366

-----
Process exited after 30.5 seconds with return value 0
Press any key to continue . . . |
```

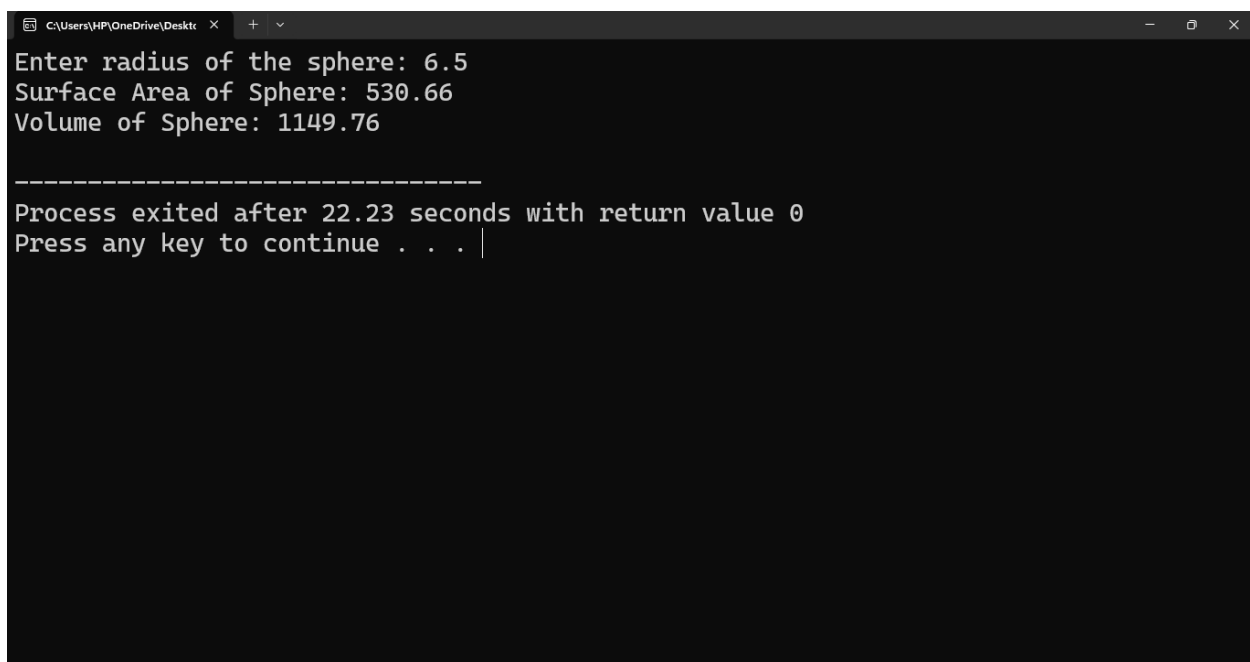
**Question 05:** Write a program to find Surface area and volume of a sphere using functions.

**Source Code:**



```
1 #include <iostream>
2 using namespace std;
3 float surfaceArea(float r);
4 float volume(float r);
5 int main() {
6     float radius;
7
8     cout << "Enter radius of the sphere: ";
9     cin >> radius;
10
11     cout << "Surface Area of Sphere: " << surfaceArea(radius) << endl;
12     cout << "Volume of Sphere: " << volume(radius) << endl;
13
14     return 0;
15 }
16
17 float surfaceArea(float r) {
18     return 4 * 3.14 * r * r;
19 }
20
21 float volume(float r) {
22     return (4.0 / 3.0) * 3.14 * r * r * r;
23 }
24
```

**Output:**



```
C:\Users\HP\OneDrive\Desktop\ x + v
Enter radius of the sphere: 6.5
Surface Area of Sphere: 530.66
Volume of Sphere: 1149.76

-----
Process exited after 22.23 seconds with return value 0
Press any key to continue . . . |
```

**Question 06:** Write a program to help a bank create its withdrawal system. Your program should allow the user to input their account type. Account types are: savings, current. Following business rules apply when withdrawing from a certain account:

**Savings:**

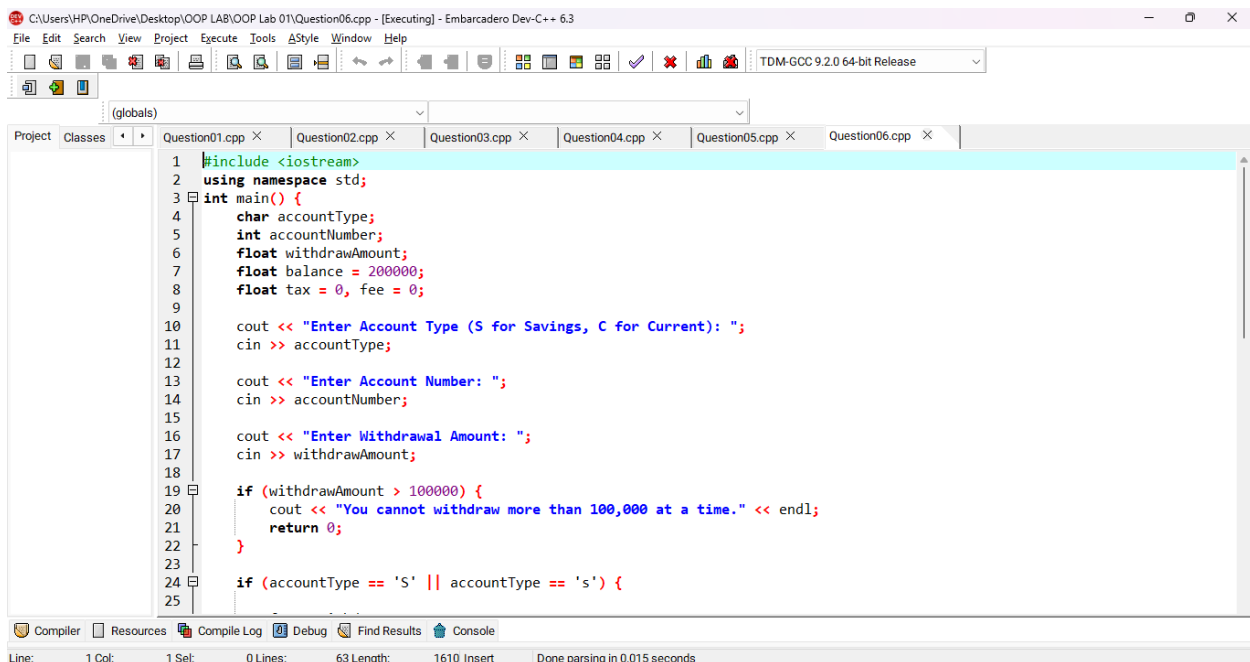
User must provide the savings account number and code 'S' (for savings). When withdrawing from a savings account, users need to pay a set 2% of the money that they withdraw. If the amount of money withdrawn is over 50,000, then a 5% tax will be deducted. The money deducted shall be from the remaining money in the account.

**Current:**

User must provide the current account number and code „C" (for current). When withdrawing from a current account, users need to pay a withdrawal fee of 100. If the amount of money withdrawn is over 50,000, then a 5% tax will be deducted. The money deducted shall be from the remaining money in the account.

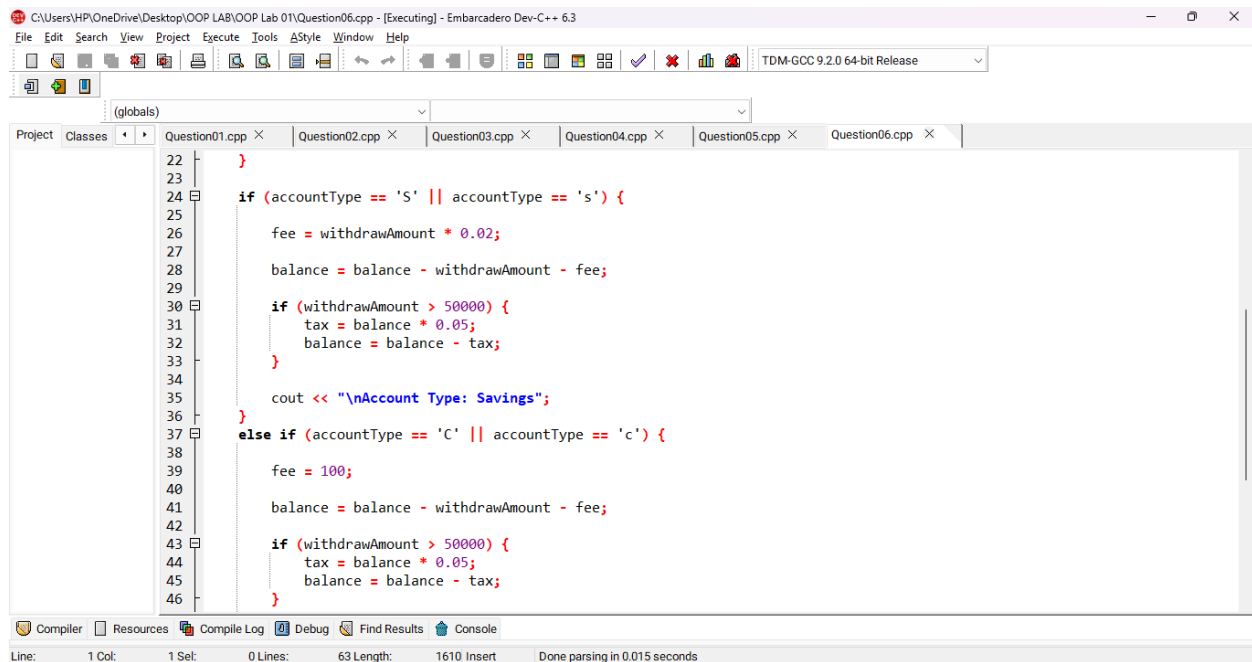
Assume all users have the 200,000 in their accounts, and cannot withdraw more than 100,000 at a time.

**Source Code:**



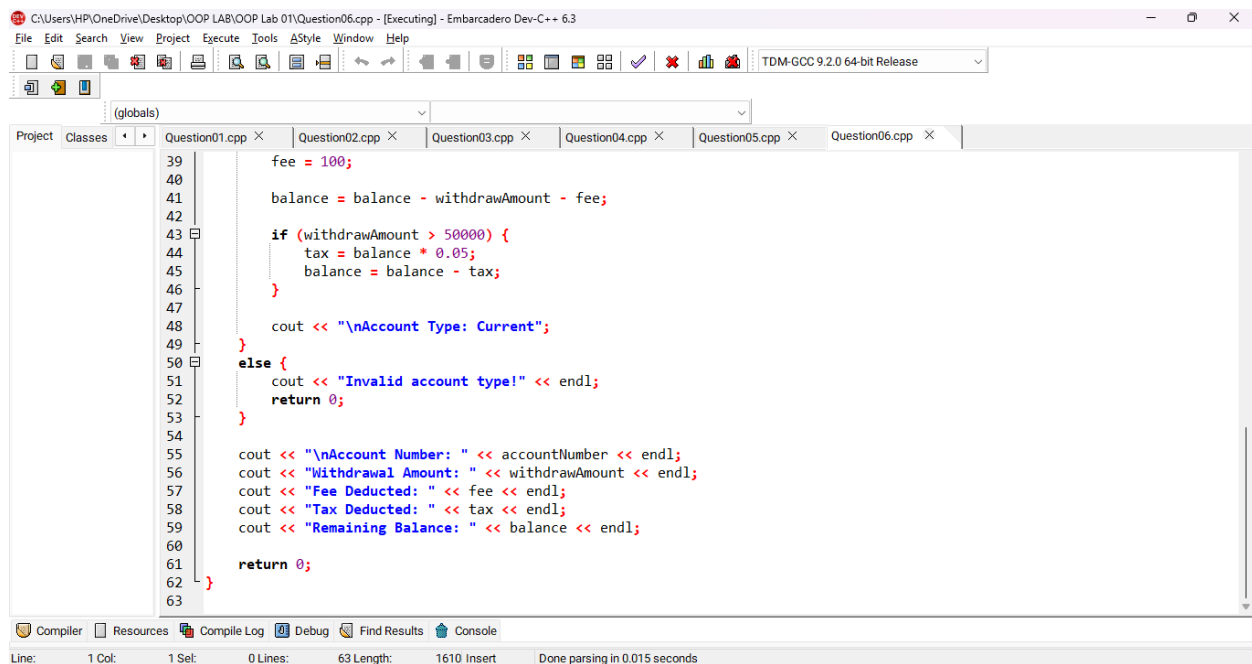
```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     char accountType;
5     int accountNumber;
6     float withdrawAmount;
7     float balance = 200000;
8     float tax = 0, fee = 0;
9
10    cout << "Enter Account Type (S for Savings, C for Current): ";
11    cin >> accountType;
12
13    cout << "Enter Account Number: ";
14    cin >> accountNumber;
15
16    cout << "Enter Withdrawal Amount: ";
17    cin >> withdrawAmount;
18
19    if (withdrawAmount > 100000) {
20        cout << "You cannot withdraw more than 100,000 at a time." << endl;
21        return 0;
22    }
23
24    if (accountType == 'S' || accountType == 's') {
```

The screenshot shows a C++ program in an IDE. The code defines a main function that prompts the user for account type (S for Savings, C for Current), account number, and withdrawal amount. It includes logic to check if the withdrawal amount exceeds 100,000, which would result in an error message and program termination. The program is currently executing, as indicated by the status bar at the bottom.



The screenshot shows a C++ IDE with the file `Question06.cpp` open. The code implements a withdrawal function for a Savings account type. It checks if the account type is 'S' or 's'. If so, it calculates a fee (0.02 \* withdrawAmount) and a tax (0.05 \* balance) if the withdrawAmount is greater than 50000. It then updates the balance by subtracting the withdrawAmount, fee, and tax. Finally, it prints the account type as "Savings".

```
22     }
23
24     if (accountType == 'S' || accountType == 's') {
25
26         fee = withdrawAmount * 0.02;
27
28         balance = balance - withdrawAmount - fee;
29
30         if (withdrawAmount > 50000) {
31             tax = balance * 0.05;
32             balance = balance - tax;
33         }
34
35         cout << "\nAccount Type: Savings";
36     }
37     else if (accountType == 'C' || accountType == 'c') {
38
39         fee = 100;
40
41         balance = balance - withdrawAmount - fee;
42
43         if (withdrawAmount > 50000) {
44             tax = balance * 0.05;
45             balance = balance - tax;
46         }
47     }
48 }
```



The screenshot shows the same C++ IDE with `Question06.cpp` open. The code continues the implementation of the withdrawal function for a Current account type. It checks if the account type is 'C' or 'c'. If so, it calculates a fee (100) and a tax (0.05 \* balance) if the withdrawAmount is greater than 50000. It then updates the balance by subtracting the withdrawAmount, fee, and tax. Finally, it prints the account type as "Current". If the account type is invalid, it prints "Invalid account type!" and returns 0. The code also prints the account number, withdrawal amount, fee deducted, tax deducted, and remaining balance.

```
39     fee = 100;
40
41     balance = balance - withdrawAmount - fee;
42
43     if (withdrawAmount > 50000) {
44         tax = balance * 0.05;
45         balance = balance - tax;
46     }
47
48     cout << "\nAccount Type: Current";
49 }
50 else {
51     cout << "Invalid account type!" << endl;
52     return 0;
53 }
54
55 cout << "\nAccount Number: " << accountNumber << endl;
56 cout << "Withdrawal Amount: " << withdrawAmount << endl;
57 cout << "Fee Deducted: " << fee << endl;
58 cout << "Tax Deducted: " << tax << endl;
59 cout << "Remaining Balance: " << balance << endl;
60
61 return 0;
62 }
63 }
```

Output:

```
C:\Users\HP\OneDrive\Desktop X + v
Enter Account Type (S for Savings, C for Current): S
Enter Account Number: 56098
Enter Withdrawal Amount: 67000

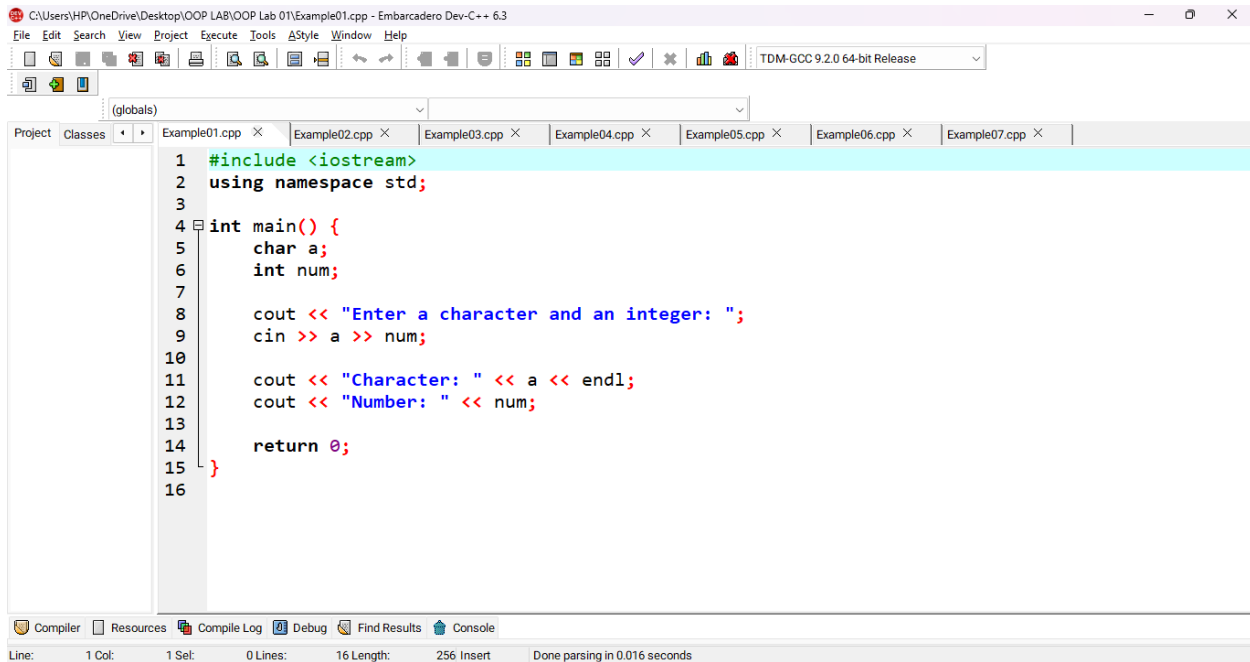
Account Type: Savings
Account Number: 56098
Withdrawal Amount: 67000
Fee Deducted: 1340
Tax Deducted: 6583
Remaining Balance: 125077

-----
Process exited after 11.06 seconds with return value 0
Press any key to continue . . . |
```

```
C:\Users\HP\OneDrive\Desktop X + v
Enter Account Type (S for Savings, C for Current): C
Enter Account Number: 567306
Enter Withdrawal Amount: 56000

Account Type: Current
Account Number: 567306
Withdrawal Amount: 56000
Fee Deducted: 100
Tax Deducted: 7195
Remaining Balance: 136705

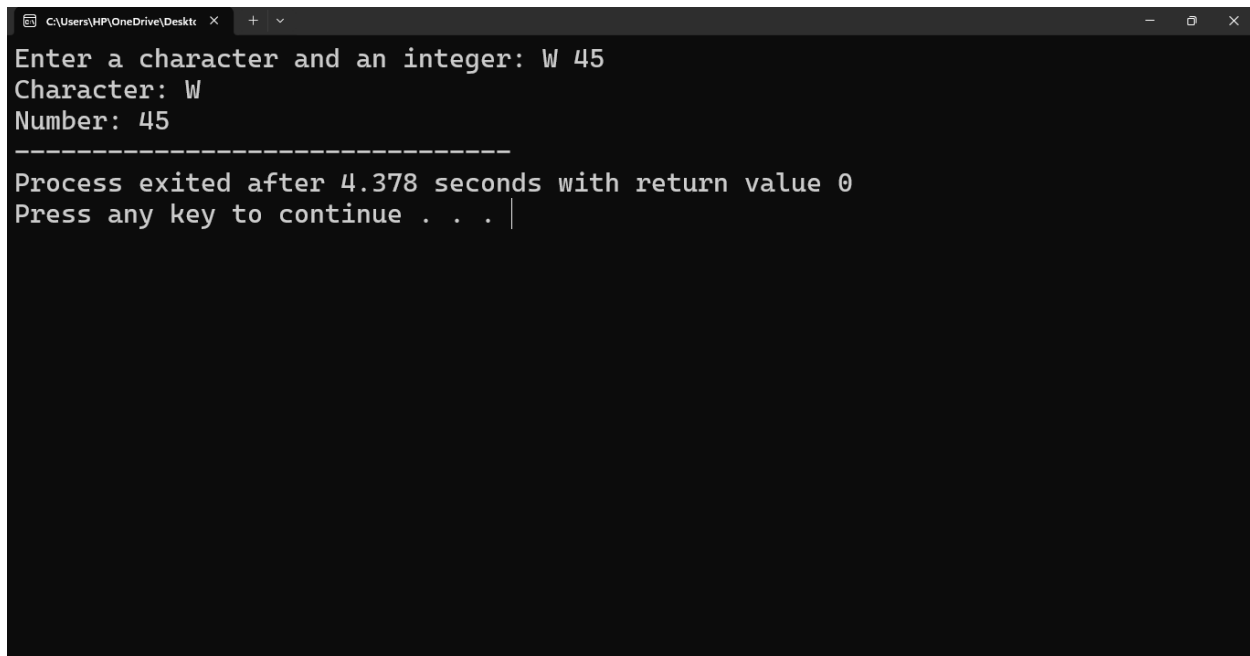
-----
Process exited after 12.7 seconds with return value 0
Press any key to continue . . . |
```

**Example 01:****Source Code:**

The screenshot shows the Embarcadero Dev-C++ IDE with the file 'Example01.cpp' open. The code is as follows:

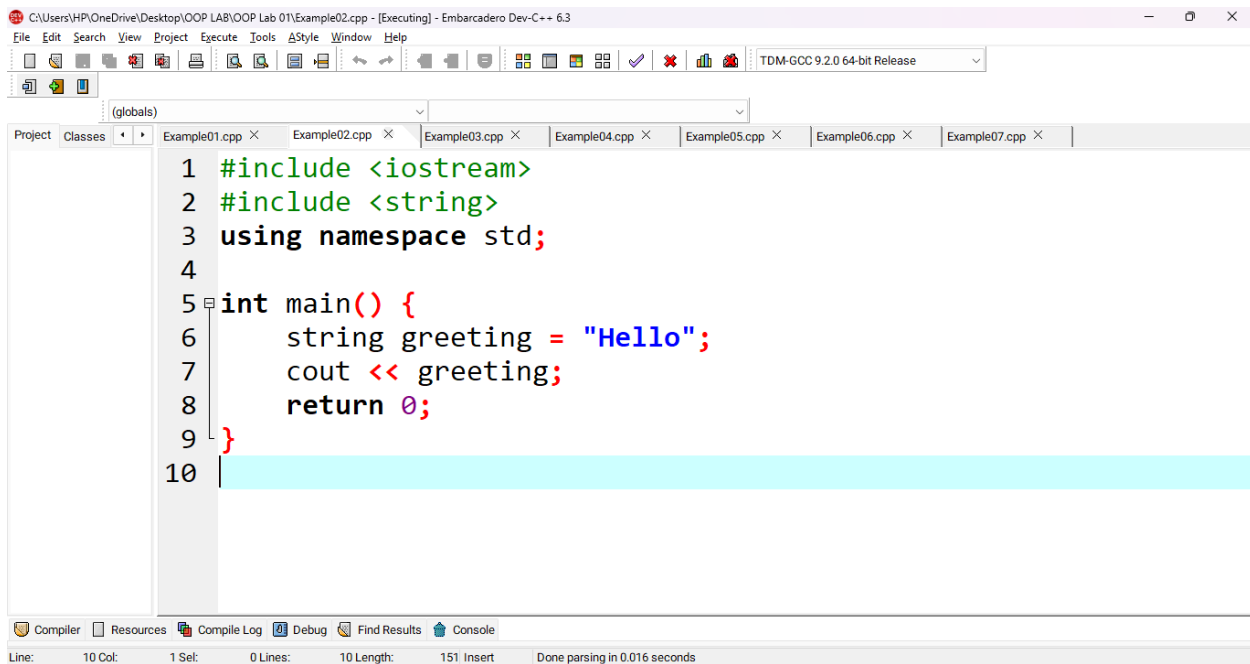
```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     char a;
6     int num;
7
8     cout << "Enter a character and an integer: ";
9     cin >> a >> num;
10
11     cout << "Character: " << a << endl;
12     cout << "Number: " << num;
13
14     return 0;
15 }
16
```

The IDE interface includes a menu bar (File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, Help), a toolbar, a project explorer on the left, and a status bar at the bottom showing compilation details.

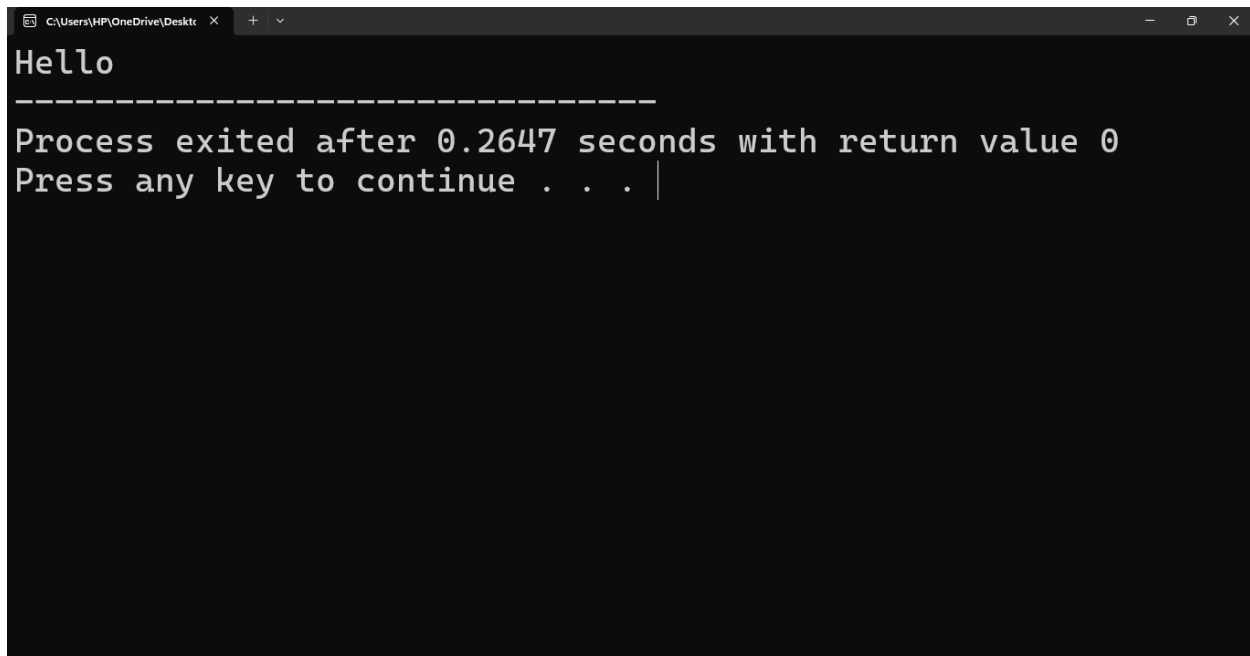
**Output:**

The screenshot shows a terminal window with the following output:

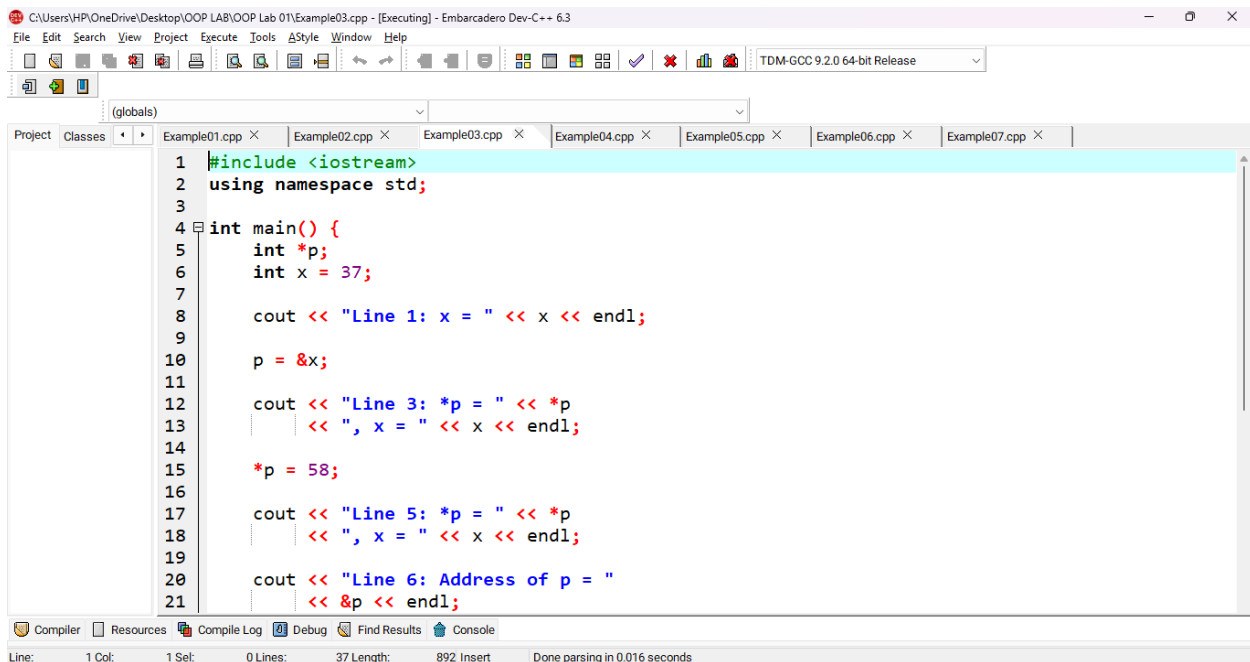
```
Enter a character and an integer: W 45
Character: W
Number: 45
-----
Process exited after 4.378 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:****Source Code:**

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     string greeting = "Hello";
7     cout << greeting;
8     return 0;
9 }
10
```

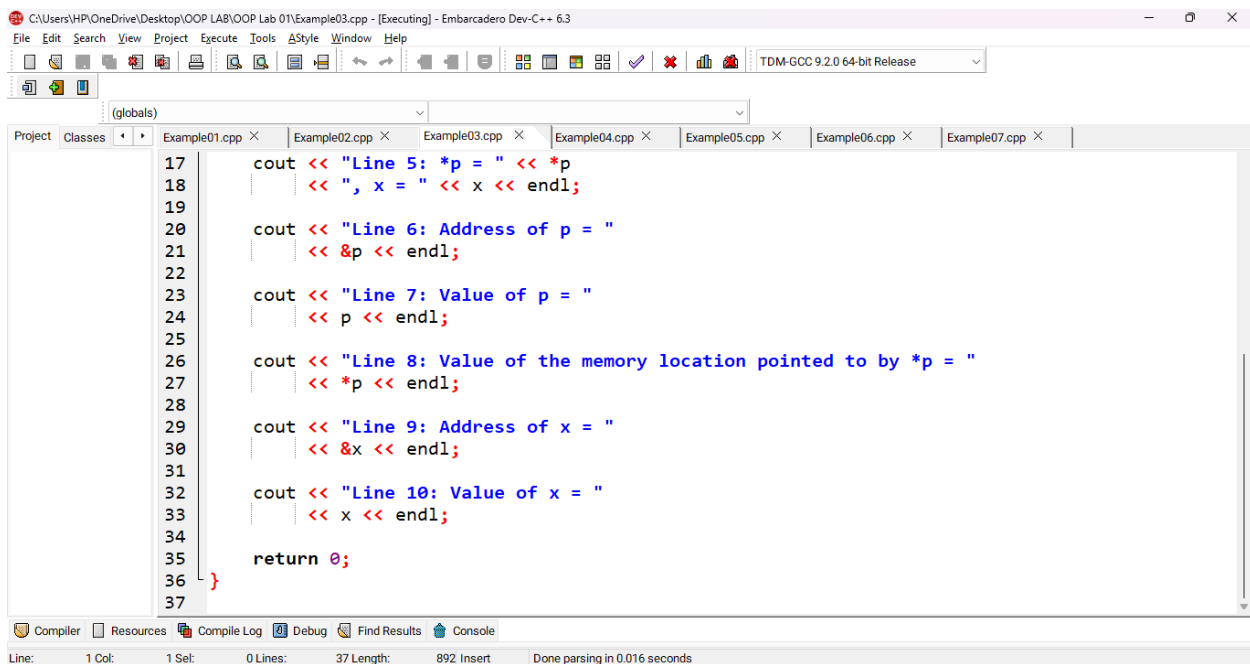
**Output:**

```
Hello
-----
Process exited after 0.2647 seconds with return value 0
Press any key to continue . . . |
```

**Example 03:****Source Code:**

The screenshot shows the first 21 lines of the C++ source code in the Example03.cpp file. The code includes the `<iostream>` header and uses the `std` namespace. The `main` function starts by declaring an integer pointer `p` and an integer `x` with the value 37. It then prints "Line 1: x = " followed by the value of `x`. Next, it assigns the address of `x` to `p` and prints "Line 3: \*p = " followed by the value of `*p` and "x = " followed by the value of `x`. Then, it assigns the value 58 to `*p` and prints "Line 5: \*p = " followed by the value of `*p` and "x = " followed by the value of `x`. Finally, it prints "Line 6: Address of p = " followed by the address of `p`.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int *p;
6     int x = 37;
7
8     cout << "Line 1: x = " << x << endl;
9
10    p = &x;
11
12    cout << "Line 3: *p = " << *p
13         << ", x = " << x << endl;
14
15    *p = 58;
16
17    cout << "Line 5: *p = " << *p
18         << ", x = " << x << endl;
19
20    cout << "Line 6: Address of p = "
21         << &p << endl;
```



The screenshot shows the continuation of the C++ source code, starting from line 17. It prints "Line 7: Value of p = " followed by the value of `p`. Then, it prints "Line 8: Value of the memory location pointed to by \*p = " followed by the value of `*p`. Next, it prints "Line 9: Address of x = " followed by the address of `x`. Then, it prints "Line 10: Value of x = " followed by the value of `x`. Finally, it returns 0.

```
17    cout << "Line 7: Value of p = "
18         << p << endl;
19
20    cout << "Line 8: Value of the memory location pointed to by *p = "
21         << *p << endl;
22
23    cout << "Line 9: Address of x = "
24         << &x << endl;
25
26    cout << "Line 10: Value of x = "
27         << x << endl;
28
29    return 0;
30 }
```

**Output:**

```
C:\Users\HP\OneDrive\Desktop >
Line 1: x = 37
Line 3: *p = 37, x = 37
Line 5: *p = 58, x = 58
Line 6: Address of p = 0x78fe48
Line 7: Value of p = 0x78fe44
Line 8: Value of the memory location pointed to by *p = 58
Line 9: Address of x = 0x78fe44
Line 10: Value of x = 58

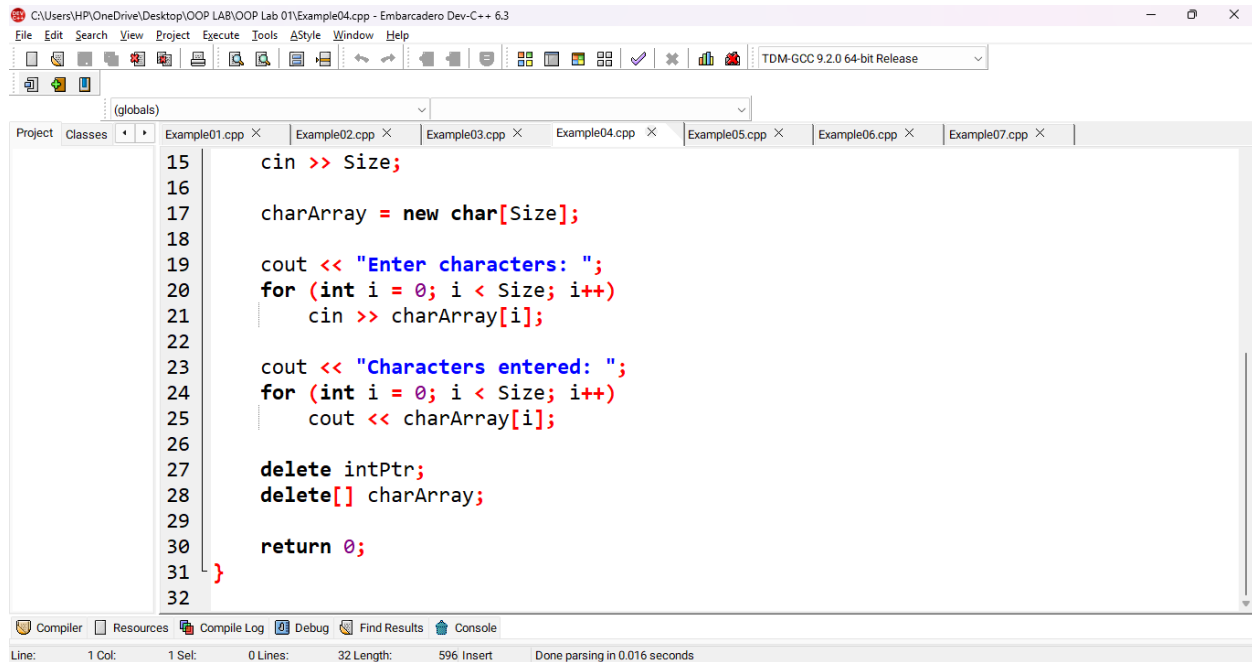
-----
Process exited after 0.265 seconds with return value 0
Press any key to continue . . .
```

**Example 04:****Source Code:**

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 01\Example04.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

(globals)
Project Classes Example01.cpp Example02.cpp Example03.cpp Example04.cpp Example05.cpp Example06.cpp Example07.cpp
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int* intPtr;
6     char* charArray;
7     int Size;
8
9     intPtr = new int;
10
11     cout << "Enter an Integer Value: ";
12     cin >> *intPtr;
13
14     cout << "Enter the size of the Character Array: ";
15     cin >> Size;
16
17     charArray = new char[Size];
18
19 }
```

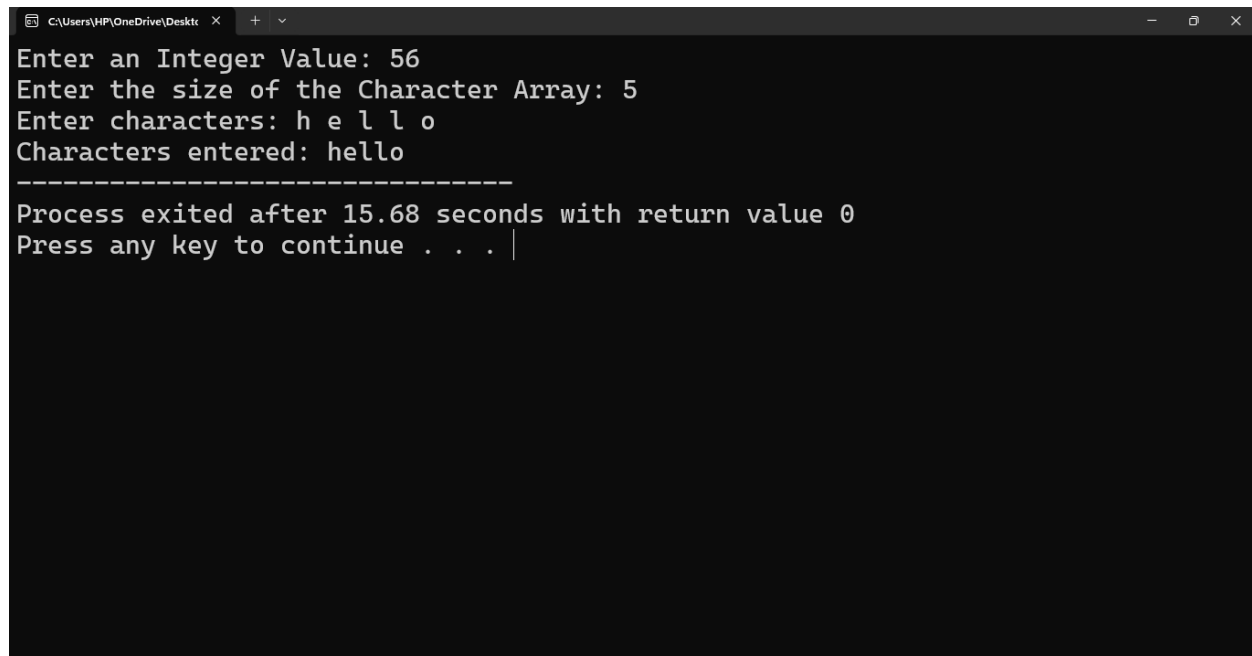
Compiler Resources Compile Log Debug Find Results Console  
Line: 1 Col: 1 Sel: 0 Lines: 32 Length: 596 Insert Done parsing in 0.016 seconds



The screenshot shows a code editor window with the following C++ code:

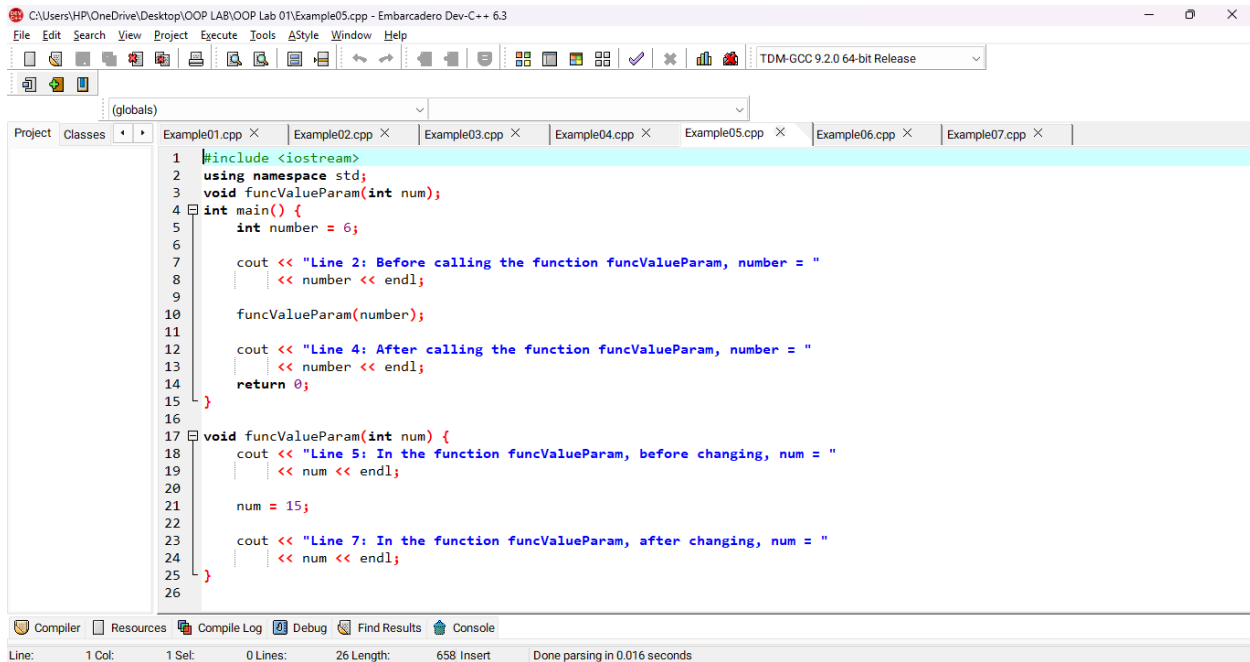
```
15     cin >> Size;  
16  
17     charArray = new char[Size];  
18  
19     cout << "Enter characters: ";  
20     for (int i = 0; i < Size; i++)  
21     |     cin >> charArray[i];  
22  
23     cout << "Characters entered: ";  
24     for (int i = 0; i < Size; i++)  
25     |     cout << charArray[i];  
26  
27     delete intPtr;  
28     delete[] charArray;  
29  
30     return 0;  
31 }  
32
```

The IDE interface includes a menu bar (File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, Help), a toolbar, and a status bar at the bottom showing compilation details: "Line: 1 Col: 1 Sel: 0 Lines: 32 Length: 596 Insert Done parsing in 0.016 seconds".

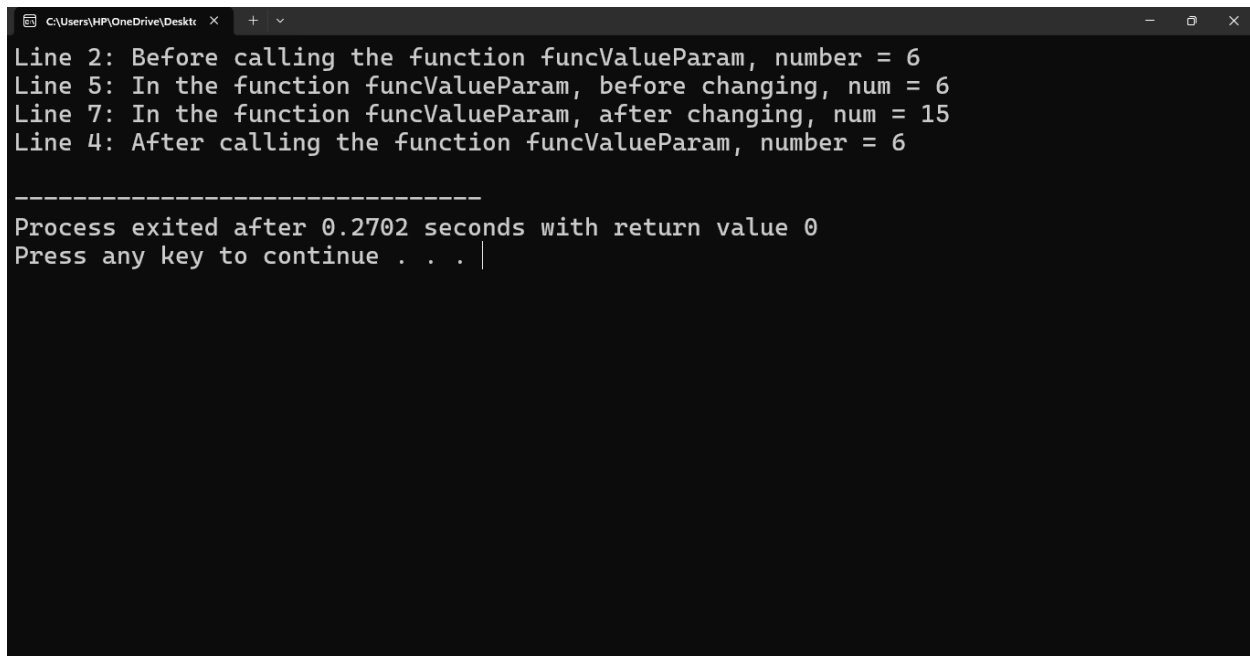
**Output:**

The screenshot shows a terminal window with the following output:

```
Enter an Integer Value: 56  
Enter the size of the Character Array: 5  
Enter characters: h e l l o  
Characters entered: hello  
-----  
Process exited after 15.68 seconds with return value 0  
Press any key to continue . . . |
```

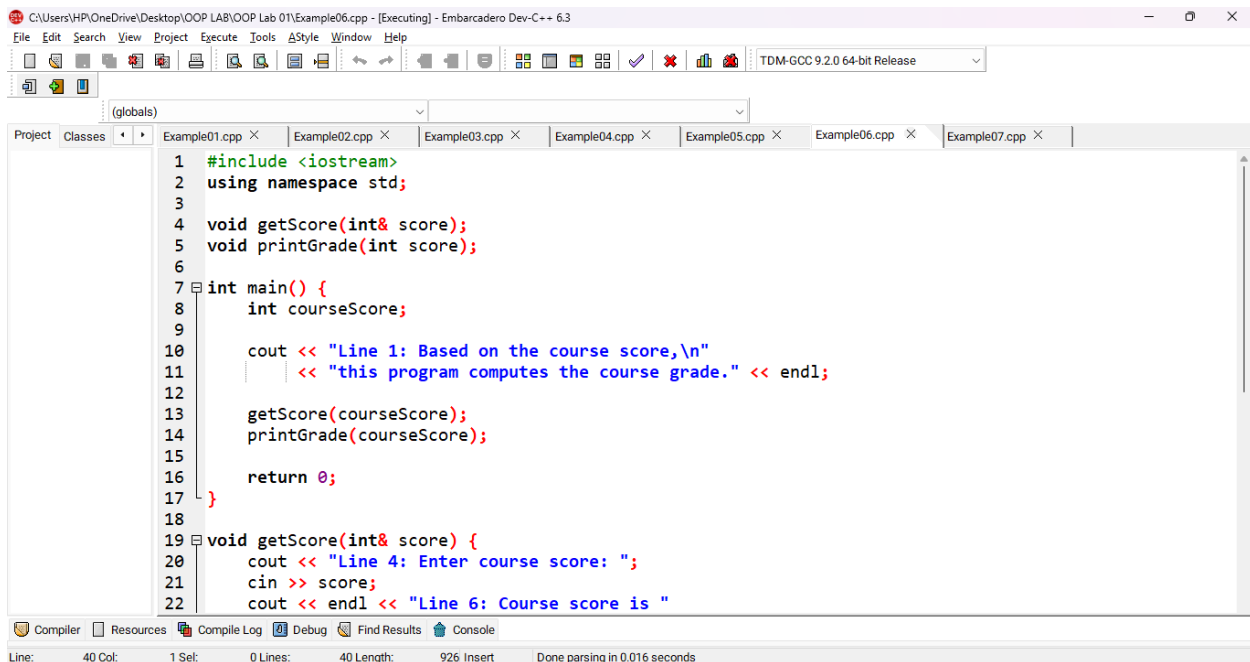
**Example 05:****Source Code:**

```
1 #include <iostream>
2 using namespace std;
3 void funcValueParam(int num);
4 int main() {
5     int number = 6;
6
7     cout << "Line 2: Before calling the function funcValueParam, number = "
8         << number << endl;
9
10    funcValueParam(number);
11
12    cout << "Line 4: After calling the function funcValueParam, number = "
13        << number << endl;
14    return 0;
15 }
16
17 void funcValueParam(int num) {
18     cout << "Line 5: In the function funcValueParam, before changing, num = "
19         << num << endl;
20
21     num = 15;
22
23     cout << "Line 7: In the function funcValueParam, after changing, num = "
24         << num << endl;
25 }
26
```

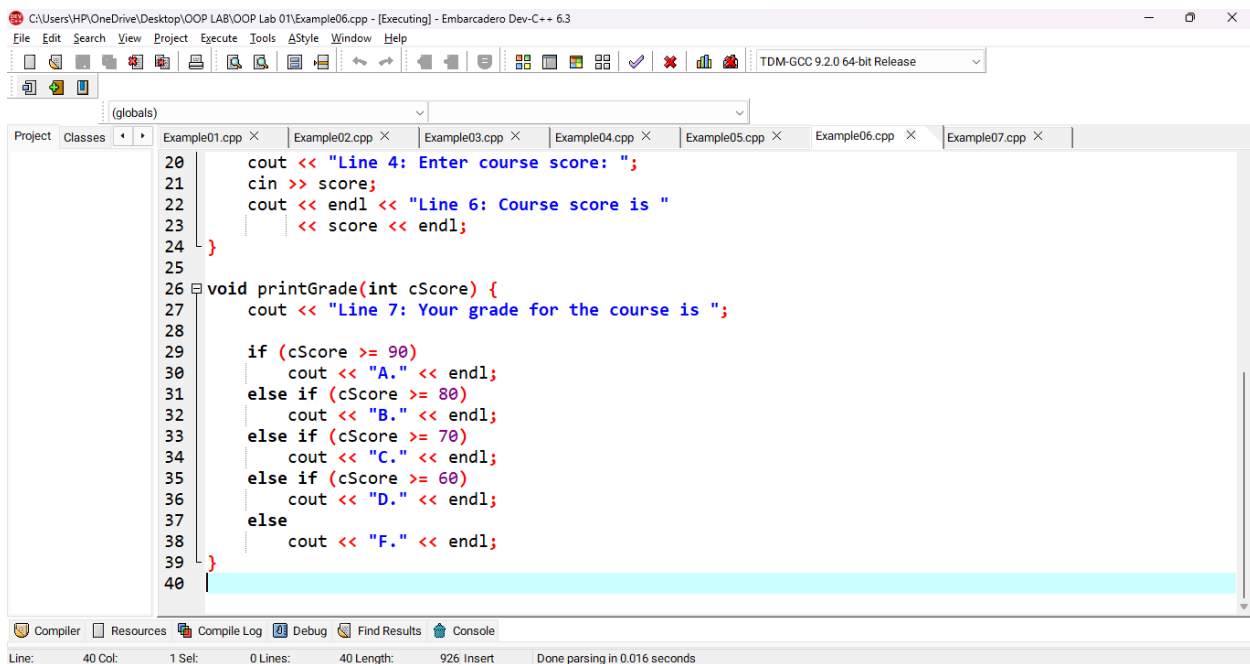
**Output:**

```
C:\Users\HP\OneDrive\Deskt... X + v
Line 2: Before calling the function funcValueParam, number = 6
Line 5: In the function funcValueParam, before changing, num = 6
Line 7: In the function funcValueParam, after changing, num = 15
Line 4: After calling the function funcValueParam, number = 6

-----
Process exited after 0.2702 seconds with return value 0
Press any key to continue . . . |
```

**Example 06:****Source Code:**

```
1 #include <iostream>
2 using namespace std;
3
4 void getScore(int& score);
5 void printGrade(int score);
6
7 int main() {
8     int courseScore;
9
10    cout << "Line 1: Based on the course score,\n"
11    << "this program computes the course grade." << endl;
12
13    getScore(courseScore);
14    printGrade(courseScore);
15
16    return 0;
17 }
18
19 void getScore(int& score) {
20     cout << "Line 4: Enter course score: ";
21     cin >> score;
22     cout << endl << "Line 6: Course score is "
```



```
20     cout << "Line 4: Enter course score: ";
21     cin >> score;
22     cout << endl << "Line 6: Course score is "
23     << score << endl;
24 }
25
26 void printGrade(int cScore) {
27     cout << "Line 7: Your grade for the course is ";
28
29     if (cScore >= 90)
30         cout << "A." << endl;
31     else if (cScore >= 80)
32         cout << "B." << endl;
33     else if (cScore >= 70)
34         cout << "C." << endl;
35     else if (cScore >= 60)
36         cout << "D." << endl;
37     else
38         cout << "F." << endl;
39 }
40
```

**Output:**

```
C:\Users\HP\OneDrive\Desktop > + -
Line 1: Based on the course score,
this program computes the course grade.
Line 4: Enter course score: 96

Line 6: Course score is 96
Line 7: Your grade for the course is A.

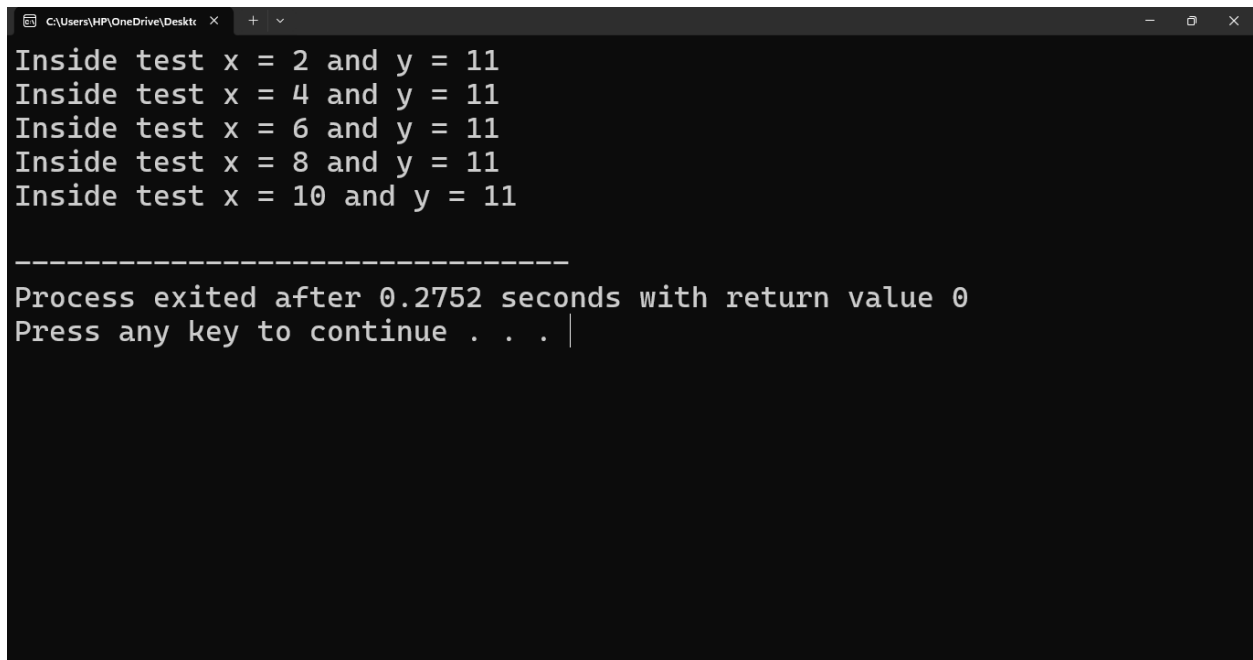
-----
Process exited after 9.836 seconds with return value 0
Press any key to continue . . . |
```

**Example 07:****Source Code:**

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 01\Example07.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

(globals)
Project Classes Example01.cpp Example02.cpp Example03.cpp Example04.cpp Example05.cpp Example06.cpp Example07.cpp
1 #include <iostream>
2 using namespace std;
3
4 void test();
5
6 int main() {
7     int count;
8     for (count = 1; count <= 5; count++)
9         test();
10
11     return 0;
12 }
13
14 void test() {
15     static int x = 0;
16     int y = 10;
17
18     x = x + 2;
19     y = y + 1;
20
21     cout << "Inside test x = " << x
22          << " and y = " << y << endl;
23 }
```

Compiler Resources Compile Log Debug Find Results Console  
Line: 1 Col: 4 Sel: 0 Lines: 24 Length: 352 Insert Done parsing in 0.016 seconds

Output:A screenshot of a Windows command prompt window. The title bar shows the file path 'C:\Users\HP\OneDrive\Desktop' and standard window controls. The output text is as follows:

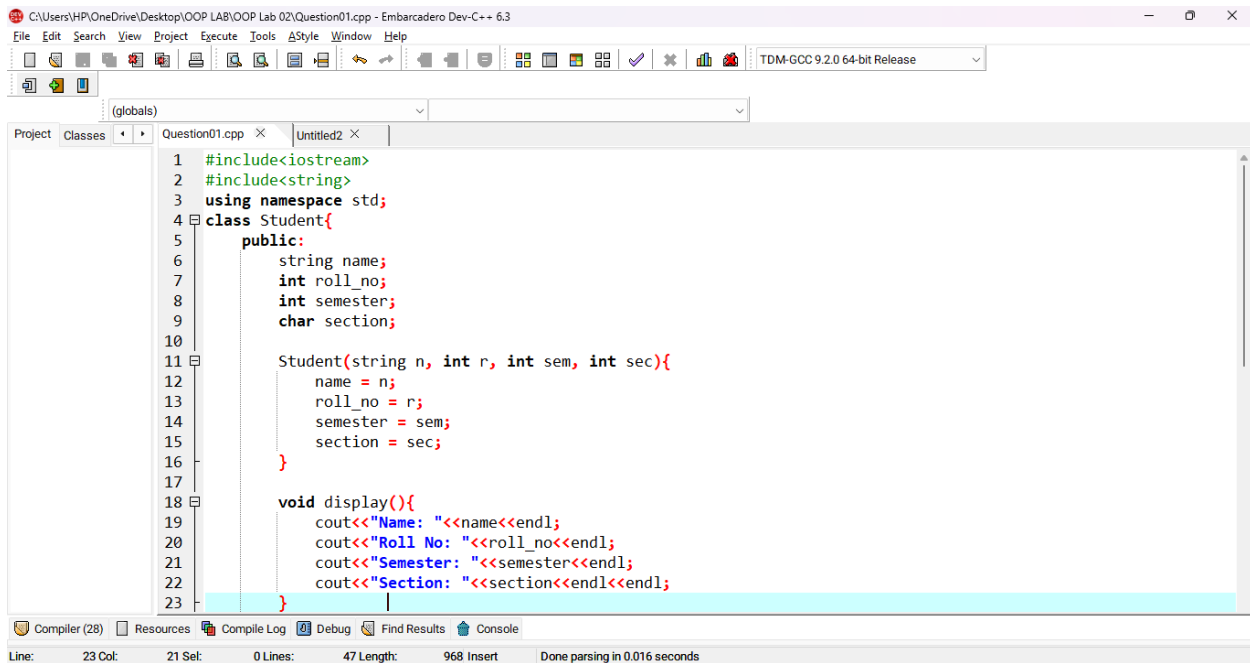
```
Inside test x = 2 and y = 11
Inside test x = 4 and y = 11
Inside test x = 6 and y = 11
Inside test x = 8 and y = 11
Inside test x = 10 and y = 11

-----
Process exited after 0.2752 seconds with return value 0
Press any key to continue . . . |
```

# LAB 02

**OBJECT ORIENTED PROGRAMMING (CT-260)**

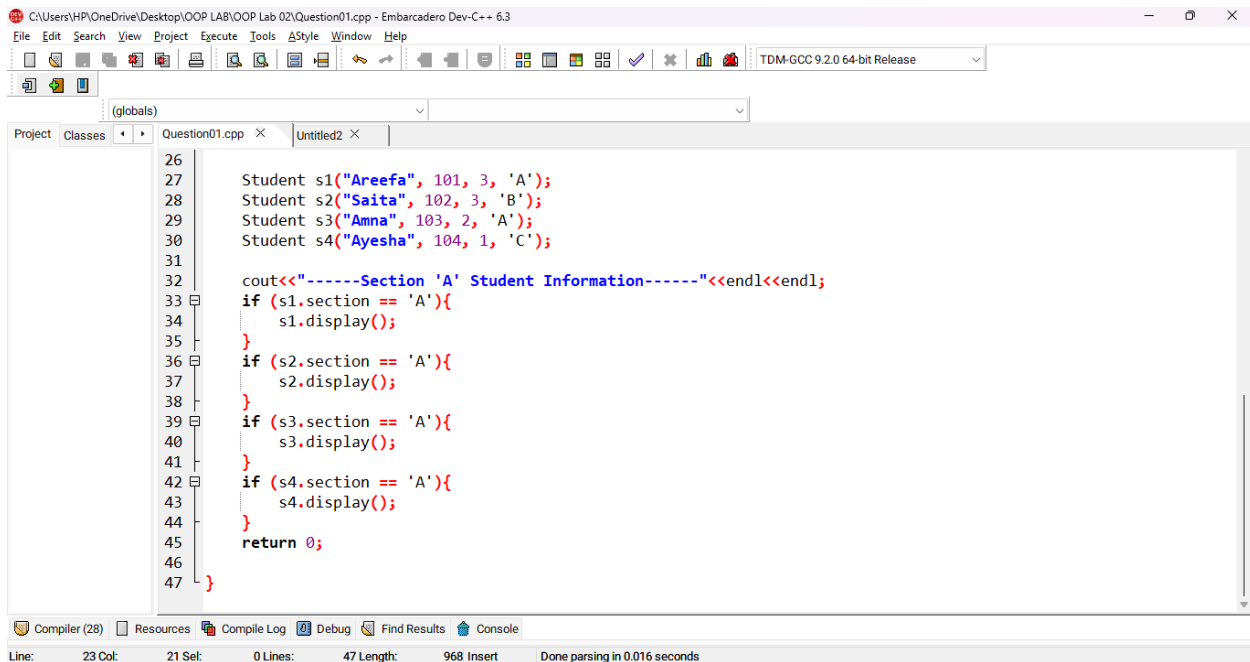
**Question 01:** Write a program in which a class named student has member variables name, roll\_no, semester and section. Create 4 objects of this class to store data of 4 different students, now display data of only those students who belong to section A.

**Source Code:**

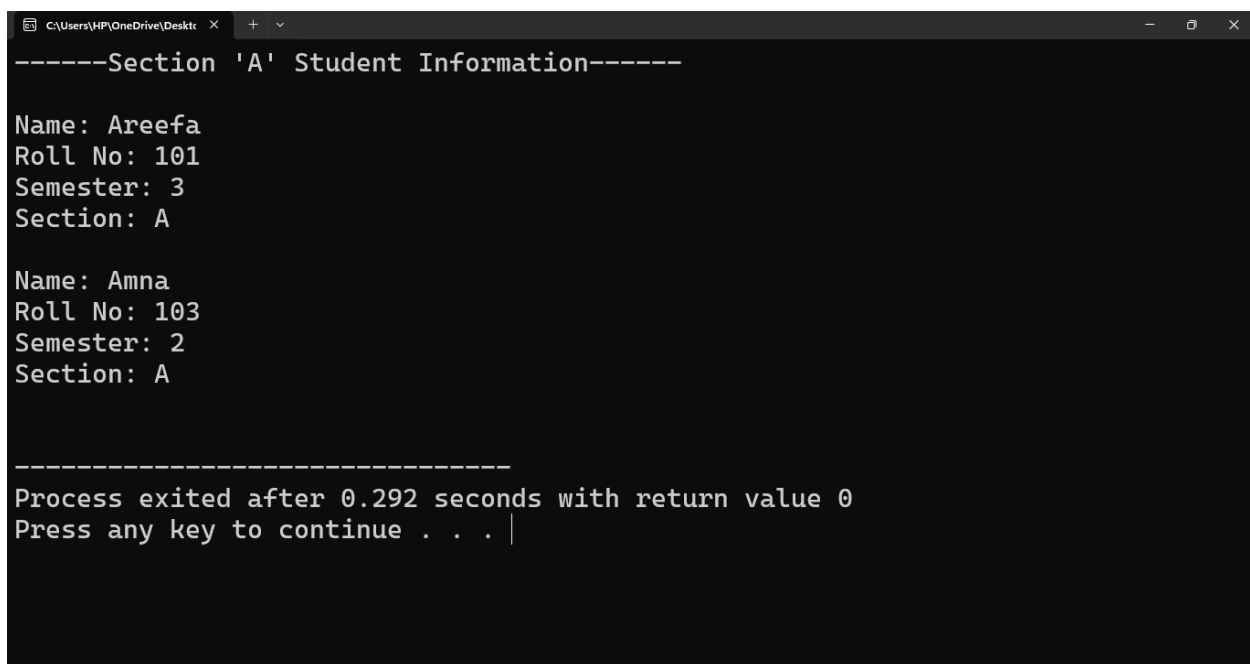
The screenshot shows a code editor window titled "Question01.cpp" in the Embarcadero Dev-C++ 6.3 IDE. The code defines a C++ class named "Student" with the following structure:

```
1 #include<iostream>
2 #include<string>
3 using namespace std;
4 class Student{
5     public:
6         string name;
7         int roll_no;
8         int semester;
9         char section;
10
11     Student(string n, int r, int sem, int sec){
12         name = n;
13         roll_no = r;
14         semester = sem;
15         section = sec;
16     }
17
18     void display(){
19         cout<<"Name: "<<name<<endl;
20         cout<<"Roll No: "<<roll_no<<endl;
21         cout<<"Semester: "<<semester<<endl;
22         cout<<"Section: "<<section<<endl<<endl;
23     }
```

The IDE interface includes a menu bar (File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, Help), a toolbar, and a status bar at the bottom showing compilation statistics: "Line: 23 Col: 21 Set: 0 Lines: 47 Length: 968 Insert Done parsing in 0.016 seconds".



```
26
27 Student s1("Areefa", 101, 3, 'A');
28 Student s2("Saita", 102, 3, 'B');
29 Student s3("Amna", 103, 2, 'A');
30 Student s4("Ayesha", 104, 1, 'C');
31
32 cout<<"-----Section 'A' Student Information-----"<<endl<<endl;
33 if (s1.section == 'A'){
34     s1.display();
35 }
36 if (s2.section == 'A'){
37     s2.display();
38 }
39 if (s3.section == 'A'){
40     s3.display();
41 }
42 if (s4.section == 'A'){
43     s4.display();
44 }
45 return 0;
46
47 }
```

**Output:**

```
-----Section 'A' Student Information-----

Name: Areefa
Roll No: 101
Semester: 3
Section: A

Name: Amna
Roll No: 103
Semester: 2
Section: A

-----
Process exited after 0.292 seconds with return value 0
Press any key to continue . . . |
```

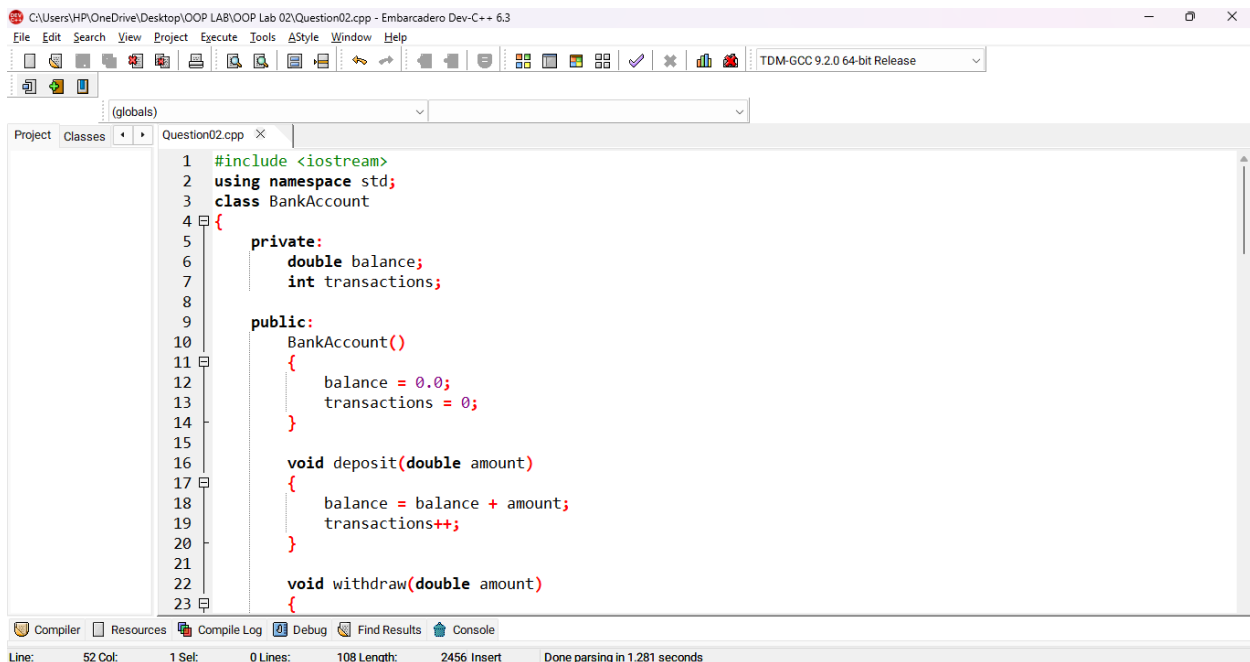
**Question 02:** You are a programmer for the ABC Bank assigned to develop a class that models the basic workings of a bank account. The class should perform the following tasks:

- o Save the account balance.
- o Save the number of transactions performed on the account.
- o Allow deposits to be made to the account.
- o Allow with draws to be taken from the account.
- o Report the current account balance at any time.
- o Report the current number of transactions at any time.

Menu:

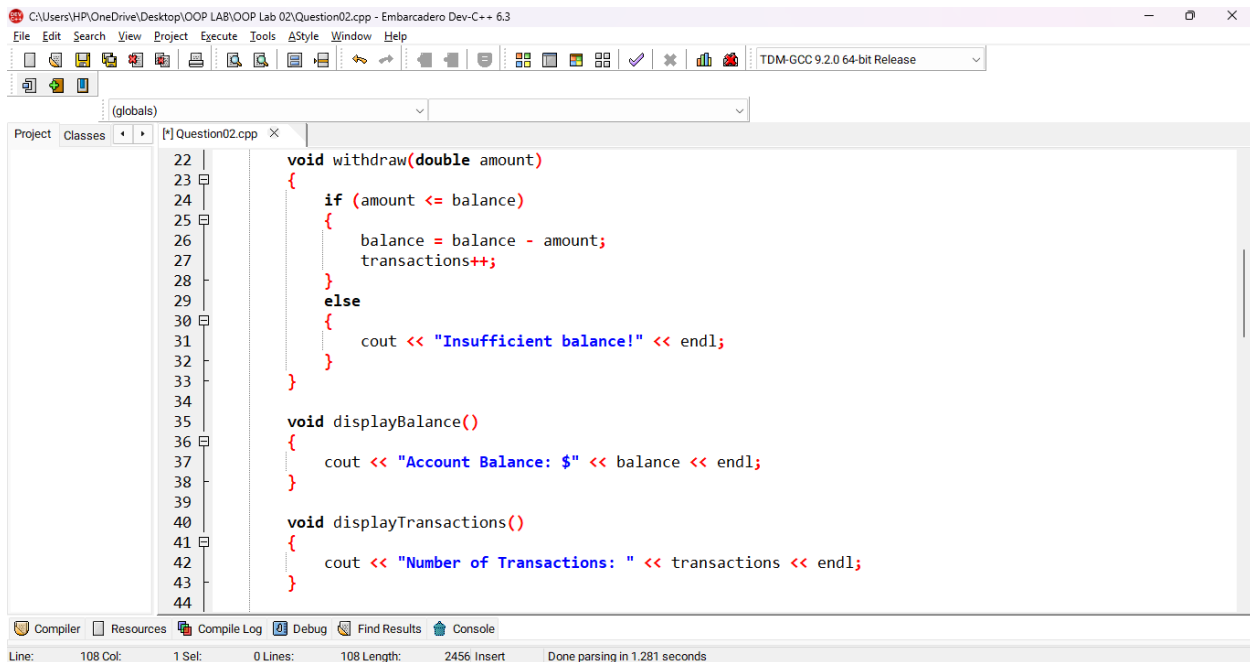
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program

**Source Code:**

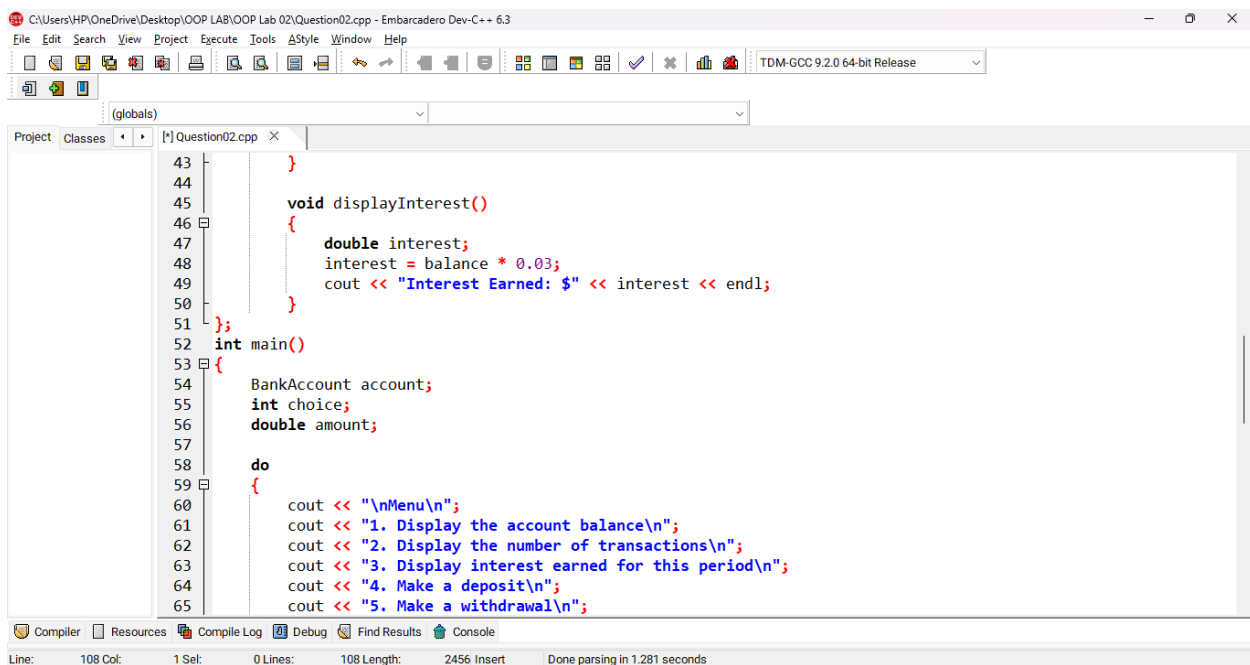


```
1 #include <iostream>
2 using namespace std;
3 class BankAccount
4 {
5     private:
6         double balance;
7         int transactions;
8     public:
9         BankAccount()
10        {
11            balance = 0.0;
12            transactions = 0;
13        }
14
15        void deposit(double amount)
16        {
17            balance = balance + amount;
18            transactions++;
19        }
20
21        void withdraw(double amount)
22        {
23            balance = balance - amount;
24            transactions++;
25        }
26    }
```

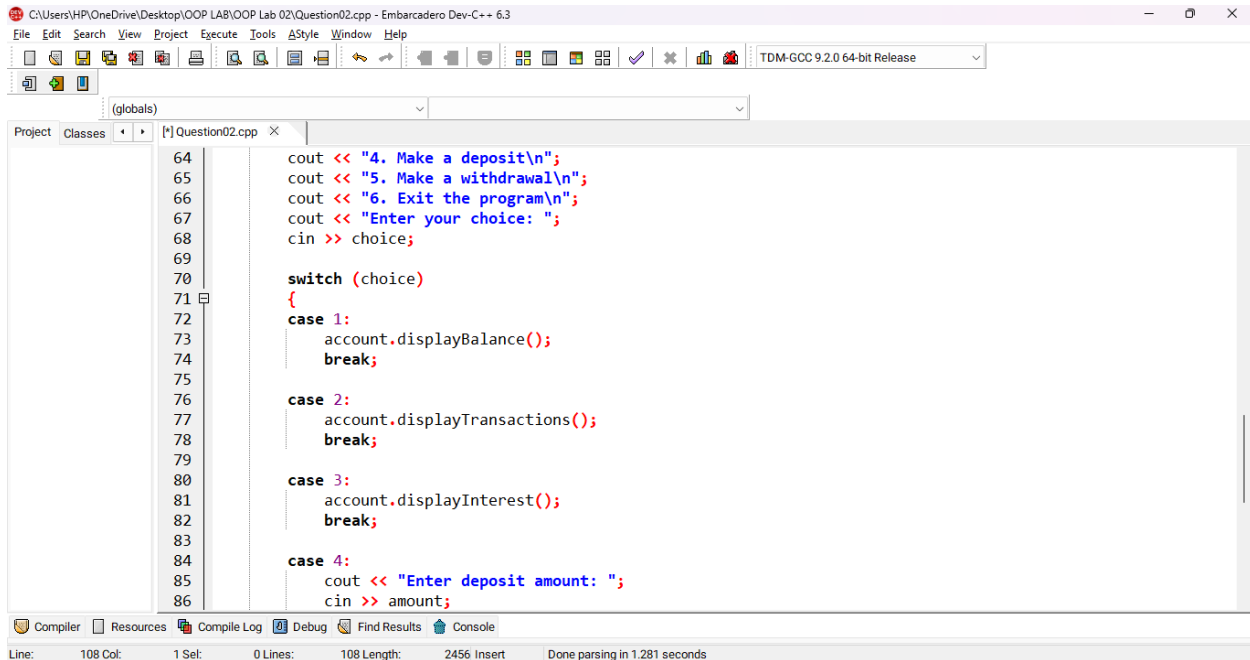
The screenshot shows a C++ IDE with the following details: The title bar indicates the file path is C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Question02.cpp and the compiler is Embarcadero Dev-C++ 6.3. The menu bar includes File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, and Help. The toolbar contains icons for file operations, compilation, and debugging. The compiler is set to TDM-GCC 9.2.0 64-bit Release. The code editor shows the source code for the BankAccount class, which includes private attributes for balance and transactions, and public methods for initialization, deposit, and withdrawal. The status bar at the bottom shows the current line and column (52 Col, 1 Sel), total lines (0 Lines), and other statistics (108 Length, 2456 Insert, Done parsing in 1.281 seconds).



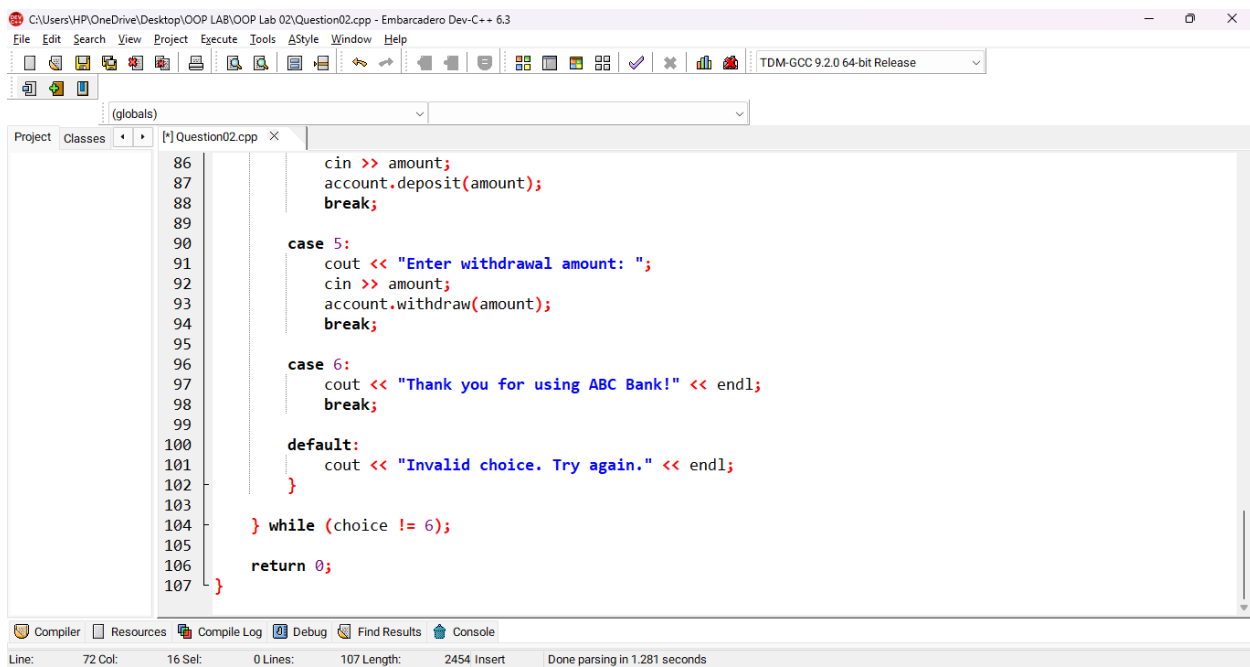
```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Question02.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes [?] Question02.cpp x
22 void withdraw(double amount)
23 {
24     if (amount <= balance)
25     {
26         balance = balance - amount;
27         transactions++;
28     }
29     else
30     {
31         cout << "Insufficient balance!" << endl;
32     }
33 }
34
35 void displayBalance()
36 {
37     cout << "Account Balance: $" << balance << endl;
38 }
39
40 void displayTransactions()
41 {
42     cout << "Number of Transactions: " << transactions << endl;
43 }
44
Compiler Resources Compile Log Debug Find Results Console
Line: 108 Col: 1 Sel: 0 Lines: 108 Length: 2456 Insert Done parsing in 1.281 seconds
```



```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Question02.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes [?] Question02.cpp x
43 }
44
45 void displayInterest()
46 {
47     double interest;
48     interest = balance * 0.03;
49     cout << "Interest Earned: $" << interest << endl;
50 }
51 };
52 int main()
53 {
54     BankAccount account;
55     int choice;
56     double amount;
57
58     do
59     {
60         cout << "\nMenu\n";
61         cout << "1. Display the account balance\n";
62         cout << "2. Display the number of transactions\n";
63         cout << "3. Display interest earned for this period\n";
64         cout << "4. Make a deposit\n";
65         cout << "5. Make a withdrawal\n";
66     }
67 }
68
Compiler Resources Compile Log Debug Find Results Console
Line: 108 Col: 1 Sel: 0 Lines: 108 Length: 2456 Insert Done parsing in 1.281 seconds
```



```
64 cout << "4. Make a deposit\n";
65 cout << "5. Make a withdrawal\n";
66 cout << "6. Exit the program\n";
67 cout << "Enter your choice: ";
68 cin >> choice;
69
70 switch (choice)
71 {
72 case 1:
73     account.displayBalance();
74     break;
75
76 case 2:
77     account.displayTransactions();
78     break;
79
80 case 3:
81     account.displayInterest();
82     break;
83
84 case 4:
85     cout << "Enter deposit amount: ";
86     cin >> amount;
```



```
86 cin >> amount;
87 account.deposit(amount);
88 break;
89
90 case 5:
91     cout << "Enter withdrawal amount: ";
92     cin >> amount;
93     account.withdraw(amount);
94     break;
95
96 case 6:
97     cout << "Thank you for using ABC Bank!" << endl;
98     break;
99
100 default:
101     cout << "Invalid choice. Try again." << endl;
102 }
103
104 } while (choice != 6);
105
106 return 0;
107 }
```

Output:

```
C:\Users\HP\OneDrive\Deskt  X + v - [ ] X
Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 4
Enter deposit amount: 25000

Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 5
Enter withdrawal amount: 4000
```

```
C:\Users\HP\OneDrive\Deskt  X + v - [ ] X
Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 1
Account Balance: $21000

Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 2
Number of Transactions: 2
```

```
C:\Users\HP\OneDrive\Desktop >
Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 3
Interest Earned: $630

Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 6
Thank you for using ABC Bank!
```

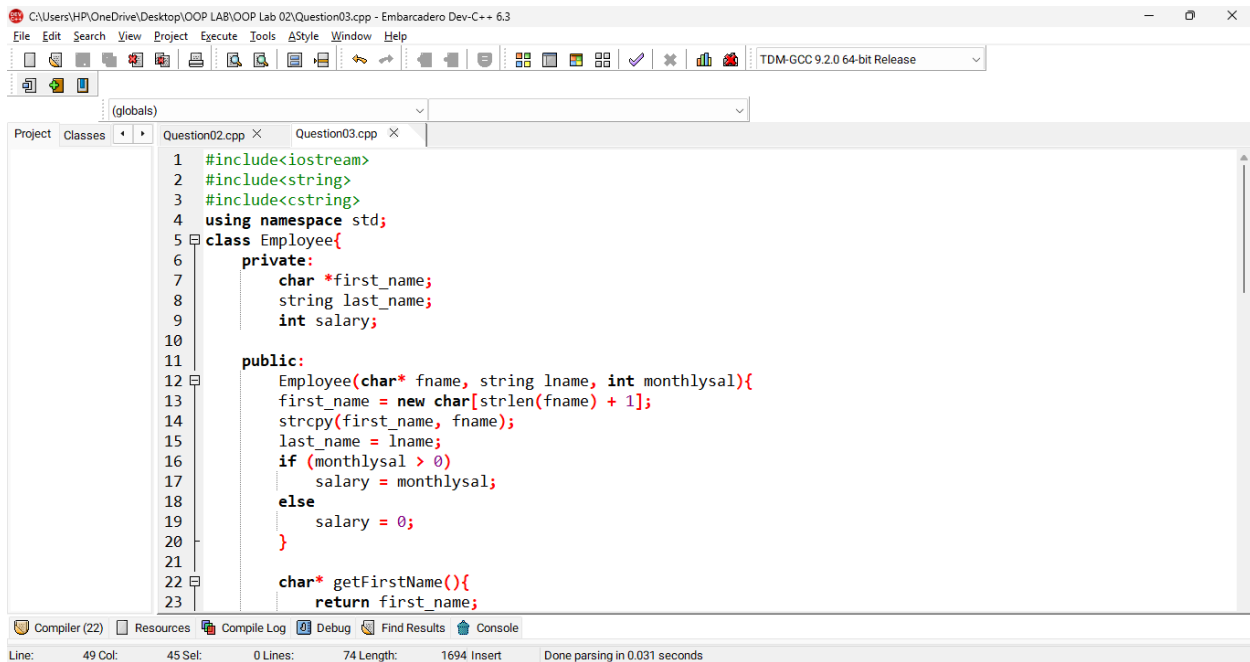
```
C:\Users\HP\OneDrive\Desktop >
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 3
Interest Earned: $630

Menu
1. Display the account balance
2. Display the number of transactions
3. Display interest earned for this period
4. Make a deposit
5. Make a withdrawal
6. Exit the program
Enter your choice: 6
Thank you for using ABC Bank!

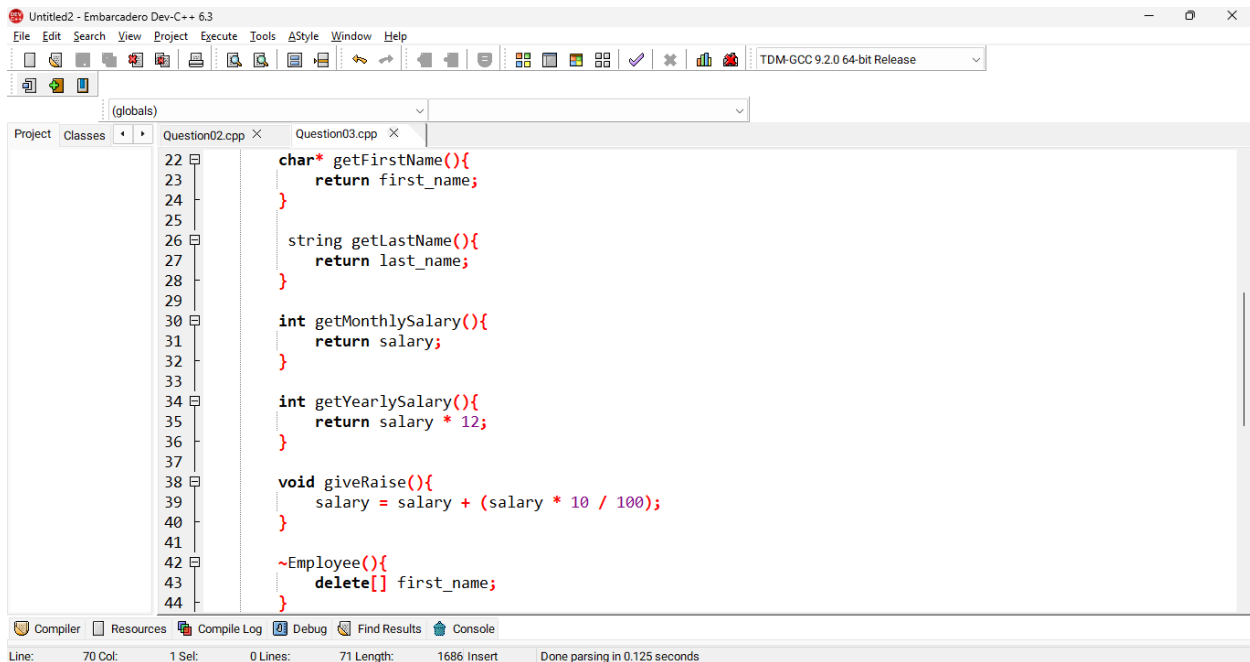
-----
Process exited after 161.1 seconds with return value 0
Press any key to continue . . . |
```

**Question 03:** Create a class called Employee that includes three pieces of information as data members—a first name (type char\* (DMA)), a last name (type string) and a monthly salary (type int). Your class should have a setter function that initializes the three data members. Provide a getter function for each data member. If the monthly salary is not positive, set it to 0. Write a test program that demonstrates class Employee's capabilities. Create two Employee objects and display each object's yearly salary. Then give each Employee a 10 percent raise and display each Employee's yearly salary again. Identify and add any other related functions to achieve the said goal.

### Source Code:



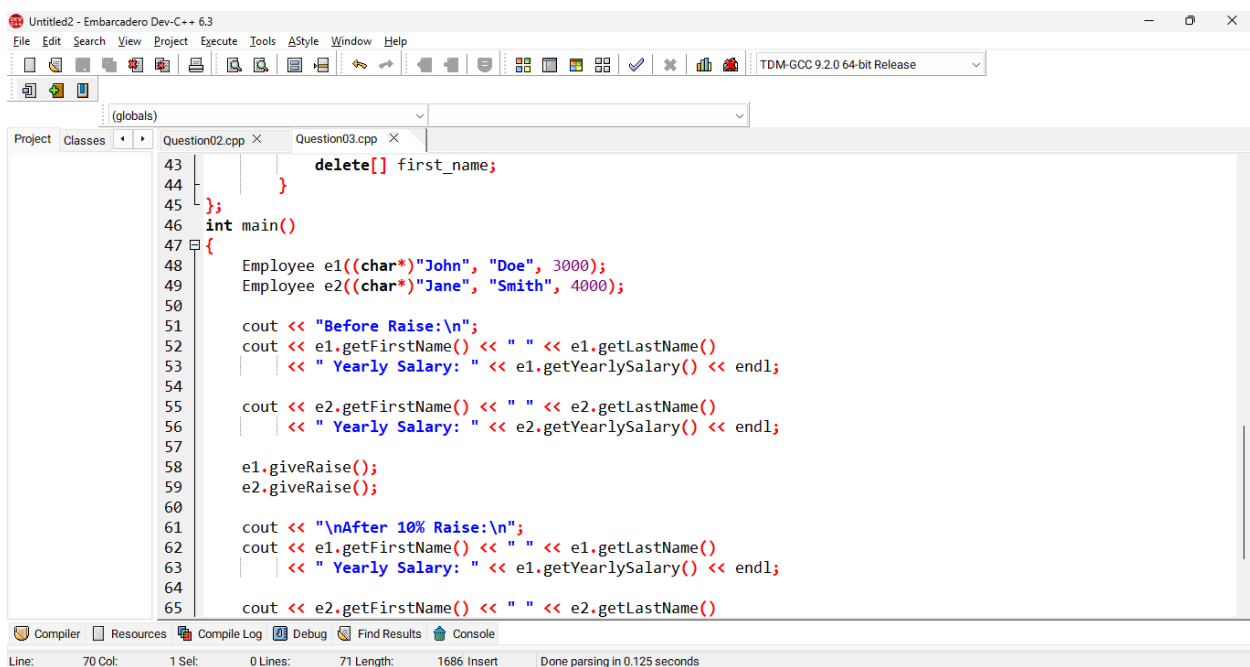
```
1 #include<iostream>
2 #include<string>
3 #include<cstring>
4 using namespace std;
5 class Employee{
6     private:
7         char *first_name;
8         string last_name;
9         int salary;
10
11     public:
12     Employee(char* fname, string lname, int monthllysal){
13         first_name = new char[strlen(fname) + 1];
14         strcpy(first_name, fname);
15         last_name = lname;
16         if (monthllysal > 0)
17             salary = monthllysal;
18         else
19             salary = 0;
20     }
21
22     char* getFirstName(){
23         return first_name;
```



Question03.cpp

```
22 char* getFirstName(){
23     return first_name;
24 }
25
26 string getLastName(){
27     return last_name;
28 }
29
30 int getMonthlySalary(){
31     return salary;
32 }
33
34 int getYearlySalary(){
35     return salary * 12;
36 }
37
38 void giveRaise(){
39     salary = salary + (salary * 10 / 100);
40 }
41
42 ~Employee(){
43     delete[] first_name;
44 }
```

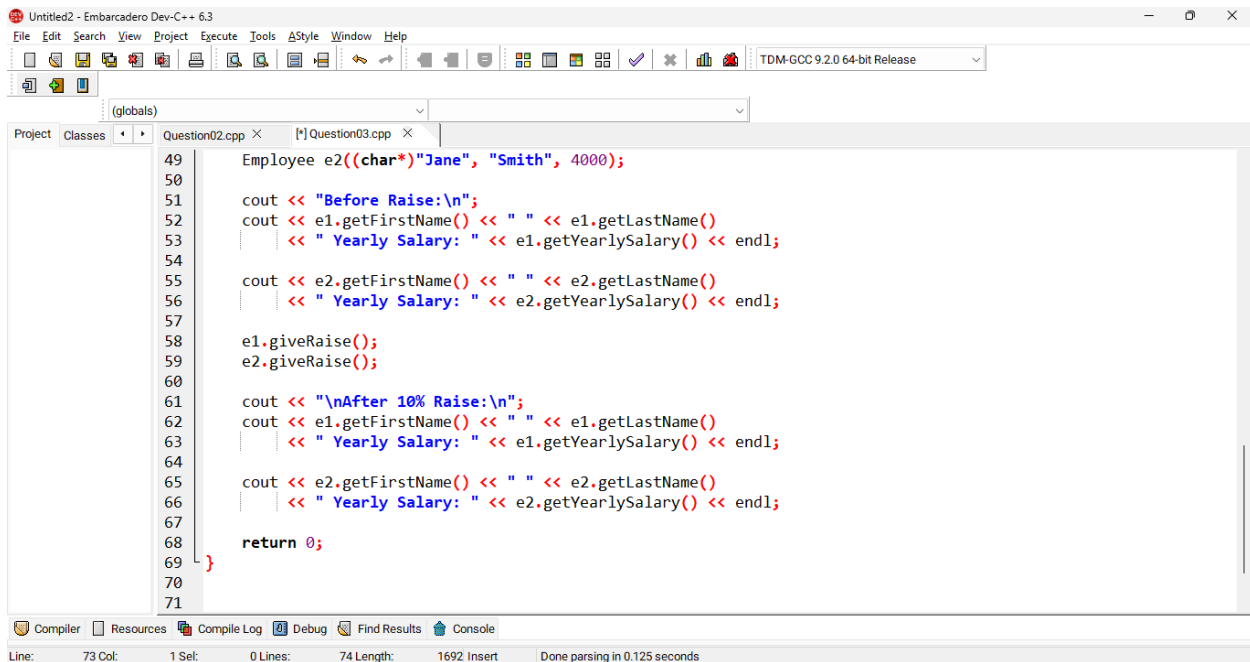
Line: 70 Col: 1 Sel: 0 Lines: 71 Length: 1686 Insert Done parsing in 0.125 seconds



Question02.cpp

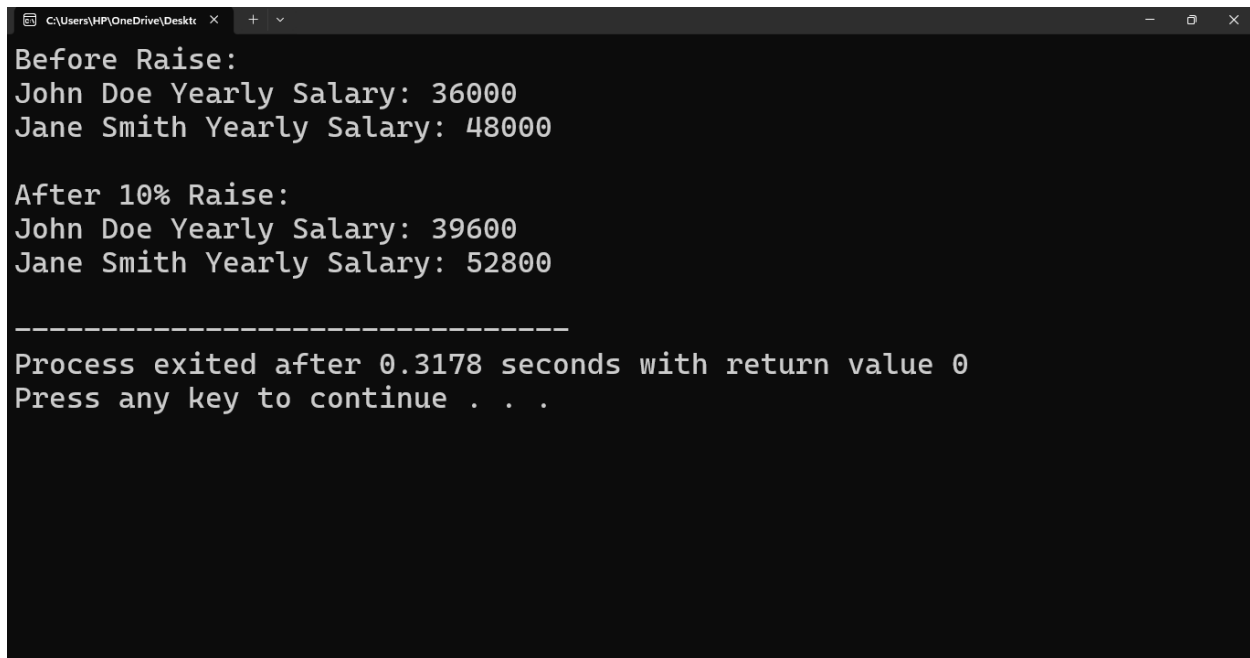
```
43     delete[] first_name;
44 }
45 };
46 int main()
47 {
48     Employee e1((char*)"John", "Doe", 3000);
49     Employee e2((char*)"Jane", "Smith", 4000);
50
51     cout << "Before Raise:\n";
52     cout << e1.getFirstName() << " " << e1.getLastName()
53     << " Yearly Salary: " << e1.getYearlySalary() << endl;
54
55     cout << e2.getFirstName() << " " << e2.getLastName()
56     << " Yearly Salary: " << e2.getYearlySalary() << endl;
57
58     e1.giveRaise();
59     e2.giveRaise();
60
61     cout << "\nAfter 10% Raise:\n";
62     cout << e1.getFirstName() << " " << e1.getLastName()
63     << " Yearly Salary: " << e1.getYearlySalary() << endl;
64
65     cout << e2.getFirstName() << " " << e2.getLastName()
```

Line: 70 Col: 1 Sel: 0 Lines: 71 Length: 1686 Insert Done parsing in 0.125 seconds



```
49 Employee e2((char*)"Jane", "Smith", 4000);
50
51 cout << "Before Raise:\n";
52 cout << e1.getFirstName() << " " << e1.getLastName()
53 | << " Yearly Salary: " << e1.getYearlySalary() << endl;
54
55 cout << e2.getFirstName() << " " << e2.getLastName()
56 | << " Yearly Salary: " << e2.getYearlySalary() << endl;
57
58 e1.giveRaise();
59 e2.giveRaise();
60
61 cout << "\nAfter 10% Raise:\n";
62 cout << e1.getFirstName() << " " << e1.getLastName()
63 | << " Yearly Salary: " << e1.getYearlySalary() << endl;
64
65 cout << e2.getFirstName() << " " << e2.getLastName()
66 | << " Yearly Salary: " << e2.getYearlySalary() << endl;
67
68 return 0;
69 }
70
71
```

### Output:



```
C:\Users\HP\OneDrive\Desktop >
Before Raise:
John Doe Yearly Salary: 36000
Jane Smith Yearly Salary: 48000

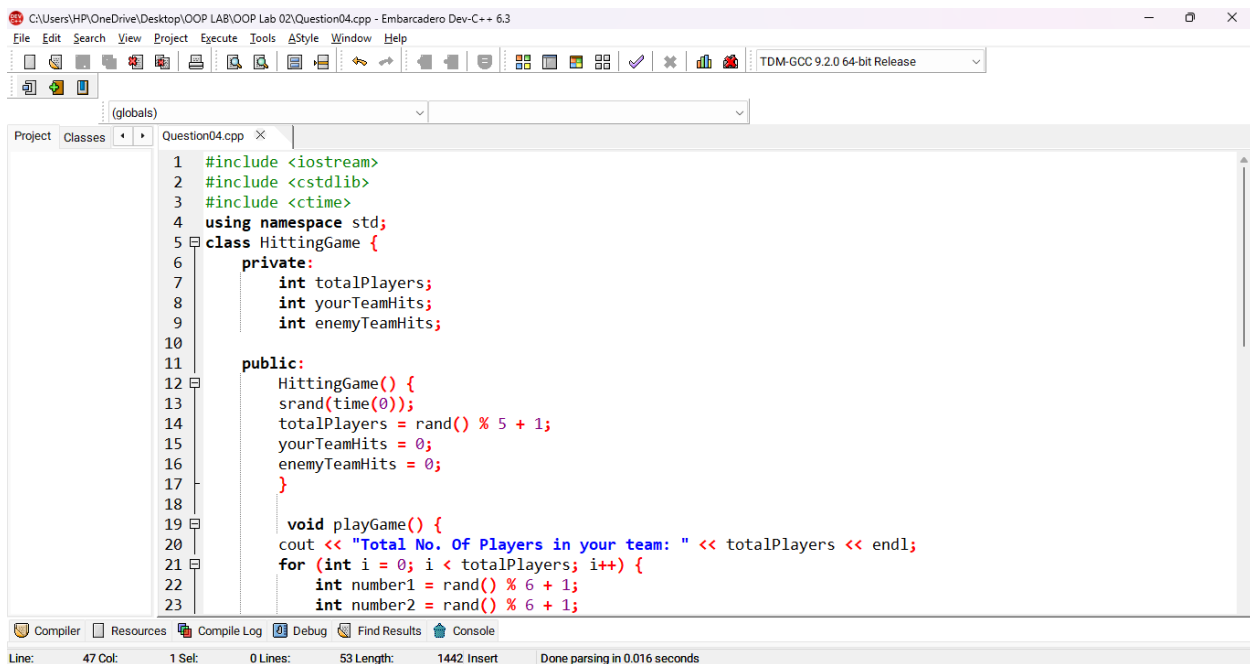
After 10% Raise:
John Doe Yearly Salary: 39600
Jane Smith Yearly Salary: 52800

-----
Process exited after 0.3178 seconds with return value 0
Press any key to continue . . .
```

**Question 04:** Write C++ code to represent a hitting game by using OOP concept. The details are as follows:

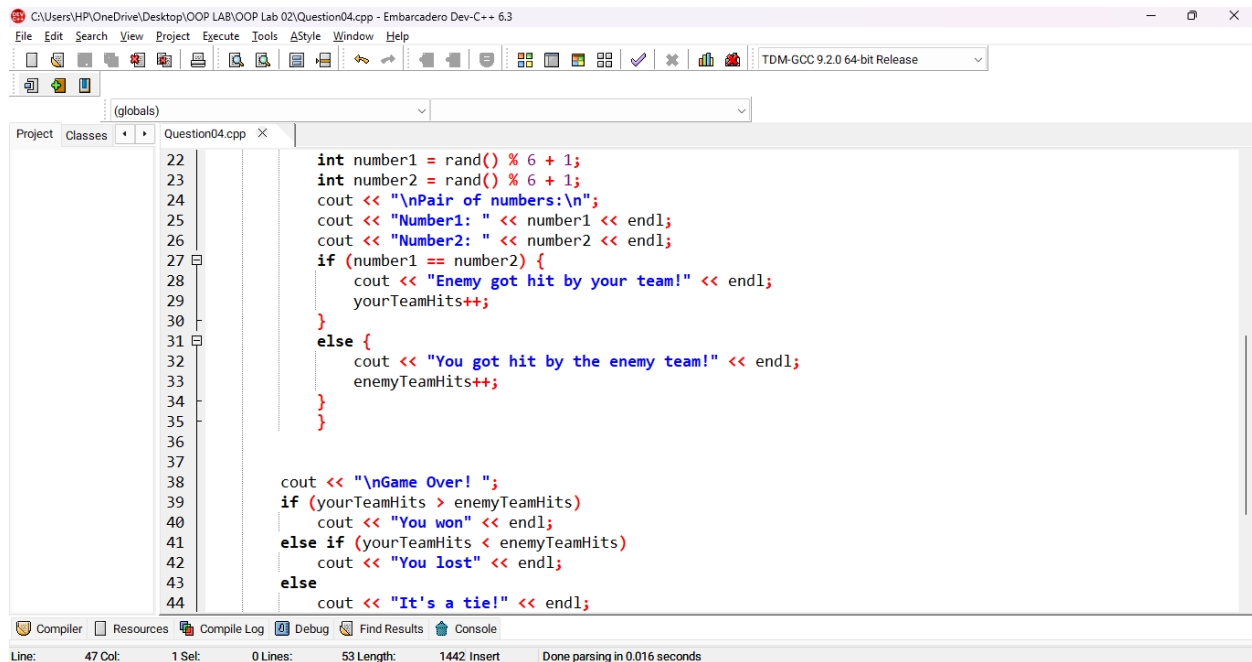
This game is being played between two teams (i.e. your team and the enemy team). The total number of players in your team is randomly generated and stored accordingly. The function generates a pair of numbers and matches each pair. If the numbers get matched, the following message is displayed: "Enemy got hit by your team!" Otherwise, the following message is displayed: "You got hit by the enemy team!" The number of hits should be equal to the number of players in your team. The program should tell the final result of your team by counting the hits of both the teams.

**Source Code:**



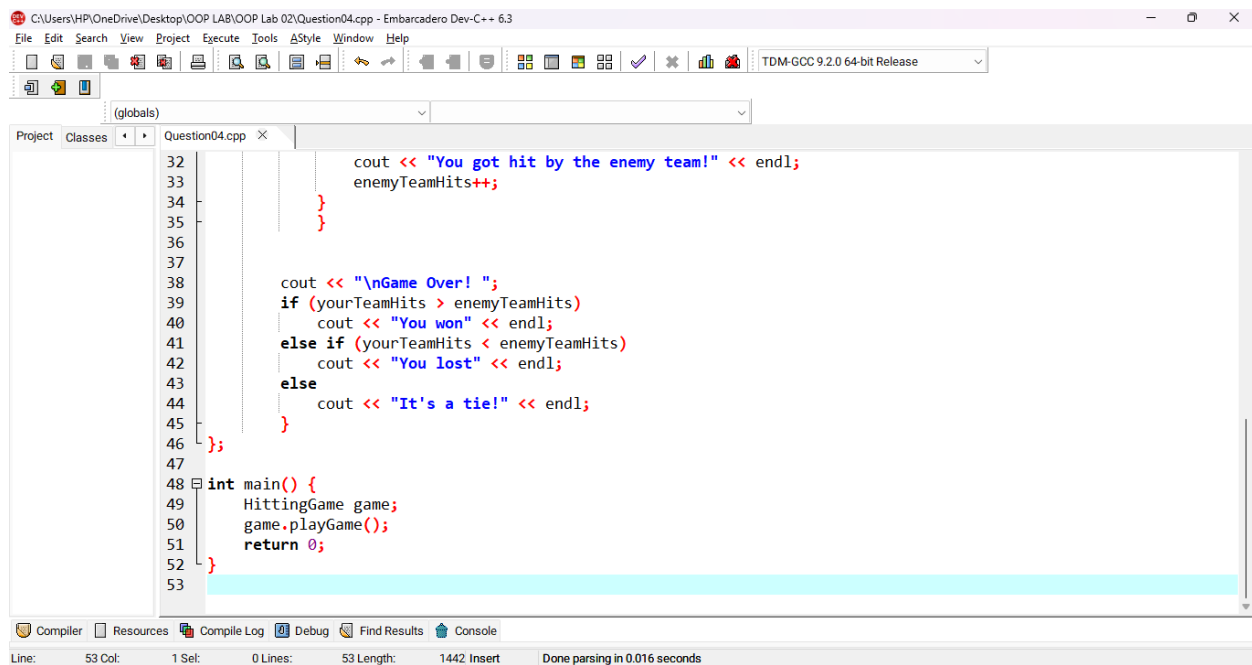
```
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4 using namespace std;
5 class HittingGame {
6     private:
7         int totalPlayers;
8         int yourTeamHits;
9         int enemyTeamHits;
10
11     public:
12         HittingGame() {
13             srand(time(0));
14             totalPlayers = rand() % 5 + 1;
15             yourTeamHits = 0;
16             enemyTeamHits = 0;
17         }
18
19         void playGame() {
20             cout << "Total No. Of Players in your team: " << totalPlayers << endl;
21             for (int i = 0; i < totalPlayers; i++) {
22                 int number1 = rand() % 6 + 1;
23                 int number2 = rand() % 6 + 1;
```

The screenshot shows a C++ IDE window titled "C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Question04.cpp - Embarcadero Dev-C++ 6.3". The code defines a class `HittingGame` with private attributes `totalPlayers`, `yourTeamHits`, and `enemyTeamHits`. The public methods include a constructor that initializes `totalPlayers` to a random value between 1 and 5, and `playGame` which prints the total number of players and enters a loop to generate random numbers for each player. The status bar at the bottom indicates the file has 47 lines, 53 characters, and 1 selection.



The screenshot shows a C++ IDE with a file named 'Question04.cpp'. The code implements a function that generates two random numbers between 1 and 6. It then compares them to determine the outcome of a game: if the numbers are equal, the player wins; if the player's number is greater, the player wins; if the player's number is less, the player loses; otherwise, it's a tie. The code uses 'rand()' for random number generation and 'cout' for output.

```
22 int number1 = rand() % 6 + 1;
23 int number2 = rand() % 6 + 1;
24 cout << "\nPair of numbers:\n";
25 cout << "Number1: " << number1 << endl;
26 cout << "Number2: " << number2 << endl;
27 if (number1 == number2) {
28     cout << "Enemy got hit by your team!" << endl;
29     yourTeamHits++;
30 }
31 else {
32     cout << "You got hit by the enemy team!" << endl;
33     enemyTeamHits++;
34 }
35 }
36
37
38 cout << "\nGame Over! ";
39 if (yourTeamHits > enemyTeamHits)
40     cout << "You won" << endl;
41 else if (yourTeamHits < enemyTeamHits)
42     cout << "You lost" << endl;
43 else
44     cout << "It's a tie!" << endl;
```



The screenshot shows the same C++ IDE with the 'main()' function. It creates a 'HittingGame' object named 'game' and calls the 'playGame()' method. The 'main()' function returns 0, indicating successful execution.

```
32 cout << "You got hit by the enemy team!" << endl;
33 enemyTeamHits++;
34 }
35 }
36
37
38 cout << "\nGame Over! ";
39 if (yourTeamHits > enemyTeamHits)
40     cout << "You won" << endl;
41 else if (yourTeamHits < enemyTeamHits)
42     cout << "You lost" << endl;
43 else
44     cout << "It's a tie!" << endl;
45 }
46 };
47
48 int main() {
49     HittingGame game;
50     game.playGame();
51     return 0;
52 }
53
```

**Output:**

```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Example01.cpp - Embarcadero Dev-C++ 6.3

Pair of numbers:
Number1: 5
Number2: 2
You got hit by the enemy team!

Pair of numbers:
Number1: 3
Number2: 4
You got hit by the enemy team!

Pair of numbers:
Number1: 4
Number2: 6
You got hit by the enemy team!

Game Over! You lost

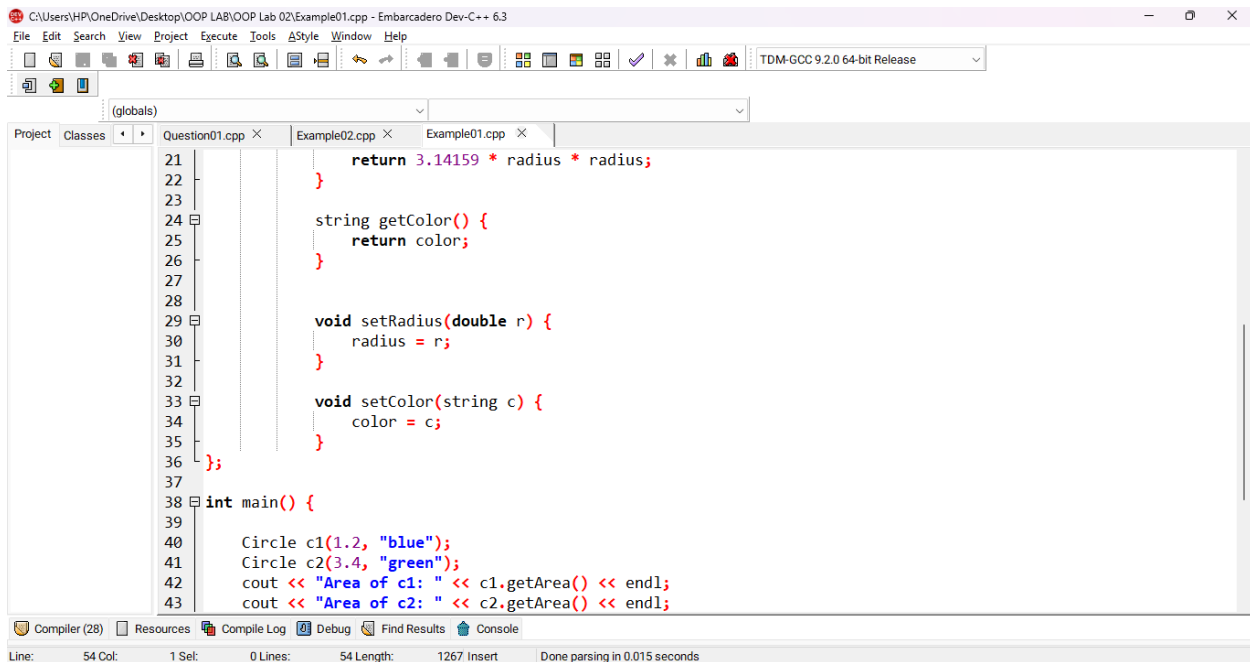
-----
Process exited after 0.2208 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

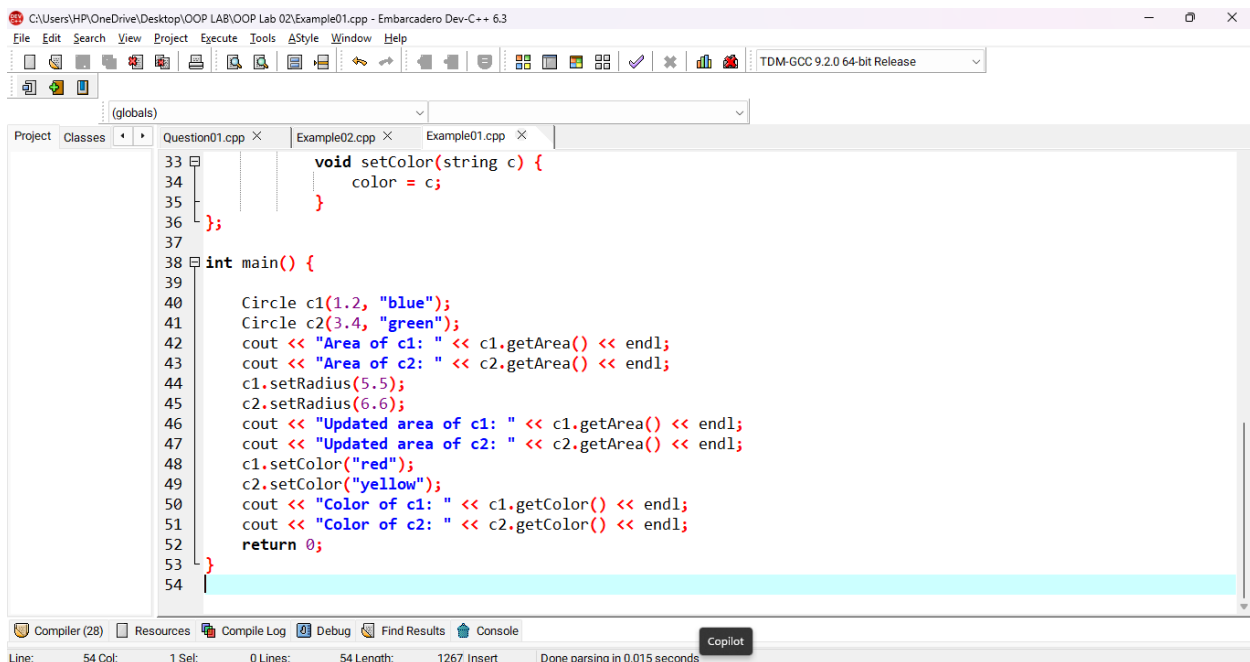
```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Example01.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release

(globals)
Project Classes Question01.cpp Example02.cpp Example01.cpp
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Circle {
6     private:
7         double radius;
8         string color;
9
10    public:
11        Circle(double r, string c) {
12            radius = r;
13            color = c;
14        }
15
16        double getRadius() {
17            return radius;
18        }
19
20        double getArea() {
21            return 3.14159 * radius * radius;
22        }
23}
```

Compiler (28) Resources Compile Log Debug Find Results Console  
Line: 32 Col: 1 Sel: 0 Lines: 54 Length: 1267 Insert Done parsing in 0.015 seconds



```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Example01.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Question01.cpp Example02.cpp Example01.cpp
21         return 3.14159 * radius * radius;
22     }
23
24     string getColor() {
25         return color;
26     }
27
28
29     void setRadius(double r) {
30         radius = r;
31     }
32
33     void setColor(string c) {
34         color = c;
35     }
36 };
37
38 int main() {
39
40     Circle c1(1.2, "blue");
41     Circle c2(3.4, "green");
42     cout << "Area of c1: " << c1.getArea() << endl;
43     cout << "Area of c2: " << c2.getArea() << endl;
44
45 Compiler (28) Resources Compile Log Debug Find Results Console
Line: 54 Col: 1 Sel: 0 Lines: 54 Length: 1267 Insert Done parsing in 0.015 seconds
```



```
C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Example01.cpp - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Question01.cpp Example02.cpp Example01.cpp
33     void setColor(string c) {
34         color = c;
35     }
36 };
37
38 int main() {
39
40     Circle c1(1.2, "blue");
41     Circle c2(3.4, "green");
42     cout << "Area of c1: " << c1.getArea() << endl;
43     cout << "Area of c2: " << c2.getArea() << endl;
44     c1.setRadius(5.5);
45     c2.setRadius(6.6);
46     cout << "Updated area of c1: " << c1.getArea() << endl;
47     cout << "Updated area of c2: " << c2.getArea() << endl;
48     c1.setColor("red");
49     c2.setColor("yellow");
50     cout << "Color of c1: " << c1.getColor() << endl;
51     cout << "Color of c2: " << c2.getColor() << endl;
52     return 0;
53 }
54
55 Compiler (28) Resources Compile Log Debug Find Results Console Copilot
Line: 54 Col: 1 Sel: 0 Lines: 54 Length: 1267 Insert Done parsing in 0.015 seconds
```

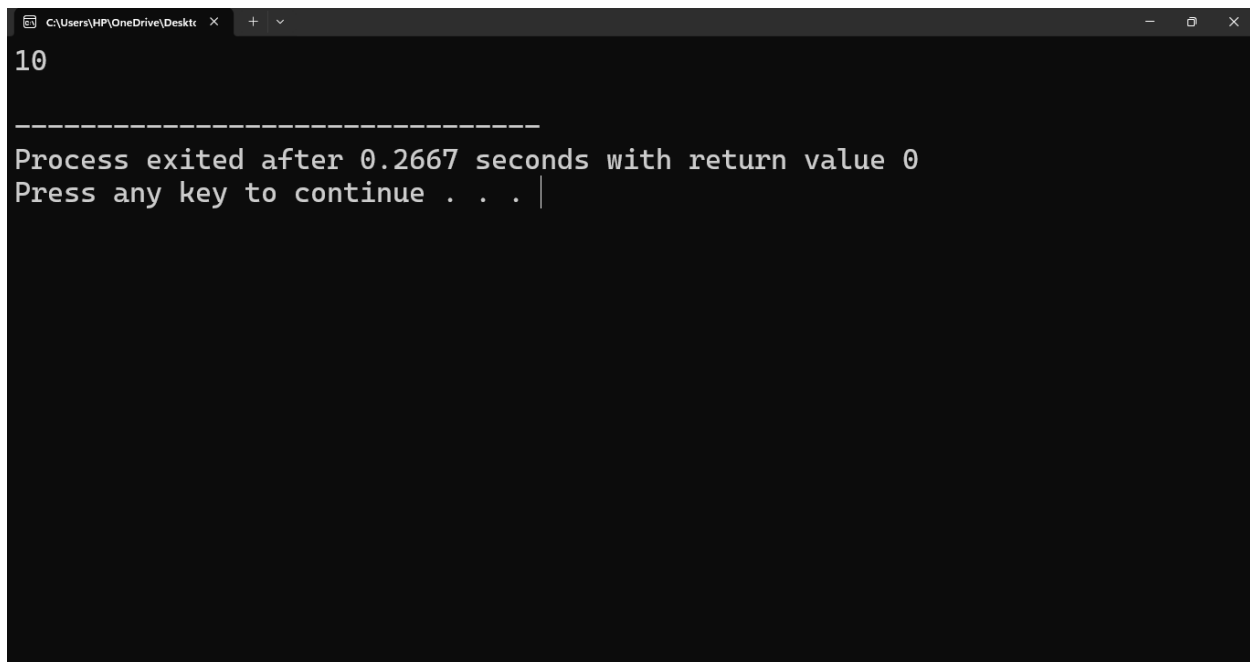
**Output:**

```
C:\Users\HP\OneDrive\Desktop x + v
Area of c1: 4.52389
Area of c2: 36.3168
Updated area of c1: 95.0331
Updated area of c2: 136.848
Color of c1: red
Color of c2: yellow

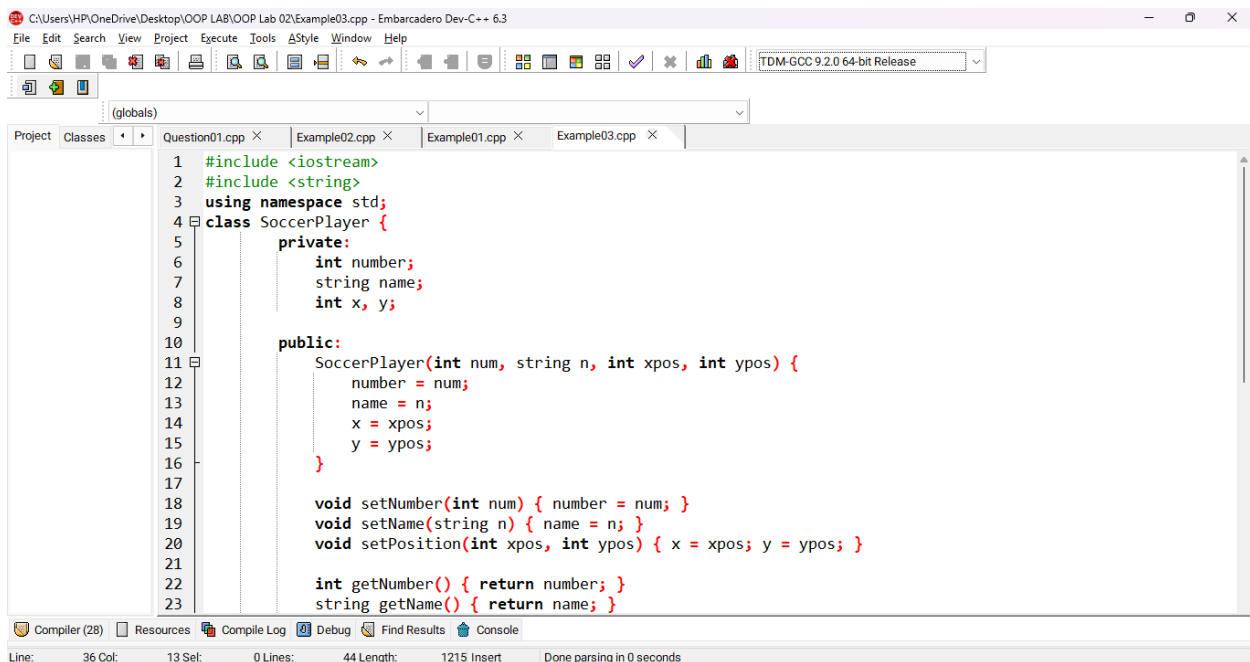
-----
Process exited after 0.2706 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:****Source Code:**

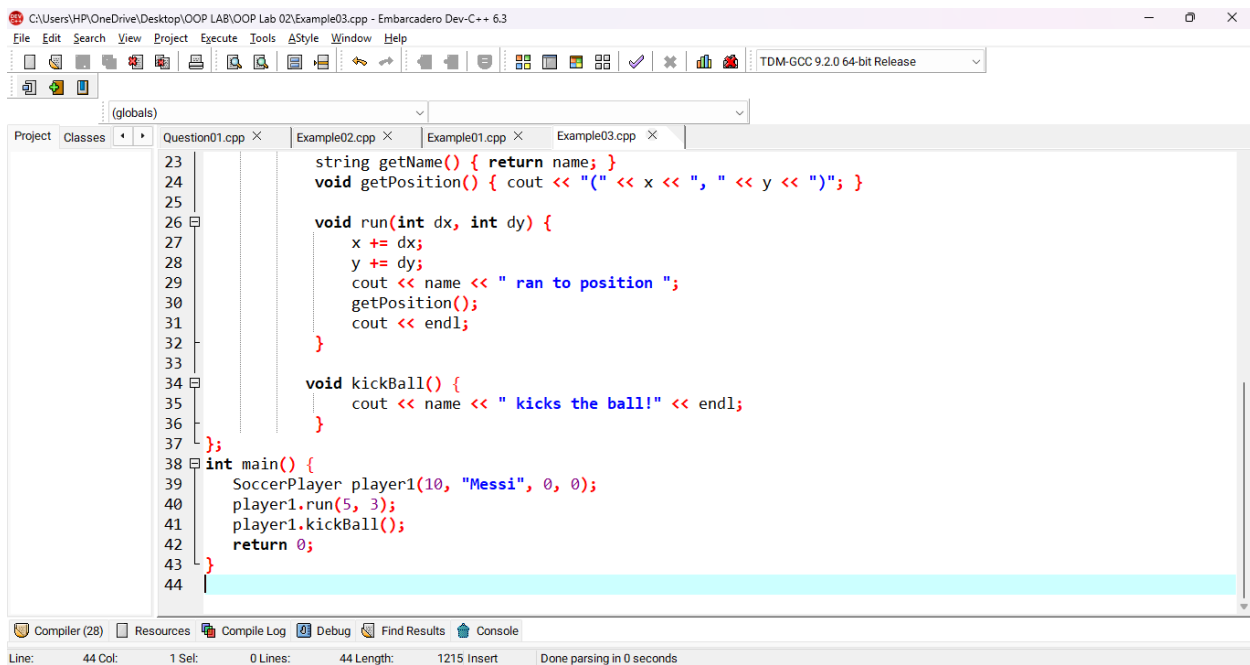
```
Untitled2 - [Executing] - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 9.2.0 64-bit Release
(globals)
Project Classes Question01.cpp Example02.cpp
1 #include <iostream>
2 using namespace std;
3
4 class firstprogram {
5 private:
6     int number1;
7
8 public:
9     void set(int input1) {
10         number1 = input1;
11     }
12
13     int get() {
14         return number1;
15     }
16 };
17 int main() {
18     firstprogram myInstance;
19     myInstance.set(10);
20     cout << myInstance.get() << endl;
21     return 0;
22 }
23
Compiler (28) Resources Compile Log Debug Find Results Console
Line: 16 Col: 3 Sel: 0 Lines: 23 Length: 361 Insert Done parsing in 0 seconds
```

**Output:**

```
10
-----
Process exited after 0.2667 seconds with return value 0
Press any key to continue . . . |
```

**Example 03:****Source Code:**

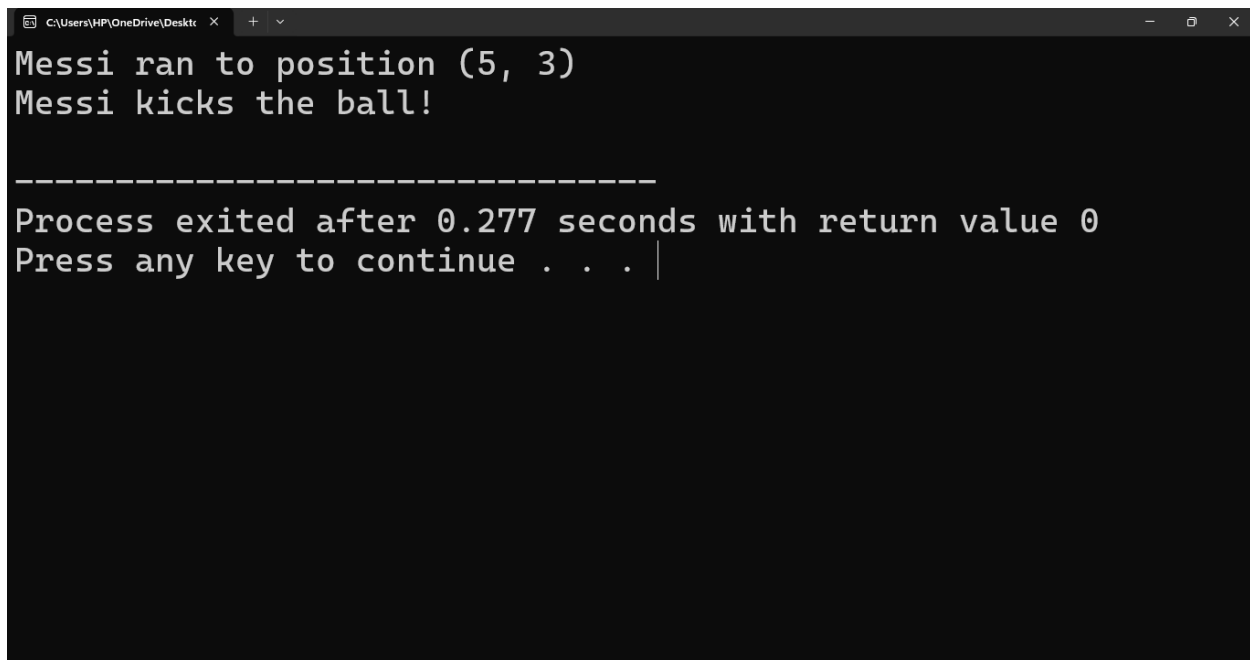
```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class SoccerPlayer {
5     private:
6         int number;
7         string name;
8         int x, y;
9
10    public:
11        SoccerPlayer(int num, string n, int xpos, int ypos) {
12            number = num;
13            name = n;
14            x = xpos;
15            y = ypos;
16        }
17
18        void setNumber(int num) { number = num; }
19        void setName(string n) { name = n; }
20        void setPosition(int xpos, int ypos) { x = xpos; y = ypos; }
21
22        int getNumber() { return number; }
23        string getName() { return name; }
```



The screenshot shows an IDE window titled "C:\Users\HP\OneDrive\Desktop\OOP LAB\OOP Lab 02\Example03.cpp - Embarcadero Dev-C++ 6.3". The code defines a `SoccerPlayer` class with methods `getName()`, `getPosition()`, `run()`, and `kickBall()`. The `main()` function creates a `SoccerPlayer` object named `player1` with the name "Messi" and coordinates (10, 0), then calls `run(5, 3)` and `kickBall()`.

```
23 string getName() { return name; }
24 void getPosition() { cout << "(" << x << ", " << y << ")"; }
25
26 void run(int dx, int dy) {
27     x += dx;
28     y += dy;
29     cout << name << " ran to position ";
30     getPosition();
31     cout << endl;
32 }
33
34 void kickBall() {
35     cout << name << " kicks the ball!" << endl;
36 }
37
38 int main() {
39     SoccerPlayer player1(10, "Messi", 0, 0);
40     player1.run(5, 3);
41     player1.kickBall();
42     return 0;
43 }
44
```

### Output:



The terminal window shows the output of the program: "Messi ran to position (5, 3)" followed by "Messi kicks the ball!". After a separator line, it shows "Process exited after 0.277 seconds with return value 0" and "Press any key to continue . . . |".

```
Messi ran to position (5, 3)
Messi kicks the ball!

-----
Process exited after 0.277 seconds with return value 0
Press any key to continue . . . |
```

# **LAB 03**

**OBJECT ORIENTED PROGRAMMING (CT-260)**

**Question 01:** Write a C++ program to copy the value of one object to another object using copy constructor.

**Source Code:**

```
#include <iostream>
using namespace std;
class Number {
private:
    int *value;

public:
    Number(int v) {
        value = new int(v);
        cout << "Parameterized Constructor called" << endl;
    }

    Number(const Number &obj) {
        value = new int(*(obj.value));
        cout << "Copy Constructor called" << endl;
    }

    void display() {
        cout << "Value: " << *value << endl;
    }

    void setValue(int v) {
        *value = v;
    }

    ~Number() {
        delete value;
        cout << "Destructor called" << endl;
    }
}
```

```
        cout << "Destructor called" << endl;
    }
};

int main() {
    Number num1(100);
    Number num2 = num1;

    cout << "Before changing num2:" << endl;
    cout << "num1: ";
    num1.display();
    cout << "num2: ";
    num2.display();

    num2.setValue(200);

    cout << "\nAfter changing num2:" << endl;
    cout << "num1: ";
    num1.display();
    cout << "num2: ";
    num2.display();

    Number num3(num2);
    num3.display();

    return 0;
}
```

Output:

```
Parameterized Constructor called
Copy Constructor called
Before changing num2:
num1: Value: 100
num2: Value: 100

After changing num2:
num1: Value: 100
num2: Value: 200
Copy Constructor called
Value: 200
Destructor called
Destructor called
Destructor called

-----
Process exited after 0.2569 seconds with return value 0
Press any key to continue . . .
```

**Question 02:** Create a class tollbooth. The two data items are a type int to hold the total number of cars and a type double to hold the total amount of money collected. A constructor initializes both these to 0. When a car passes the toll, a member function called payingCar() increments the car total and adds 0.50 to the cash total. Another member function displays the two totals. DESIGN and IMPLEMENT this case. Make assumptions (if required) and include it in the description before designing the solution.

Source Code:

```
#include <iostream>
using namespace std;
class TollBooth {
private:
    int cars_total;
    double amount;

public:
    TollBooth() {
        cars_total = 0;
        amount = 0.0;
    }

    void payingCar() {
        cars_total += 1;
        amount += 0.50;
    }

    void Display() {
        cout << "Number of Cars: " << cars_total << endl;
        cout << "Total Amount Collected: $" << amount << endl;
    }
};

int main() {
    TollBooth booth1, booth2;

    booth1.payingCar();
```

```
int main() {
    TollBooth booth1, booth2;

    booth1.payingCar();
    booth1.payingCar();
    booth1.payingCar();

    booth2.payingCar();
    booth2.payingCar();

    cout << "Booth 1 totals:" << endl;
    booth1.Display();

    cout << "\nBooth 2 totals:" << endl;
    booth2.Display();

    return 0;
}
/*The program successfully simulates multiple tollbooths,
updates totals whenever a car passes, and displays accurate
totals for each booth. Each TollBooth instance maintains its
own car count and total collected amount.*/
```

**Output:**

```
Booth 1 totals:
Number of Cars: 3
Total Amount Collected: $1.5

Booth 2 totals:
Number of Cars: 2
Total Amount Collected: $1

-----
Process exited after 0.3419 seconds with return value 0
Press any key to continue . . .
```

**Question 03:** Some of the characteristics of a book are the title, author(s), publisher, ISBN, price, and year of publication. Design a class bookType that defines the book as an ADT.

- Each object of the class bookType can hold the following information about a book: title, up to four authors, publisher, ISBN, price, and number of copies in stock. To keep track of the number of authors, add another member variable.
- Include the member functions to perform the various operations on objects of type bookType. For example, the usual operations that can be performed on the title are to

show the title, set the title, and check whether a title is the same as the actual title of the book. Similarly, the typical operations that can be performed on the number of copies in stock are to show the number of copies in stock, set the number of copies in stock, update the number of copies in stock, and return the number of copies in stock. Add similar operations for the publisher, ISBN, book price, and authors. Add the appropriate constructors and a destructor (if one is needed).

- Write the definitions of the member functions of the class bookType.
- Write a program that uses the class bookType and tests various operations on the objects of the class bookType. Declare an array of 100 components of type bookType. Some of the operations that you should perform are to search for a book by its title, search by ISBN, and update the number of copies of a book.

#### Source Code:

```
#include <iostream>
#include <string>
using namespace std;
class bookType{
private:
    string title;
    string authors[4];
    int authorCount;
    string publisher;
    string ISBN;
    double price;
    int copiesInStock;

public:
    bookType(){
        title = "";
        publisher = "";
        ISBN = "";
        price = 0.0;
        copiesInStock = 0;
        authorCount = 0;
    }

    void setBookDetails(string t, string a[], int aCount, string pub, string isbn, double p, int copies){
        title = t;
        publisher = pub;
        ISBN = isbn;
        price = p;
    }
}
```

```
    price = p;
    copiesInStock = copies;

    if (aCount > 4)
        authorCount = 4;
    else
        authorCount = aCount;

    for (int i = 0; i < authorCount; i++)
        authors[i] = a[i];
}

void displayBook(){
    cout << "Title: " << title << endl;
    cout << "Authors: ";
    for (int i = 0; i < authorCount; i++){
        cout << authors[i];
        if (i < authorCount - 1) cout << ", ";
    }
    cout << endl;
    cout << "Publisher: " << publisher << endl;
    cout << "ISBN: " << ISBN << endl;
    cout << "Price: " << price << endl;
    cout << "Copies in Stock: " << copiesInStock << endl;
    cout << "-----" << endl;
}

bool checkTitle(string t){
```

```
bool checkTitle(string t){
    if (title == t)
        return true;
    else
        return false;
}

bool checkISBN(string isbn){
    if (ISBN == isbn)
        return true;
    else
        return false;
}

void updateCopies(int newCopies){
    copiesInStock = newCopies;
}

int getCopies(){
    return copiesInStock;
}

};

void searchByTitle(bookType library[], int size, string searchTitle){
    bool found = false;
    for (int i = 0; i < size; i++){
        if (library[i].checkTitle(searchTitle)){
            cout << "\nBook found by title:\n";
        }
    }
}
```

```

void searchByTitle(bookType library[], int size, string searchTitle){
    bool found = false;
    for (int i = 0; i < size; i++){
        if (library[i].checkTitle(searchTitle)){
            cout << "\nBook found by title:\n";
            library[i].displayBook();
            found = true;
            break;
        }
    }
    if (!found)
        cout << "\nBook not found by title.\n";
}

void searchByISBN(bookType library[], int size, string searchISBN){
    bool found = false;
    for (int i = 0; i < size; i++){
        if (library[i].checkISBN(searchISBN)){
            cout << "\nBook found by ISBN:\n";
            library[i].displayBook();
            found = true;
            break;
        }
    }
    if (!found)
        cout << "\nBook not found by ISBN.\n";
}

if (!found)
    cout << "\nBook not found by ISBN.\n";
}

int main(){
    bookType library[100];
    int bookCount = 2;

    string authors1[2] = {"Dennis Ritchie", "Brian Kernighan"};
    library[0].setBookDetails("The C Programming Language", authors1, 2, "Prentice Hall", "9780131103627", 650.0, 10);

    string authors2[1] = {"Mark Allen Weiss"};
    library[1].setBookDetails(
        "Data Structures and Algorithm Analysis", authors2, 1, "Pearson", "9780132847377", 550.0, 5);

    string searchTitle = "The C Programming Language";
    searchByTitle(library, bookCount, searchTitle);

    string searchISBN = "9780132847377";
    searchByISBN(library, bookCount, searchISBN);

    library[0].updateCopies(15);
    cout << "\nAfter updating copies of first book:\n";
    library[0].displayBook();

    return 0;
}

```

**Output:**

```
Book found by title:
Title: The C Programming Language
Authors: Dennis Ritchie, Brian Kernighan
Publisher: Prentice Hall
ISBN: 9780131103627
Price: 650
Copies in Stock: 10
-----

Book found by ISBN:
Title: Data Structures and Algorithm Analysis
Authors: Mark Allen Weiss
Publisher: Pearson
ISBN: 9780132847377
Price: 550
Copies in Stock: 5
-----

After updating copies of first book:
Title: The C Programming Language
Authors: Dennis Ritchie, Brian Kernighan
Publisher: Prentice Hall
ISBN: 9780131103627
Price: 650
Copies in Stock: 15
-----

-----

Process exited after 0.3318 seconds with return value 0
Press any key to continue . . .
```

Example 01:

Source Code:

```
#include <iostream>
using namespace std;
class construct {
public:
    int a, b;
    construct(){
        a = 10;
        b = 20;
    }
};

int main() {
    construct c;
    cout << "a: " << c.a << endl << "b: " << c.b;
    return 1;
}
```

Output:

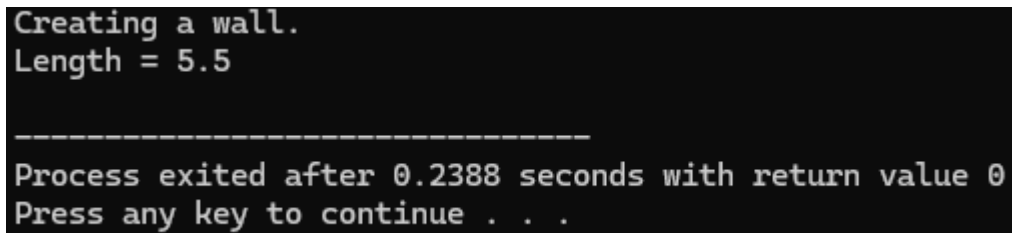
```
a: 10
b: 20
-----
Process exited after 0.2677 seconds with return value 1
Press any key to continue . . .
```

Example 02:Source Code:

```
#include <iostream>
using namespace std;
class Wall {
private:
    double length;

public:
    Wall(){
        length = 5.5;
        cout << "Creating a wall." << endl;
        cout << "Length = " << length << endl;
    }
};

int main() {
    Wall wall1;
    return 0;
}
```

**Output:**A screenshot of a terminal window showing the output of the C++ program. The output consists of two lines: "Creating a wall." followed by "Length = 5.5". Below these lines is a horizontal dashed line. At the bottom, it says "Process exited after 0.2388 seconds with return value 0" and "Press any key to continue . . .".

```
Creating a wall.
Length = 5.5

-----
Process exited after 0.2388 seconds with return value 0
Press any key to continue . . .
```

**Example 03:****Source Code:**

```
#include <iostream>
using namespace std;
class Point {
private:
    int x, y;

public:
    Point(int x1, int y1) {
        x = x1;
        y = y1;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }
};

int main() {
    Point p1(10, 15);
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    return 0;
}
```

**Output:**

```
p1.x = 10, p1.y = 15
-----
Process exited after 0.2284 seconds with return value 0
Press any key to continue . . . |
```

**Example 04:****Source Code:**

```
#include <iostream>
using namespace std;
class Wall {
private:
    double length;
    double height;

public:
    Wall(double len, double hgt) {
        length = len;
        height = hgt;
    }

    double calculateArea() {
        return length * height;
    }
};

int main() {
    Wall wall1(10.5, 8.6);
    Wall wall2(8.5, 6.3);

    cout << "Area of Wall 1: " << wall1.calculateArea() << endl;
    cout << "Area of Wall 2: " << wall2.calculateArea();

    return 0;
}
```

**Output:**

```
Area of Wall 1: 90.3
Area of Wall 2: 53.55
-----
Process exited after 0.2561 seconds with return value 0
Press any key to continue . . .
```

**Example 05:****Source Code:**

```
#include <iostream>
#include <conio.h>
using namespace std;
class Example {
    int a, b;

public:
    Example(int x, int y) {
        a = x;
        b = y;
        cout << "\nI'm Constructor";
    }

    Example(const Example& obj) {
        a = obj.a;
        b = obj.b;
        cout << "\nI'm Copy Constructor";
    }

    void Display() {
        cout << "\nValues: " << a << "\t" << b;
    }
};

int main() {
    Example Object(10, 20);
    Example Object2(Object);
    Example Object3 = Object;

    Object.Display();
    Object2.Display();
    Object3.Display();

    return 0;
}
```

**Output:**

```
I'm Constructor
I'm Copy Constructor
I'm Copy Constructor
Values: 10      20
Values: 10      20
Values: 10      20
-----
Process exited after 0.2821 seconds with return value 0
Press any key to continue . . . |
```

**Example 06:****Source Code:**

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Wall {
private:
    double *length;
    double *height;

public:
    Wall() {
        length = nullptr;
        height = nullptr;
    }

    Wall(double len, double hgt) {
        length = new double;
        height = new double;
        if (length != nullptr && height != nullptr) {
            *length = len;
            *height = hgt;
        } else {
            exit(1);
        }
    }

    Wall(const Wall &obj) {
        length = new double;
        height = new double;
        *length = *(obj.length);
        *height = *(obj.height);
    }

    void set_values(double len, double hei) {
```

```
void set_values(double len, double hei) {
    *length = len;
    *height = hei;
}

double calculateArea() const {
    return (*length) * (*height);
}

~Wall() {
    delete length;
    delete height;
}

};

int main() {
    Wall wall1(10.5, 8.6);
    Wall wall2 = wall1;

    cout << "Area of Wall 1: " << wall1.calculateArea() << endl;
    cout << "Area of Wall 2: " << wall2.calculateArea() << endl;

    wall2.set_values(2.3, 1.5);
    cout << "After changing values for Wall 2:" << endl;
    cout << "Area of Wall 1: " << wall1.calculateArea() << endl;
    cout << "Area of Wall 2: " << wall2.calculateArea() << endl;

    return 0;
}
```

**Output:**

```
Area of Wall 1: 90.3
Area of Wall 2: 90.3
After changing values for Wall 2:
Area of Wall 1: 90.3
Area of Wall 2: 3.45

-----
Process exited after 0.2948 seconds with return value 0
Press any key to continue . . .
```

Example 07:Source Code:

```
#include <iostream>
using namespace std;
class HelloWorld {
public:
    HelloWorld() {
        cout << "Constructor is called" << endl;
    }

    ~HelloWorld() {
        cout << "Destructor is called" << endl;
    }

    void display() {
        cout << "Hello World!" << endl;
    }
};

int main() {
    HelloWorld obj;
    obj.display();
    return 0;
}
```

Output:

```
Constructor is called
Hello World!
Destructor is called

-----
Process exited after 0.2756 seconds with return value 0
Press any key to continue . . .
```

# LAB 04

**OBJECT ORIENTED PROGRAMMING (CT-260)**

**Question 01:** Create a class 'Employee' having two data members 'EmployeeName'(char\*) and 'EmployeeId' (int). Keep both data members private. Create three initialized objects 'Employee1', 'Employee2' and 'Employee3' of type 'Employee' in such a way that the employee name for each employee can be changed when required but the employee Id for each employee must be initialized only once and should remain same always. Use member initializer list, accessors/getters and mutators/setters for appropriate data members. The result must be displayed by calling the accessors.

**Source Code:**

```
#include <iostream>
using namespace std;

class Employee {
private:
    string EmployeeName;
    const int EmployeeId;

public:
    Employee(string name, int id) : EmployeeName(name), EmployeeId(id) { }

    void setEmployeeName(string name) {
        EmployeeName = name;
    }

    string getEmployeeName() const {
        return EmployeeName;
    }

    int getEmployeeId() const {
        return EmployeeId;
    }
};

int main() {
    Employee Employee1("Alice", 101);
    Employee Employee2("Bob", 102);
    Employee Employee3("Charlie", 103);
```

```
int main() {
    Employee Employee1("Alice", 101);
    Employee Employee2("Bob", 102);
    Employee Employee3("Charlie", 103);

    cout << "Employee1: " << Employee1.getEmployeeName() << ", ID: " << Employee1.getEmployeeId() << endl;
    cout << "Employee2: " << Employee2.getEmployeeName() << ", ID: " << Employee2.getEmployeeId() << endl;
    cout << "Employee3: " << Employee3.getEmployeeName() << ", ID: " << Employee3.getEmployeeId() << endl;

    Employee1.setEmployeeName("Alicia");
    Employee2.setEmployeeName("Robert");

    cout << "\nAfter name change:" << endl;
    cout << "Employee1: " << Employee1.getEmployeeName() << ", ID: " << Employee1.getEmployeeId() << endl;
    cout << "Employee2: " << Employee2.getEmployeeName() << ", ID: " << Employee2.getEmployeeId() << endl;
    cout << "Employee3: " << Employee3.getEmployeeName() << ", ID: " << Employee3.getEmployeeId() << endl;

    return 0;
}
```

**Output:**

```
Employee1: Alice, ID: 101
Employee2: Bob, ID: 102
Employee3: Charlie, ID: 103

After name change:
Employee1: Alicia, ID: 101
Employee2: Robert, ID: 102
Employee3: Charlie, ID: 103

-----
Process exited after 0.3309 seconds with return value 0
Press any key to continue . . .
```

**Question 02:** Create a class called DynamicArray. The class should contain a pointer to the array, and size of the array as data members. Create the parameterized constructor which take size of the array as input and initializes all the values with 0. Create the member function "add" which takes value as parameter and adds it to the end of the array. Create the member function size() which returns the size of the array.

**Source Code:**

```
#include <iostream>
using namespace std;

class DynamicArray {
private:
    int* arr;
    int capacity;
    int count;
public:
    DynamicArray(int size) {
        capacity = size;
        arr = new int[capacity];
        count = 0;
        for(int i = 0; i < capacity; i++) {
            arr[i] = 0;
        }
    }

    void add(int value) {
        if(count < capacity) {
            arr[count] = value;
            count++;
        } else {
            cout << "Array is full! Cannot add more elements." << endl;
        }
    }

    int size() const {
        return count;
    }
}
```

```
void display() const {
    for(int i = 0; i < count; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

~DynamicArray() {
    delete[] arr;
}

};

int main() {
    DynamicArray arr(5);

    arr.add(10);
    arr.add(20);
    arr.add(30);

    cout << "Array elements: ";
    arr.display();

    cout << "Current size: " << arr.size() << endl;

    arr.add(40);
    arr.add(50);
    arr.add(60);

    cout << "Array elements: ";
    arr.display();
    cout << "Current size: " << arr.size() << endl;

    return 0;
}
```

**Output:**

```
Array elements: 10 20 30
Current size: 3
Array is full! Cannot add more elements.
Array elements: 10 20 30 40 50
Current size: 5

-----
Process exited after 0.2855 seconds with return value 0
Press any key to continue . . .
```

**Question 03:** Write a program in which a class named Account has private member variables named account\_no ,account\_bal ,security\_code. Use a public function to initialize the variables and print all data. Keep track of number of objects using the keyword static.

**Source Code:**

```
#include <iostream>
using namespace std;

class Account {
private:
    int account_no;
    double account_bal;
    int security_code;
    static int object_count;

public:
    void initialize(int accNo, double bal, int code) {
        account_no = accNo;
        account_bal = bal;
        security_code = code;
        object_count++;
    }

    void print() const {
        cout << "Account No: " << account_no << ", Balance: " << account_bal
            << ", Security Code: " << security_code << endl;
    }

    static int getObjectCount() {
        return object_count;
    }

};

static int getObjectCount() {
    return object_count;
}

int Account::object_count = 0;

int main() {
    Account acc1, acc2, acc3;

    acc1.initialize(101, 5000.50, 1234);
    acc2.initialize(102, 10000.00, 5678);
    acc3.initialize(103, 7500.25, 9012);

    acc1.print();
    acc2.print();
    acc3.print();

    cout << "Total number of Account objects initialized: " << Account::getObjectCount() << endl;

    return 0;
}
```

**Output:**

```
Account No: 101, Balance: 5000.5, Security Code: 1234
Account No: 102, Balance: 10000, Security Code: 5678
Account No: 103, Balance: 7500.25, Security Code: 9012
Total number of Account objects initialized: 3

-----
Process exited after 0.3148 seconds with return value 0
Press any key to continue . . . |
```

**Question 04:** Write a program of your own in which you demonstrate the concept of constant keyword.

**Source Code:**

```
#include <iostream>
using namespace std;

class Circle {
private:
    double radius;
    const double PI;    // constant data member

public:
    // Constructor using initializer list for const data member
    Circle(double r) : radius(r), PI(3.14159) { }

    // Const member function
    double getArea() const {
        return PI * radius * radius;
    }

    // Non-const member function
    void setRadius(double r) {
        radius = r;
    }

    // Const member function
    double getRadius() const {
        return radius;
    }

    // Const member function to access const data member
    double getPI() const {
        return PI;
    }
};
```

```
// Const member function to access const data member
double getPI() const {
    return PI;
}
};

int main() {
    // Const variable
    const double fixedValue = 100.5;
    cout << "Const variable value: " << fixedValue << endl;

    Circle c1(5.0);
    cout << "Radius: " << c1.getRadius() << endl;
    cout << "Area: " << c1.getArea() << endl;

    c1.setRadius(10.0);
    cout << "New Radius: " << c1.getRadius() << endl;
    cout << "New Area: " << c1.getArea() << endl;

    // Const object
    const Circle c2(7.0);
    cout << "\nConst object radius: " << c2.getRadius() << endl;
    cout << "Const object area: " << c2.getArea() << endl;
    cout << "Const data member PI: " << c2.getPI() << endl;
    // c2.setRadius(5.0); // Error: cannot call non-const function on const object

    return 0;
}
```

**Output:**

```
Const variable value: 100.5
Radius: 5
Area: 78.5397
New Radius: 10
New Area: 314.159

Const object radius: 7
Const object area: 153.938
Const data member PI: 3.14159

-----
Process exited after 0.3252 seconds with return value 0
Press any key to continue . . . |
```

**Question 05:** "Hotel Mercato" requires a system module that will help the hotel to calculate the rent of the customers. You are required to develop one module of the system according to the following requirements:

- The hotel wants such a system that should have the feature to change the implementation independently of the interface. This will help when dealing with changing requirements.
- The hotel charges each customer 1000.85/- per day. This amount is being decided by the hotel committee and cannot be changed fulfilling certain complex formalities.
- The module then analyses the number of days. If the customer has stayed for more than a week in the hotel, he gets discount on the rent. Otherwise, he is being charged normally.
- The discounted rent is being calculated after subtracting one day from the total number of days.
- In the end, the module displays the following details:
  - o Customer name
  - o Days
  - o Rent

Note that, the function used for displaying purpose must not have the ability to modify any data member.

**Source Code:**

```
#include <iostream>
#include <string>
using namespace std;

class HotelMercato {
private:
    string customerName;
    int daysStayed;
    static const double dailyRent;

public:
    HotelMercato(string name, int days): customerName(name), daysStayed(days) { }

    double calculateRent() const {
        int billableDays = daysStayed;

        if (daysStayed > 7) {
            billableDays -= 1;
        }

        return billableDays * dailyRent;
    }

    void display() const {
        cout << "Customer Name: " << customerName << endl;
        cout << "Days Stayed: " << daysStayed << endl;
        cout << "Total Rent: " << calculateRent() << endl;
    }
};

const double HotelMercato::dailyRent = 1000.85;
```

```
const double HotelMercato::dailyRent = 1000.85;

int main() {
    HotelMercato customer1("Alice", 5);
    HotelMercato customer2("Bob", 10);

    cout << "Customer 1 Details:" << endl;
    customer1.display();
    cout << endl;

    cout << "Customer 2 Details:" << endl;
    customer2.display();
    cout << endl;

    return 0;
}
```

**Output:**

```
Customer 1 Details:
Customer Name: Alice
Days Stayed: 5
Total Rent: 5004.25

Customer 2 Details:
Customer Name: Bob
Days Stayed: 10
Total Rent: 9007.65

-----
Process exited after 0.2785 seconds with return value 0
Press any key to continue . . . |
```

**Example 01:****Source Code:**

```
#include<iostream>
using namespace std;

class example{
private:
    int x;
public:
    void set(int x){
        (*this).x = x;
    }
    int get( ){
        return x;
    }
    void printAddressAndValue( ){
        cout << "The address is "<<this<<"and the value is "<<(*this).x<<endl;
    }
};

int main(){
    example obj1, obj2;
    obj1.set(7);
    obj1.printAddressAndValue();
    obj2.set(10);
    obj2.printAddressAndValue();
    return 0;
}
```

**Output:**

```
The address is 0x78fe4cand the value is 7
The address is 0x78fe48and the value is 10

-----
Process exited after 0.3547 seconds with return value 0
Press any key to continue . . . |
```

Example 02:Source Code:

```
#include<iostream>
using namespace std;

class Test{
private:
    int x;
    int y;
public:
    Test(int x = 0, int y = 0){
        this->x = x;
        this->y = y;
    }
    void print( ){
        cout<<"x="<<x<<" y="<<y <<endl;
    }
};

int main( ){
    Test obj1(10, 20);
    obj1.print( );
}
```

Output:

```
x=10 y=20

-----
Process exited after 0.3309 seconds with return value 0
Press any key to continue . . . |
```

Example 03:Source Code:

```
#include<iostream>
using namespace std;

class Point {
private:
    int x;
    int y;
public:
    Point(int i = 0, int j = 0):x(i), y(j) { }
    int getX( ) const { return x; }
    int getY( ) const { return y; }
};

int main( ) {
    Point t1(10, 15);
    cout<<"x = "<<t1.getX( )<<" , ";
    cout<<"y = "<<t1.getY( );
    return 0;
}
```

Output:

```
x = 10, y = 15
-----
Process exited after 0.3493 seconds with return value 0
Press any key to continue . . . |
```

**Example 04:****Source Code:**

```
#include <iostream>
using namespace std;

class Students{
private:
    string name;
    const int rollno;
    float cgpa;

public:
    Students (int rno): rollno(rno){ }

    void set(string sname, float cg){
        name = sname;
        cgpa = cg;
    }

    void print( ){
        cout<<"Name: "<<name<<"", Roll #"<<rollno<<"", CGPA, "<<cgpa<<endl;
    }
};

int main( ){
    Students s(12);
    s.set("Ahmad",3.67);
    s.print( );
}
```

**Output:**

```
Name: Ahmad, Roll #12, CGPA, 3.67
```

```
-----
Process exited after 0.322 seconds with return value 0
Press any key to continue . . . |
```

Example 05:Source Code:

```
#include<iostream>
using namespace std;

class test{
private:
    int a;
public:
    int nonconstFuction(int a){
        cout<<"Non Constant Function is called"<<endl;
        a = a + 10;
        return a;
    }

    int constFuction(int a) const{
        return a;
    }
};

main( ){
    test t;
    cout<<"Constant Function is called"<<endl;
    cout<<t.nonconstFuction(10)<<endl;
    cout<<t.constFuction(30);
    return 0;
}
```

Output:

```
Constant Function is called
Non Constant Function is called
20
30
-----
Process exited after 0.3434 seconds with return value 0
Press any key to continue . . .
```

Example 06:

Source Code:

```

#include<iostream>
using namespace std;

class test{
public:
    int a;

    test( ){
        a = 8;
    }

    int nonconstFuction( ) const{
        cout<<"Non Constant Function is called "<<endl;
        // a = a + 10; // error: cannot modify data member in const function
        return a;
    }

    int constFuction(int a) const{
        // this->a = a + 10; // error: const function cannot modify object
        return a + 10;
    }
};

int main( ){
    const test t;

    cout<<"Constant Function is called"<<endl;

    // t.a = 10; // error: cannot modify data member of const object

    // cout << t.nonconstFuction(); // error (before fix): non-const function call on const object
    cout << t.nonconstFuction() << endl;

int main( ){
    const test t;

    cout<<"Constant Function is called"<<endl;

    // t.a = 10; // error: cannot modify data member of const object

    // cout << t.nonconstFuction(); // error (before fix): non-const function call on const object
    cout << t.nonconstFuction() << endl;

    cout << t.constFuction(10);

    return 0;
}

```

Output:

```
Constant Function is called
Non Constant Function is called
8
20
-----
Process exited after 0.3251 seconds with return value 0
Press any key to continue . . . |
```

**Example 07:****Source Code:**

```
#include <iostream>
using namespace std;

class Demo{
private:
    static int x;
public:
    static void fun( ){
        cout<<"value of X: " << x << endl;
    }
};

//defining
int Demo::x = 10;

int main( ){
    Demo X;
    X.fun( );
    return 0;
}
```

**Output:**

```
value of X: 10
-----
Process exited after 0.2857 seconds with return value 0
Press any key to continue . . . |
```

**Example 08:**

Source Code:

```
#include <iostream>
using namespace std;

class Demo{
public:
    static int ABC;
};

//defining
int Demo::ABC = 10;

int main( ){
    cout<<"\nValue of ABC: "<<Demo::ABC;
    return 0;
}
```

Output:

```
Value of ABC: 10
-----
Process exited after 0.3586 seconds with return value 0
Press any key to continue . . . |
```

# **LAB 05**

**OBJECT ORIENTED PROGRAMMING (CT-260)**

**Question 01:** Create a base class with the following members:

- Private integer privateInt
- Protected integer protectedInt
- Public integer publicInt

Create getters and setters for each of these variables. Derive 3 classes from the base class with the three types of inheritance based on visibility(public, protected, private). You can name these classes as publicChild, privateChild and protectedChild. After doing this, try and figure out which member you can access publicly or through getters and setters. Then print out the way that you were able to access them. For example, if you did private inheritance, you could not be able to access the members in the child classes, and would need to use their getters or setters.

**Source Code:**

```
#include <iostream>
using namespace std;

class Base {
private:
    int privateInt;

protected:
    int protectedInt;

public:
    int publicInt;

    void setPrivate(int x) {
        privateInt = x;
    }

    int getPrivate() const {
        return privateInt;
    }

    void setProtected(int x) {
        protectedInt = x;
    }

    int getProtected() const {
        return protectedInt;
    }

    void setPublic(int x) {
        publicInt = x;
    }

    int getPublic() const {
        return publicInt;
    }
};
```

```
class publicChild : public Base {
public:
    void showAccess() {
        protectedInt = 20;
        publicInt = 30;

        cout << "publicChild Access:" << endl;
        cout << "Protected accessed directly: " << protectedInt << endl;
        cout << "Public accessed directly: " << publicInt << endl;
        cout << "Private accessed using getter: " << getPrivate() << endl;
    }
};

class protectedChild : protected Base {
public:
    void setValue(int x) {
        setPrivate(x);
    }

    void showAccess() {
        protectedInt = 40;
        publicInt = 50;

        cout << "\nprotectedChild Access:" << endl;
        cout << "Protected accessed directly: " << protectedInt << endl;
        cout << "Public accessed directly (now protected): " << publicInt << endl;
        cout << "Private accessed using getter: " << getPrivate() << endl;
    }
};
```

```
class privateChild : private Base {
public:
    void setValue(int x) {
        setPrivate(x);
    }

    void showAccess() {
        protectedInt = 60;
        publicInt = 70;

        cout << "\nprivateChild Access:" << endl;
        cout << "Protected accessed directly: " << protectedInt << endl;
        cout << "Public accessed directly (now private): " << publicInt << endl;
        cout << "Private accessed using getter: " << getPrivate() << endl;
    }
};

int main() {
    publicChild obj1;
    obj1.setPrivate(10);
    obj1.showAccess();
    cout << "Accessing publicInt directly from object: " << obj1.publicInt << endl;

    protectedChild obj2;
    obj2.setValue(15);
    obj2.showAccess();
    // obj2.publicInt; // Not accessible outside

    privateChild obj3;
    obj3.setValue(25);
    obj3.showAccess();
    // obj3.publicInt; // Not accessible outside

    return 0;
}
```

Output:

```
publicChild Access:
Protected accessed directly: 20
Public accessed directly: 30
Private accessed using getter: 10
Accessing publicInt directly from object: 30

protectedChild Access:
Protected accessed directly: 40
Public accessed directly (now protected): 50
Private accessed using getter: 15

privateChild Access:
Protected accessed directly: 60
Public accessed directly (now private): 70
Private accessed using getter: 25

-----
Process exited after 0.3034 seconds with return value 0
Press any key to continue . . .
```

**Question 02:** Create a class Teacher with the following private attributes: Name, Age, and Institute. Derive three classes from it that has the following names: HumanitiesTeacher, ScienceTeacher, MathsTeacher. These classes should have the following members:

- Department (this should have the value "science", "maths" or "humanities")
- Course Name
- Designation (for example, lecturer, professor, etc)

Create proper accessors and mutators for the attributes. Create objects for each of the classes and display the values. You can ask the user to input the values.

**Source Code:**

```
#include <iostream>
using namespace std;

class Teacher {
private:
    string Name;
    int Age;
    string Institute;

public:
    void setName(string n) { Name = n; }
    void setAge(int a) { Age = a; }
    void setInstitute(string i) { Institute = i; }

    string getName() const { return Name; }
    int getAge() const { return Age; }
    string getInstitute() const { return Institute; }
};

class HumanitiesTeacher : public Teacher {
private:
    string Department = "Humanities";
    string CourseName;
    string Designation;

public:
    void setCourse(string c) { CourseName = c; }
    void setDesignation(string d) { Designation = d; }

    string getDepartment() const { return Department; }
    string getCourse() const { return CourseName; }
    string getDesignation() const { return Designation; }
```

```
void display() const {
    cout << "\n--- Humanities Teacher ---\n";
    cout << "Name: " << getName() << endl;
    cout << "Age: " << getAge() << endl;
    cout << "Institute: " << getInstitute() << endl;
    cout << "Department: " << Department << endl;
    cout << "Course: " << CourseName << endl;
    cout << "Designation: " << Designation << endl;
}

};

class ScienceTeacher : public Teacher {
private:
    string Department = "Science";
    string CourseName;
    string Designation;

public:
    void setCourse(string c) { CourseName = c; }
    void setDesignation(string d) { Designation = d; }

    string getDepartment() const { return Department; }
    string getCourse() const { return CourseName; }
    string getDesignation() const { return Designation; }

    void display() const {
        cout << "\n--- Science Teacher ---\n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: " << Department << endl;
        cout << "Course: " << CourseName << endl;
        cout << "Designation: " << Designation << endl;
    }
};
```

```
class MathsTeacher : public Teacher {
private:
    string Department = "Maths";
    string CourseName;
    string Designation;

public:
    void setCourse(string c) { CourseName = c; }
    void setDesignation(string d) { Designation = d; }

    string getDepartment() const { return Department; }
    string getCourse() const { return CourseName; }
    string getDesignation() const { return Designation; }

    void display() const {
        cout << "\n--- Maths Teacher ---\n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: " << Department << endl;
        cout << "Course: " << CourseName << endl;
        cout << "Designation: " << Designation << endl;
    }
};

int main() {
    HumanitiesTeacher h;
    ScienceTeacher s;
    MathsTeacher m;

    string name, institute, course, designation;
    int age;

    cout << "Enter Humanities Teacher Details\n";
    cout << "Name: "; cin >> name;
    cout << "Age: "; cin >> age;
    cout << "Institute: "; cin >> institute;
    cout << "Course: "; cin >> course;
    cout << "Designation: "; cin >> designation;

    h.setName(name);
    h.setAge(age);
    h.setInstitute(institute);
    h.setCourse(course);
    h.setDesignation(designation);

    cout << "\nEnter Science Teacher Details\n";
    cout << "Name: "; cin >> name;
    cout << "Age: "; cin >> age;
    cout << "Institute: "; cin >> institute;
    cout << "Course: "; cin >> course;
    cout << "Designation: "; cin >> designation;

    s.setName(name);
    s.setAge(age);
    s.setInstitute(institute);
    s.setCourse(course);
```

```
s.setName(name);
s.setAge(age);
s.setInstitute(institute);
s.setCourse(course);
s.setDesignation(designation);

cout << "\nEnter Maths Teacher Details\n";
cout << "Name: "; cin >> name;
cout << "Age: "; cin >> age;
cout << "Institute: "; cin >> institute;
cout << "Course: "; cin >> course;
cout << "Designation: "; cin >> designation;

m.setName(name);
m.setAge(age);
m.setInstitute(institute);
m.setCourse(course);
m.setDesignation(designation);

h.display();
s.display();
m.display();

return 0;
}
```

Output:

```
Enter Humanities Teacher Details
Name: Sana
Age: 25
Institute: NED
Course: Humanities
Designation: Lecturer

Enter Science Teacher Details
Name: Ayesha
Age: 29
Institute: NED
Course: Science
Designation: Professor

Enter Maths Teacher Details
Name: Arsalan
Age: 34
Institute: NED
Course: Mathematics
Designation: Professor

--- Humanities Teacher ---
Name: Sana
Age: 25
Institute: NED
Department: Humanities
Course: Humanities
Designation: Lecturer

--- Science Teacher ---
Name: Ayesha
Age: 29
Institute: NED
Department: Science
Course: Science
Designation: Professor

--- Maths Teacher ---
Name: Arsalan
Age: 34
Institute: NED
Department: Maths
Course: Mathematics
Designation: Professor

-----
Process exited after 68.32 seconds with return value 0
Press any key to continue . . .
```

**Question 03:** A defense organization is making a hierarchy of different types of weapons. They have classified the nuclear bomb as follows: Weapons → Hot Weapons → Bombs → Nuclear Bombs. Create classes and apply inheritance as necessary for the above hierarchy. Each class should have a method called: "xxxxxDescription", where xxxx would be class name. The method should print out what that weapon does. Eg. Hot weapons uses gunpowder, or explode. Bombs blow up. Nuclear bombs blow up, and use nuclear fission and fusion.

**Source Code:**

```
#include <iostream>
using namespace std;

class Weapons {
public:
    void WeaponsDescription() {
        cout << "Weapons are used for defense and warfare." << endl;
    }
};

class HotWeapons : public Weapons {
public:
    void HotWeaponsDescription() {
        cout << "Hot weapons use heat, fire, or gunpowder to cause damage." << endl;
    }
};

class Bombs : public HotWeapons {
public:
    void BombsDescription() {
        cout << "Bombs explode and cause large-scale destruction." << endl;
    }
};

class NuclearBombs : public Bombs {
public:
    void NuclearBombsDescription() {
        cout << "Nuclear bombs explode using nuclear fission and fusion, causing massive destruction." << endl;
    }
};

int main() {
    NuclearBombs nb;

    nb.WeaponsDescription();
    nb.HotWeaponsDescription();
    nb.BombsDescription();
    nb.NuclearBombsDescription();

    return 0;
}
```

**Output:**

```
Weapons are used for defense and warfare.
Hot weapons use heat, fire, or gunpowder to cause damage.
Bombs explode and cause large-scale destruction.
Nuclear bombs explode using nuclear fission and fusion, causing massive destruction.

-----
Process exited after 0.2839 seconds with return value 0
Press any key to continue . . . |
```

**Question 04:** A bakery is making a program to calculate the bills for each of their customers. The items in the bakery are categorized as follows:

Item:

- Name

- Quantity

Baked Goods: (derived from item)

- Discount = 10%

Cakes: (derived from baked goods)

- Price = 600

Bread: (derived from baked goods)

- Price = 200

Drinks: (derived from item)

- Discount = 5%

- Price = 100

In your main function, as a bakery personnel, input the items and their quantities and calculate the bill accordingly.

**Source Code:**

```
#include <iostream>
#include <iomanip>
using namespace std;

class Item {
protected:
    string Name;
    int Quantity;

public:
    void setItem(string n, int q) {
        Name = n;
        Quantity = q;
    }

    int getQuantity() const { return Quantity; }
    string getName() const { return Name; }

    virtual double calculatePrice() const = 0;
};
```

```
class BakedGoods : public Item {
protected:
    double discount = 0.10;
};

class Cakes : public BakedGoods {
private:
    double price = 600;

public:
    double calculatePrice() const override {
        double total = price * Quantity;
        total -= total * discount;
        return total;
    }
};

class Bread : public BakedGoods {
private:
    double price = 200;

public:
    double calculatePrice() const override {
        double total = price * Quantity;
        total -= total * discount;
        return total;
    }
};
```

```

class Drinks : public Item {
private:
    double price = 100;
    double discount = 0.05;

public:
    double calculatePrice() const override {
        double total = price * Quantity;
        total -= total * discount;
        return total;
    }
};

int main() {
    cout << fixed << setprecision(2);

    Cakes cake;
    Bread bread;
    Drinks drink;

    int q;

    cout << "Enter quantity of cakes: ";
    cin >> q;
    cake.setItem("Cake", q);

    cout << "Enter quantity of bread: ";
    cin >> q;
    bread.setItem("Bread", q);

    cout << "Enter quantity of drinks: ";
    cin >> q;
    drink.setItem("Drinks", q);


    cout << "Enter quantity of drinks: ";
    cin >> q;
    drink.setItem("Drinks", q);

    double totalBill = cake.calculatePrice() +
        bread.calculatePrice() +
        drink.calculatePrice();

    cout << "\n--- BILL ---\n";
    cout << cake.getName() << " (Quantity: " << cake.getQuantity()
        << ") : " << cake.calculatePrice() << endl;

    cout << bread.getName() << " (Quantity: " << bread.getQuantity()
        << ") : " << bread.calculatePrice() << endl;

    cout << drink.getName() << " (Quantity: " << drink.getQuantity()
        << ") : " << drink.calculatePrice() << endl;

    cout << "Total Bill: " << totalBill << endl;

    return 0;
}

```

Output:

```
Enter quantity of cakes: 12
Enter quantity of bread: 24
Enter quantity of drinks: 25

--- BILL ---
Cake (Quantity: 12) : 6480.00
Bread (Quantity: 24) : 4320.00
Drinks (Quantity: 25) : 2375.00
Total Bill: 13175.00

-----
Process exited after 10.34 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include <iostream>
using namespace std;

class base { // single base class
public:
    int x;

    void getdata() {
        cout << "Enter the value of x = ";
        cin >> x;
    }
};

class derive : public base { // single derived class
private:
    int y;
public:
    void readdata() {
        cout << "Enter the value of y = ";
        cin >> y;
    }

    void product() {
        cout << "Product = " << x * y;
    }
};

int main() {
    derive a; // object of derived class

    a.getdata();
    a.readdata();
    a.product();

    return 0;
} //end program
```

**Output:**

```
Enter the value of x = 56
Enter the value of y = 5
Product = 280
-----
Process exited after 5.963 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:****Source Code:**

```
#include <iostream>
using namespace std;

class base { // single base class
public:
    int x;

    void getdata() {
        cout << "Enter value of x= ";
        cin >> x;
    }
};

class derive1 : public base { // derived class from base class
public:
    int y;

    void readdata() {
        cout << "\nEnter value of y= ";
        cin >> y;
    }
};

class derive2 : public derive1 { // derived from class derive1
private:
    int z;

public:
    void indata() {
        cout << "\nEnter value of z= ";
        cin >> z;
    }

    void product() {
        cout << "\nProduct= " << x * y * z;
    }
};

int main() {
    derive2 a; // object of derived class

    a.getdata();
    a.readdata();
    a.indata();
    a.product();

    return 0;
} //end program
```

**Output:**

```
Enter value of x= 6

Enter value of y= 7

Enter value of z= 9

Product= 378

-----
Process exited after 5.842 seconds with return value 0
Press any key to continue . . .
```

**Example 03:****Source Code:**

```
#include <iostream>
using namespace std;

class A { // single base class
public:
    int x, y;

    void getdata() {
        cout << "\nEnter value of x and y:\n";
        cin >> x >> y;
    }
};

class B : public A { // B is derived from class A
public:
    void product() {
        cout << "\nProduct= " << x * y;
    }
};

class C : public A { // C is also derived from class A
public:
    void sum() {
        cout << "\nSum= " << x + y;
    }
};

int main() {
    B obj1; // object of derived class B
    C obj2; // object of derived class C

    obj1.getdata();
    obj1.product();

    obj2.getdata();
    obj2.sum();

    return 0;
} //end program
```

**Output:**

```
Enter value of x and y:
5 6

Product= 30
Enter value of x and y:
5 8

Sum= 13

-----
Process exited after 6.297 seconds with return value 0
Press any key to continue . . .
```

## Difference Between Multilevel and Multiple Inheritance:

### 01: Multilevel Inheritance:

#### Definition:

Multilevel inheritance occurs when a class is derived from another derived class, creating a linear "chain" of inheritance. In this structure, a child class becomes the parent for the next generation.

#### Key Differences:

- **Hierarchy:** It follows a vertical, linear structure (Grandparent -> Parent -> Child).
- **Complexity:** Very low complexity; it is easy to read and maintain.
- **Relationship:** Represents a direct lineage or specialized "levels" of an object.
- **Data Flow:** The bottom-most class has access to all public/protected members of every class above it in the chain.

#### Code Example:

##### Source Code:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    void fuel() { cout << "Vehicles need fuel." << endl; }
};

class Car : public Vehicle {
public:
    void wheels() { cout << "Cars have 4 wheels." << endl; }
};

class SportsCar : public Car {
public:
    void speed() { cout << "Sports cars go fast!" << endl; }
};

int main() {
    SportsCar myCar;
    myCar.fuel();
    myCar.wheels();
    myCar.speed();
    return 0;
}
```

Output:

```
Vehicles need fuel.  
Cars have 4 wheels.  
Sports cars go fast!  
  
-----  
Process exited after 0.2192 seconds with return value 0  
Press any key to continue . . .
```

## 2. Multiple Inheritance:

### Definition:

Multiple inheritance is a feature where a single derived class can inherit features from more than one base class simultaneously. This allows a class to combine behaviors from different sources.

### Key Differences:

- **Hierarchy:** It follows a horizontal or convergent structure (Parent A + Parent B -> Child).
- **Complexity:** Higher complexity; can lead to name conflicts or the "Diamond Problem."
- **Relationship:** Represents a combination of distinct traits from unrelated parents.
- **Data Flow:** The child class collects features from all its parents at the same level.

Code Example:

Source Code:

```
#include <iostream>
using namespace std;

class Teacher {
public:
    void teach() { cout << "I can teach lessons." << endl; }
};

class Author {
public:
    void write() { cout << "I can write books." << endl; }
};

class Scholar : public Teacher, public Author {
public:
    void research() { cout << "I am doing research." << endl; }
};

int main() {
    Scholar myScholar;
    myScholar.teach();
    myScholar.write();
    myScholar.research();
    return 0;
}
```

Output:

```
I can teach lessons.
I can write books.
I am doing research.

-----
Process exited after 0.2281 seconds with return value 0
Press any key to continue . . . |
```

# **LAB 06**

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include<iostream>
using namespace std;
class Shape{
protected:
    int NoOfSides;
    double area;

public:
    Shape(int s):NoOfSides(s), area(0) {}

    int getNoOfSides(){
        return NoOfSides;
    }

    double getarea(){
        return area;
    }

    void setNoOfSides(int s){
        NoOfSides = s;
    }

    void setarea(double a){
        area = a;
    }
};

class Rectangle : public Shape{
protected:
    double length;
    double width;

public:
    Rectangle(double l, double w) : Shape(4), length(l), width(w) {}
}
```

```
class Rectangle : public Shape{
protected:
    double length;
    double width;

public:
    Rectangle(double l, double w) : Shape(4), length(l), width(w) {}

    void generateArea(){
        area = length * width;
    }
};

class Circle : public Shape{
private:
    double radius;

public:
    Circle(double r) : Shape(1), radius(r) {}

    void generateArea(){
        area = 3.1416 * radius * radius;
    }
};

class Triangle : public Shape{
private:
    double height;
    double base;

public:
    Triangle(double h, double b) : Shape(3), height(h), base(b) {}

    void generateArea(){
        area = (height * base) / 2;
    }
};
```

---

```
class Triangle : public Shape{
private:
    double height;
    double base;

public:
    Triangle(double h, double b) : Shape(3), height(h), base(b) {}

    void generateArea(){
        area = (height * base) / 2;
    }
};

class Square : public Rectangle{
public:
    Square(double side) : Rectangle(side, side) {}

    void checkSides(){
        if(length == width)
            cout << "Both sides are equal." << endl;
        else
            cout << "Sides are not equal." << endl;
    }

    void generateArea(){
        area = length * width;
    }
};

int main(){
    Rectangle r(5, 4);
    r.generateArea();
    cout << "Rectangle Area: " << r.getarea() << endl;

    Circle c(3);
    c.generateArea();
    cout << "Circle Area: " << c.getarea() << endl;

    Triangle t(6, 4);
    t.generateArea();
    cout << "Triangle Area: " << t.getarea() << endl;

    Square s(5);
    s.checkSides();
    s.generateArea();
    cout << "Square Area: " << s.getarea() << endl;

    return 0;
}
```

Output:

```
Rectangle Area: 20
Circle Area: 28.2744
Triangle Area: 12
Both sides are equal.
Square Area: 25

-----
Process exited after 0.3051 seconds with return value 0
Press any key to continue . . .
```

Question 02:Source Code:

```
#include<iostream>
using namespace std;
class Calculator{
private:
    double Num1;
    double Num2;
    double Num3;

public:
    Calculator(double n1, double n2, double n3) : Num1(n1), Num2(n2), Num3(n3) {}

    double add(double a, double b){
        return a + b;
    }

    double add(double a, double b, double c){
        return a + b + c;
    }

    double multiply(double a, double b){
        return a * b;
    }

    double multiply(double a, double b, double c){
        return a * b * c;
    }

    double subtract(){
        return Num1 - Num2;
    }
}
```

```
        double divide(){
            if(Num2 != 0)
                return Num1 / Num2;
            else{
                cout << "Division by zero not allowed." << endl;
                return 0;
            }
        }
    };

int main(){
    double n1, n2, n3;
    cout << "Enter three numbers: ";
    cin >> n1 >> n2 >> n3;
    Calculator calc(n1, n2, n3);

    cout << "Addition (2 numbers): " << calc.add(n1, n2) << endl;
    cout << "Addition (3 numbers): " << calc.add(n1, n2, n3) << endl;

    cout << "Multiplication (2 numbers): " << calc.multiply(n1, n2) << endl;
    cout << "Multiplication (3 numbers): " << calc.multiply(n1, n2, n3) << endl;

    cout << "Subtraction (Num1 - Num2): " << calc.subtract() << endl;
    cout << "Division (Num1 / Num2): " << calc.divide() << endl;

    return 0;
}
```

**Output:**

```
Enter three numbers: 3 4 5
Addition (2 numbers): 7
Addition (3 numbers): 12
Multiplication (2 numbers): 12
Multiplication (3 numbers): 60
Subtraction (Num1 - Num2): -1
Division (Num1 / Num2): 0.75

-----
Process exited after 4.562 seconds with return value 0
Press any key to continue . . . |
```

**Question 03:****Source Code:**

```
#include<iostream>
#include<cmath>
using namespace std;
class Vector{
    private:
        double x;
        double y;

    public:

        Vector(double xVal, double yVal) : x(xVal), y(yVal) {}

        Vector operator + (const Vector& obj){
            return Vector(x + obj.x, y + obj.y);
        }

        Vector operator - (const Vector& obj){
            return Vector(x - obj.x, y - obj.y);
        }

        Vector operator * (double scalar){
            return Vector(x * scalar, y * scalar);
        }

        Vector operator / (double scalar){
            if(scalar != 0)
                return Vector(x / scalar, y / scalar);
            else{
                cout << "Division by zero not allowed." << endl;
                return Vector(0, 0);
            }
        }

        double magnitude(){
            return sqrt(x*x + y*y);
        }
}
```

```
        void display(){
            cout << "(" << x << ", " << y << ")" << endl;
        }
};

int main(){
    Vector v1(3, 4);
    Vector v2(1, 2);

    Vector sum = v1 + v2;
    Vector diff = v1 - v2;
    Vector mul = v1 * 2;
    Vector div = v1 / 2;

    cout << "Vector 1: ";
    v1.display();

    cout << "Vector 2: ";
    v2.display();

    cout << "Addition: ";
    sum.display();

    cout << "Subtraction: ";
    diff.display();

    cout << "Scalar Multiplication: ";
    mul.display();

    cout << "Scalar Division: ";
    div.display();

    cout << "Magnitude of Vector 1: " << v1.magnitude() << endl;

    return 0;
}
```

**Output:**

```
Vector 1: (3, 4)
Vector 2: (1, 2)
Addition: (4, 6)
Subtraction: (2, 2)
Scalar Multiplication: (6, 8)
Scalar Division: (1.5, 2)
Magnitude of Vector 1: 5

-----
Process exited after 0.2794 seconds with return value 0
Press any key to continue . . . |
```

**Question 04:****Source Code:**

```
#include <iostream>
#include <string>
using namespace std;
class Person {
protected:
    string name;
    string occupation;
public:
    Person(string _occupation) : occupation(_occupation) {}

    virtual void Draw() {
        cout << "A person can draw in many ways." << endl;
    }
};

class Artist : public Person {
public:
    Artist() : Person("Artist") {}

    void Draw() {
        cout << "An artist can draw with a paint brush." << endl;
    }
};

class Gunman : public Person {
public:
    Gunman() : Person("Gunman") {}

    void Draw() {
        cout << "A gunman draws a gun to shoot." << endl;
    }
};

int main() {
    Person* people[3];

    people[0] = new Person("Person");
    people[1] = new Artist();
    people[2] = new Gunman();

    for (int i = 0; i < 3; i++) {
        people[i]->Draw();
    }

    for (int i = 0; i < 3; i++) {
        delete people[i];
    }

    return 0;
}
```

Output:

```
A person can draw in many ways.
An artist can draw with a paint brush.
A gunman draws a gun to shoot.

-----
Process exited after 0.2657 seconds with return value 0
Press any key to continue . . . |
```

Example 01:Source Code:

```
#include<iostream>
using namespace std;
class overload{
public:
    // function with 1 int parameter
    void func(int x){
        cout << "value of x is " << x << endl;
    }

    // function with 1 double parameter
    void func(double x){
        cout << "value of x is " << x << endl;
    }

    // function with 2 int parameters
    void func(int x, int y){
        cout << "value of x and y is " << x << ", " << y << endl;
    }
};

int main(){
    overload obj1;

    // Calls func(int)
    obj1.func(7);

    // Calls func(double)
    obj1.func(9.132);

    // Calls func(int, int)
    obj1.func(85, 64);

    return 0;
}
```

Output:

```
value of x is 7
value of x is 9.132
value of x and y is 85, 64

-----
Process exited after 0.2772 seconds with return value 0
Press any key to continue . . . |
```

Example 02:Source Code:

```
#include<iostream>
using namespace std;

class Complex{
private:
    float real;
    float imag;

public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}

    void input(){
        cout << "Enter real and imaginary parts respectively: ";
        cin >> real;
        cin >> imag;
    }

    // Overload + operator
    Complex operator + (const Complex& obj){
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    void display(){
        cout << real << " + " << imag << "i" << endl;
    }
};
```

•

```
int main(){
    Complex c1, c2, result;

    cout << "Enter the first complex number:" << endl;
    c1.input();

    cout << "Enter the second complex number:" << endl;
    c2.input();

    result = c1 + c2;

    cout << "Sum of the two complex numbers: ";
    result.display();

    return 0;
}
```

**Output:**

```
Enter the first complex number:
Enter real and imaginary parts respectively: 45 34
Enter the second complex number:
Enter real and imaginary parts respectively: 67 89
Sum of the two complex numbers: 112 + 123i

-----
Process exited after 9.859 seconds with return value 0
Press any key to continue . . .
```

**Example 03:****Source Code:**

```
#include<iostream>
using namespace std;

class BaseClass{
public:
    void disp(){
        cout << "Function of Parent Class";
    }
};

class DerivedClass : public BaseClass{
public:
    void disp(){
        cout << "Function of Child Class";
    }
};

int main(){
    DerivedClass obj = DerivedClass();
    obj.disp();
    return 0;
}
```

**Output:**

```
Function of Child Class
-----
Process exited after 0.2554 seconds with return value 0
Press any key to continue . . . |
```

**Example 04:****Source Code:**

```
#include<iostream>
using namespace std;

class BaseClass{
public:
    virtual void Display(){
        cout << "\nThis is Display() method of BaseClass";
    }

    void Show(){
        cout << "\nThis is Show() method of BaseClass";
    }
};

class DerivedClass : public BaseClass{
public:
    // Overriding method
    void Display(){
        cout << "\nThis is Display() method of DerivedClass";
    }
};

int main(){
    DerivedClass dr;
    BaseClass &bs = dr;

    bs.Display();    // Calls Derived version (Runtime Polymorphism)
    dr.Show();       // Calls Base version (Not virtual)

    return 0;
}
```

**Output:**

```
This is Display() method of DerivedClass
This is Show() method of BaseClass
-----
Process exited after 0.2875 seconds with return value 0
Press any key to continue . . . |
```

# LAB 07

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include<iostream>
#include<cstdlib>
using namespace std;

class Array2D{
    int rows, cols;
    int **ptr;

public:
    Array2D(int r = 0, int c = 0) : rows(r), cols(c){
        if(rows > 0 && cols > 0){
            ptr = new int*[rows];
            for(int i = 0; i < rows; i++){
                ptr[i] = new int[cols];
            }
        }
        else{
            ptr = NULL;
        }
    }

    Array2D(const Array2D& ob){
        rows = ob.rows;
        cols = ob.cols;
        if(rows > 0 && cols > 0){
            ptr = new int*[rows];
            for(int i = 0; i < rows; i++){
                ptr[i] = new int[cols];
                for(int j = 0; j < cols; j++){
                    ptr[i][j] = ob.ptr[i][j];
                }
            }
        }
        else{
            ptr = NULL;
        }
    }
}
```

```

Array2D& operator=(const Array2D& ob){
    if(this != &ob){
        for(int i = 0; i < rows; i++) delete[] ptr[i];
        delete[] ptr;

        rows = ob.rows;
        cols = ob.cols;
        ptr = new int*[rows];
        for(int i = 0; i < rows; i++){
            ptr[i] = new int[cols];
            for(int j = 0; j < cols; j++){
                ptr[i][j] = ob.ptr[i][j];
            }
        }
    }
    return *this;
}

int* operator[](int index){
    if(index >= rows || index < 0){
        cout << "Row index " << index << " out of bounds" << endl;
        exit(1);
    }
    return ptr[index];
}

Array2D operator+(const Array2D& ob){
    if(rows == ob.rows && cols == ob.cols){
        Array2D temp(rows, cols);
        for(int i = 0; i < rows; i++){
            for(int j = 0; j < cols; j++){
                temp[i][j] = ptr[i][j] + ob.ptr[i][j];
            }
        }
        return temp;
    }
    else{
        cout << "Dimension mismatch for addition!" << endl;
        exit(1);
    }
}

```

```

Array2D operator-(const Array2D& ob){
    if(rows == ob.rows && cols == ob.cols){
        Array2D temp(rows, cols);
        for(int i = 0; i < rows; i++){
            for(int j = 0; j < cols; j++){
                temp[i][j] = ptr[i][j] - ob.ptr[i][j];
            }
        }
        return temp;
    }
    else{
        cout << "Dimension mismatch for subtraction!" << endl;
        exit(1);
    }
}

Array2D operator*(const Array2D& ob){
    if(cols == ob.rows){
        Array2D temp(rows, ob.cols);
        for(int i = 0; i < rows; i++){
            for(int j = 0; j < ob.cols; j++){
                temp[i][j] = 0;
                for(int k = 0; k < cols; k++){
                    temp[i][j] += ptr[i][k] * ob.ptr[k][j];
                }
            }
        }
        return temp;
    }
    else{
        cout << "Dimension mismatch for multiplication!" << endl;
        exit(1);
    }
}

~Array2D(){
    for(int i = 0; i < rows; i++){
        delete[] ptr[i];
    }
    delete[] ptr;
}

void display(){
    for(int i = 0; i < rows; i++){
        for(int j = 0; j < cols; j++){
            cout << ptr[i][j] << "\t";
        }
        cout << endl;
    }
}
};

```

```
int main(){
    Array2D m1(2, 2), m2(2, 2);

    for(int i = 0; i < 2; i++){
        for(int j = 0; j < 2; j++){
            m1[i][j] = i + j + 1;
            m2[i][j] = (i + 1) * 2;
        }
    }

    cout << "Matrix 1:" << endl;
    m1.display();

    cout << "Matrix 2:" << endl;
    m2.display();

    Array2D mSum = m1 + m2;
    cout << "Matrix Sum (m1 + m2):" << endl;
    mSum.display();

    Array2D mProd = m1 * m2;
    cout << "Matrix Product (m1 * m2):" << endl;
    mProd.display();

    return 0;
}
```

### Output:

```
Matrix 1:
1      2
2      3
Matrix 2:
2      2
4      4
Matrix Sum (m1 + m2):
3      4
6      7
Matrix Product (m1 * m2):
10     10
16     16

-----
Process exited after 0.3317 seconds with return value 0
Press any key to continue . . .
```

**Question 02:****Source Code:**

```
#include<iostream>
#include<cstdlib>
using namespace std;

class Array2D{
    int rows, cols;
    int **ptr;

public:
    Array2D(int r = 0, int c = 0) : rows(r), cols(c){
        if(rows > 0 && cols > 0){
            ptr = new int*[rows];
            for(int i = 0; i < rows; i++){
                ptr[i] = new int[cols];
            }
        }
        else{
            ptr = NULL;
        }
    }

    int* operator[](int index){
        return ptr[index];
    }

    bool search(int target){
        if(ptr == NULL || rows == 0 || cols == 0) return false;

        int low = 0;
        int high = (rows * cols) - 1;

        while(low <= high){
            int mid = low + (high - low) / 2;
            int midValue = ptr[mid / cols][mid % cols];

            if(midValue == target){
                return true;
            }
        }
    }
}
```

```

        if(midValue == target){
            return true;
        }
        else if(midValue < target){
            low = mid + 1;
        }
        else{
            high = mid - 1;
        }
    }
    return false;
}

~Array2D(){
    for(int i = 0; i < rows; i++){
        delete[] ptr[i];
    }
    delete[] ptr;
}

};

int main(){
    int m = 3, n = 4;
    Array2D matrix(m, n);

    int values[3][4] = {
        {1, 3, 5, 7},
        {10, 11, 16, 20},
        {23, 30, 34, 60}
    };

    for(int i = 0; i < m; i++){
        for(int j = 0; j < n; j++){
            matrix[i][j] = values[i][j];
        }
    }

    int target1 = 3;
    cout << "Target " << target1 << " found: " << (matrix.search(target1) ? "true" : "false") << endl;

    int target2 = 13;
    cout << "Target " << target2 << " found: " << (matrix.search(target2) ? "true" : "false") << endl;

    return 0;
}

```

**Output:**

```

Target 3 found: true
Target 13 found: false

-----
Process exited after 0.3358 seconds with return value 0
Press any key to continue . . .

```

**Question 03:****Source Code:**

```
#include<iostream>
#include<string>
using namespace std;

class Employee{
private:
    string name;
    int id;
    string designation;
    double salary;

public:
    Employee(string n, int i, string d, double s) : name(n), id(i), designation(d), salary(s) { }

    void display(){
        cout << "ID: " << id << " | Name: " << name << " | Designation: " << designation << " | Salary: " << salary << endl;
    }

    friend class Payroll;
};

class Payroll{
public:
    void updateSalary(Employee &emp, double percentage){
        cout << "Updating salary for " << emp.name << "..." << endl;

        double increase = emp.salary * (percentage / 100.0);
        emp.salary += increase;
    }

    void setRevisedSalary(Employee &emp, double newSalary){
        emp.salary = newSalary;
    }
};

int main(){
    Employee emp1("John Doe", 101, "Software Engineer", 50000);
    Employee emp2("Jane Smith", 102, "Project Manager", 75000);

    Payroll admin;

    cout << "--- Current Payroll Records ---" << endl;
    emp1.display();
    emp2.display();

    cout << "\n--- Applying Salary Revisions ---" << endl;
    admin.updateSalary(emp1, 10);

    admin.setRevisedSalary(emp2, 82000);

    cout << "\n--- Updated Payroll Records ---" << endl;
    emp1.display();
    emp2.display();

    return 0;
}
```

**Output:**

```
--- Current Payroll Records ---
ID: 101 | Name: John Doe | Designation: Software Engineer | Salary: 50000
ID: 102 | Name: Jane Smith | Designation: Project Manager | Salary: 75000

--- Applying Salary Revisions ---
Updating salary for John Doe...

--- Updated Payroll Records ---
ID: 101 | Name: John Doe | Designation: Software Engineer | Salary: 55000
ID: 102 | Name: Jane Smith | Designation: Project Manager | Salary: 82000

-----
Process exited after 0.3159 seconds with return value 0
Press any key to continue . . .
```

**Question 04:****Source Code:**

```
#include<iostream>
#include<string>
using namespace std;

class Employee;

class Payroll{
public:
    void updateByFriend(Employee &emp, double newAmount);
};

class Employee{
private:
    string name;
    int id;
    string designation;
    double salary;

public:
    Employee(string n, int i, string d, double s) : name(n), id(i), designation(d), salary(s) { }

    void display(){
        cout << "ID: " << id << " | Name: " << name << " | Salary: " << salary << endl;
    }

    friend void Payroll::updateByFriend(Employee &emp, double newAmount);
};

void Payroll::updateByFriend(Employee &emp, double newAmount){
    emp.salary = newAmount;
    cout << "Salary revised for " << emp.name << " via Friend Function." << endl;
}
```

```
int main(){
    Employee e2("Bob", 102, "Developer", 50000);
    Payroll p;

    e2.display();
    p.updateByFriend(e2, 55000);
    e2.display();

    return 0;
}
```

**Output:**

```
ID: 102 | Name: Bob | Salary: 50000
Salary revised for Bob via Friend Function.
ID: 102 | Name: Bob | Salary: 55000

-----
Process exited after 0.3178 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include<iostream>
#include <cstdlib>
using namespace std;

class Array{
    int size;
    int *ptr;

public:
    // Constructor
    Array(int s = 0) : size(s), ptr(NULL){
        if(size > 0){
            ptr = new int[size];
            if(ptr == NULL){
                exit(1);
            }
        }
    }

    // Copy Constructor
    Array(const Array& ob){
        if(ob.size > 0){
            size = ob.size;
            ptr = new int[size];
            if(ptr == NULL){
                exit(1);
            }
            for(int i = 0; i < size; i++){
                ptr[i] = ob.ptr[i];
            }
        }
        else{
            size = 0;
        }
    }
};
```

```
}
else{
    size = 0;
    ptr = NULL;
}
}

// Subscript operator
int& operator[](int in){
    if(in >= size || in < 0){
        cout << "index " << in << " is out of bound" << endl;
        exit(1);
    }
    return ptr[in];
}

// Addition operator (Array + Array)
Array operator+(Array& ob){
    if (size == ob.size){
        Array temp(size);
        for(int i = 0; i < size; i++){
            temp[i] = ptr[i] + ob[i];
        }
        return temp;
    }
    else{
        cout << "The size of array does not match!" << endl;
        exit(1);
    }
}
```

```

        // Multiplication operator (Dot Product)
        int operator*(Array& ob){
            if (size == ob.size){
                int temp = 0;
                for(int i = 0; i < size; i++){
                    temp += ptr[i] * ob[i];
                }
                return temp;
            }
            else{
                cout << "The size of array does not match!" << endl;
                exit(1);
            }
        }

        // Destructor
        ~Array(){
            delete[] ptr;
        }
};

int main(){
    Array a1(5), a2(5);

    for(int i = 0; i < 5; i++){
        a1[i] = i * 2;
        a2[i] = i + 3;
    }

    cout << "Array 1 elements:" << endl;
    for (int i = 0; i < 5; i++)
        cout << a1[i] << " ";
    cout << endl;

    cout << "Array 2 elements:" << endl;
    for (int i = 0; i < 5; i++)
        cout << a2[i] << " ";
    cout << endl;

    Array a3(a1);
    Array a4 = a1 + a2;

    cout << "Array 4 (Sum of a1 and a2):" << endl;
    for(int i = 0; i < 5; i++){
        cout << a4[i] << "\t";
    }
    cout << endl;

    cout << "Dot product of a1 and a2:" << endl;
    cout << a1 * a2 << endl;

    return 0;
}

```

**Output:**

```
Array 1 elements:
0 2 4 6 8
Array 2 elements:
3 4 5 6 7
Array 4 (Sum of a1 and a2):
3      6      9      12      15
Dot product of a1 and a2:
120

-----
Process exited after 0.3412 seconds with return value 0
Press any key to continue . . . |
```

### Example 02:

#### Source Code:

```
#include<iostream>
using namespace std;

class beta; // Forward declaration

class alpha{
private:
    int data;

public:
    alpha() : data(3) { }

    // Friend function declaration
    friend int frifunc(alpha, beta);
};

class beta{
private:
    int data;

public:
    beta() : data(7) { }

    // Friend function declaration
    friend int frifunc(alpha, beta);
};

// Function definition accessing private data from both classes
int frifunc(alpha a, beta b){
    return(a.data + b.data);
}
```

```
int main(){
    alpha aa;
    beta bb;

    cout << frifunc(aa, bb) << endl;

    return 0;
}
```

### Output:

```
10
-----
Process exited after 0.3492 seconds with return value 0
Press any key to continue . . . |
```

### Example 03:

#### Source Code:

```
#include<iostream>
#include<cstdlib>
using namespace std;

class Array{
    int size;
    int *ptr;

public:
    // Constructor
    Array(int s = 0) : size(s), ptr(NULL){
        if(size > 0){
            ptr = new int[size];
            if(ptr == NULL){
                exit(1);
            }
        }
    }

    // Copy Constructor (Required for passing by value)
    Array(const Array& ob){
        size = ob.size;
        ptr = new int[size];
        for(int i = 0; i < size; i++){
            ptr[i] = ob.ptr[i];
        }
    }
}
```

```

        // Subscript operator
        int& operator[](int in){
            if(in >= size || in < 0){
                cout << "index " << in << " is out of bound" << endl;
                exit(1);
            }
            return ptr[in];
        }

        // Destructor
        ~Array(){
            delete[] ptr;
        }

        friend int dotproduct(Array, Array);
};

// Friend function for dot product
int dotproduct(Array a1, Array a2){
    if(a1.size == a2.size){
        int temp = 0;
        for(int i = 0; i < a1.size; i++){
            temp += a1[i] * a2[i];
        }
        return temp;
    }
    else{
        cout << "Size mismatch error" << endl;
        exit(1);
    }
}

int main(){
    Array a1(5), a2(5);

    for(int i = 0; i < 5; i++){
        a1[i] = i * 2;
        a2[i] = i + 3;
    }

    cout << "Array 1 elements:" << endl;
    for (int i = 0; i < 5; i++)
        cout << a1[i] << endl;

    cout << "Array 2 elements:" << endl;
    for (int i = 0; i < 5; i++)
        cout << a2[i] << endl;

    cout << "Dot Product result:" << endl;
    cout << dotproduct(a1, a2) << endl;

    return 0;
}

```

Output:

```
Array 1 elements:
0
2
4
6
8
Array 2 elements:
3
4
5
6
7
Dot Product result:
120

-----
Process exited after 0.3332 seconds with return value 0
Press any key to continue . . .
```

Example 04:Source Code:

```
#include<iostream>
using namespace std;

class alpha{
private:
    int data1;

public:
    alpha() : data1(99) { }

    // beta is a friend class of alpha
    friend class beta;
};

class beta{
public:
    void func1(alpha a){
        cout << "\ndata1=" << a.data1;
    }

    void func2(alpha a){
        cout << "\ndata1=" << a.data1;
    }
};
```

```
int main(){  
    alpha a;  
    beta b;  
  
    b.func1(a);  
    b.func2(a);  
  
    cout << endl;  
    return 0;  
}
```

**Output:**

```
data1=99  
data1=99  
  
-----  
Process exited after 0.3363 seconds with return value 0  
Press any key to continue . . .
```

# **LAB 08**

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include <iostream>
#include <string>
using namespace std;
class Vehicle {
protected:
    string type;
    string make;
    string model;
    string color;
    int year;
    int milesDriven;

public:
    Vehicle(string t, string mk, string md, string c, int y, int mdriven): type(t), make(mk), model(md), color(c), year(y), milesDriven(mdriven) {}

    string getType(){
        return type;
    }

    string getMake(){
        return make;
    }
    string getModel(){
        return model;
    }

    string getColor(){
        return color;
    }

    int getYear(){
        return year;
    }

    int getMilesDriven(){
        return milesDriven;
    }
};

class GasVehicle : virtual public Vehicle {
protected:
    int fuelTankSize;

public:
    GasVehicle(string t, string mk, string md, string c, int y, int mdriven, int fTank)
        : Vehicle(t, mk, md, c, y, mdriven), fuelTankSize(fTank) {}

    int getFuelTankSize(){
        return fuelTankSize;
    }
};

class ElectricVehicle : virtual public Vehicle {
protected:
    int energyStorage;

public:
    ElectricVehicle(string t, string mk, string md, string c, int y, int mdriven, int estorage)
        : Vehicle(t, mk, md, c, y, mdriven), energyStorage(estorage) {}

    int getEnergyStorage(){
        return energyStorage;
    }
};
```

```

class HighPerformance : public GasVehicle {
protected:
    int horsepower;
    int topSpeed;

public:
    HighPerformance(string t, string mk, string md, string c, int y, int mdriven, int fTank, int hp, int ts)
        : Vehicle(t, mk, md, c, y, mdriven), GasVehicle(t, mk, md, c, y, mdriven, fTank), horsepower(hp), topSpeed(ts) {}

    int getHorsePower(){
        return horsepower;
    }
    int getTopSpeed(){
        return topSpeed;
    }
};

class SportsCar : public HighPerformance {
protected:
    string gearbox;
    string driveSystem;

public:
    SportsCar(string t, string mk, string md, string c, int y, int mdriven, int fTank, int hp, int ts, string gb, string ds)
        : Vehicle(t, mk, md, c, y, mdriven), HighPerformance(t, mk, md, c, y, mdriven, fTank, hp, ts), gearbox(gb), driveSystem(ds) {}

    string getGearbox(){
        return gearbox;
    }
    string getDriveSystem(){
        return driveSystem;
    }
};

class HeavyVehicle : public GasVehicle, public ElectricVehicle {
protected:
    int maxWeight;
    int numWheels;
    int length;

public:
    HeavyVehicle(string t, string mk, string md, string c, int y, int mdriven, int fTank, int eStorage, int weight, int wheels, int len)
        : Vehicle(t, mk, md, c, y, mdriven),
          GasVehicle(t, mk, md, c, y, mdriven, fTank),
          ElectricVehicle(t, mk, md, c, y, mdriven, eStorage),
          maxWeight(weight), numWheels(wheels), length(len) {}

    int getMaxWeight(){
        return maxWeight;
    }
    int getNumWheels(){
        return numWheels;
    }
    int getLength(){
        return length;
    }
};

class ConstructionTruck : public HeavyVehicle {
protected:
    string cargo;

public:
    ConstructionTruck(string t, string mk, string md, string c, int y, int mdriven, int fTank, int eStorage, int weight, int wheels, int len, string cg)
        : Vehicle(t, mk, md, c, y, mdriven), HeavyVehicle(t, mk, md, c, y, mdriven, fTank, eStorage, weight, wheels, len), cargo(cg) {}

    string getCargo(){
        return cargo;
    }
};

```

```

class Bus : public HeavyVehicle {
protected:
    int numSeats;

public:
    Bus(string t, string mk, string md, string c, int y, int mdriven, int fTank, int eStorage, int weight, int wheels, int len, int seats)
        : Vehicle(t, mk, md, c, y, mdriven), HeavyVehicle(t, mk, md, c, y, mdriven, fTank, eStorage, weight, wheels, len), numSeats(seats) {}

    int getNumSeats(){
        return numSeats;
    }

    void display() {
        cout << "==== Bus Details ====" << endl;
        cout << "Type: " << type << endl;
        cout << "Make: " << make << endl;
        cout << "Model: " << model << endl;
        cout << "Color: " << color << endl;
        cout << "Year: " << year << endl;
        cout << "Miles Driven: " << milesDriven << endl;
        cout << "Fuel Tank Size: " << fuelTankSize << endl;
        cout << "Energy Storage: " << energyStorage << endl;
        cout << "Maximum Weight: " << maxWeight << endl;
        cout << "Number of Wheels: " << numWheels << endl;
        cout << "Length: " << length << endl;
        cout << "Number of Seats: " << numSeats << endl;
        cout << "=====" << endl;
    }
};

int main(void) {
    Bus myBus("Passenger Bus", "Higer", "9700", "White", 2022, 15000, 110, 50, 40000, 6, 45, 55);
    myBus.display();

    return 0;
}

```

**Output:**

```

==== Bus Details ====
Type: Passenger Bus
Make: Higer
Model: 9700
Color: White
Year: 2022
Miles Driven: 15000
Fuel Tank Size: 110
Energy Storage: 50
Maximum Weight: 40000
Number of Wheels: 6
Length: 45
Number of Seats: 55
=====

-----
Process exited after 0.2948 seconds with return value 0
Press any key to continue . . . |

```

**Question 02:****Source Code:**

```
#include <iostream>
#include <string>
using namespace std;

class Character {
protected:
    string name;
    int level;
    int health;

public:
    Character(string n, int l, int h): name(n), level(l), health(h) {}

    string getName(){
        return name;
    }

    int getLevel(){
        return level;
    }

    int getHealth(){
        return health;
    }
};

class Warrior : virtual public Character {
protected:
    int strength;
    string meleeWeaponsProficiency;

public:
    Warrior(string n, int l, int h, int str, string meleeProf)
        : Character(n, l, h), strength(str), meleeWeaponsProficiency(meleeProf) {}

    int getStrength(){
        return strength;
    }

    string getMeleeWeaponsProficiency(){
        return meleeWeaponsProficiency;
    }

    void slash(){
        cout << name << " performs a powerful slash attack!" << endl;
    }
};
```

```
class Mage : virtual public Character {
protected:
    int intelligence;
    string spellCastingProficiency;

public:
    Mage(string n, int l, int h, int intel, string spellProf)
        : Character(n, l, h), intelligence(intel), spellCastingProficiency(spellProf) {}

    int getIntelligence(){
        return intelligence;
    }

    string getSpellCastingProficiency(){
        return spellCastingProficiency;
    }

    void fireball(){
        cout << name << " casts a blazing fireball!" << endl;
    }
};

class Archer : virtual public Character {
protected:
    int dexterity;
    string rangedWeaponsProficiency;

public:
    Archer(string n, int l, int h, int dex, string rangedProf)
        : Character(n, l, h), dexterity(dex), rangedWeaponsProficiency(rangedProf) {}

    int getDexterity(){
        return dexterity;
    }

    string getRangedWeaponsProficiency(){
        return rangedWeaponsProficiency;
    }

    void rapidShot(){
        cout << name << " fires a rapid shot!" << endl;
    }
};
```

```

class NPC : virtual public Character {
protected:
    string movementPattern;
    string scriptedDialogue;

public:
    NPC(string n, int l, int h, string movement, string dialogue)
        : Character(n, l, h), movementPattern(movement), scriptedDialogue(dialogue) {}

    string getMovementPattern(){
        return movementPattern;
    }

    string getScriptedDialogue(){
        return scriptedDialogue;
    }

    void speak(){
        cout << name << " says: \"" << scriptedDialogue << "\"" << endl;
    }
};

class Mighty : public Warrior, public Mage {
public:
    Mighty(string n, int l, int h, int str, string meleeProf, int intel, string spellProf)
        : Character(n, l, h),
        Warrior(n, l, h, str, meleeProf),
        Mage(n, l, h, intel, spellProf) {}

    void displayMightyStats(){
        cout << "==== Mighty Character Stats =====" << endl;
        cout << "Name: " << name << endl;
        cout << "Level: " << level << endl;
        cout << "Health: " << health << endl;
        cout << "Strength: " << strength << endl;
        cout << "Melee Proficiency: " << meleeWeaponsProficiency << endl;
        cout << "Intelligence: " << intelligence << endl;
        cout << "Spell Proficiency: " << spellCastingProficiency << endl;
        cout << "===== " << endl;
    }
};

int main(void) {
    Mighty myHero("Arthur The Mighty", 50, 1000, 85, "Master Swordsman", 90, "Grand Sorcerer");

    myHero.displayMightyStats();

    cout << "\n--- Action Demonstration ---" << endl;
    myHero.slash();
    myHero.fireball();

    cout << "\n--- NPC Demonstration ---" << endl;
    NPC villager("Bob", 1, 100, "Wander around town", "Hello there, traveler!");
    villager.speak();

    cout << "\n--- Archer Demonstration ---" << endl;
    Archer ranger("Legolas", 30, 800, 95, "Master Archer");
    ranger.rapidShot();

    return 0;
}

```

Output:

```
==== Mighty Character Stats ====
Name: Arthur The Mighty
Level: 50
Health: 1000
Strength: 85
Melee Proficiency: Master Swordsman
Intelligence: 90
Spell Proficiency: Grand Sorcerer
=====

--- Action Demonstration ---
Arthur The Mighty performs a powerful slash attack!
Arthur The Mighty casts a blazing fireball!

--- NPC Demonstration ---
Bob says: "Hello there, traveler!"

--- Archer Demonstration ---
Legolas fires a rapid shot!

-----
Process exited after 0.5228 seconds with return value 0
Press any key to continue . . . |
```

**Question 03:****Source Code:**

```
#include <iostream>
using namespace std;

class Account {
protected:
    double Balance;

public:
    Account() {
        cout << "Enter initial balance: ";
        cin >> Balance;
    }

    Account(double b) : Balance(b) {}

    virtual void deposit(double amount) {
        Balance += amount;
        cout << "Deposited Rs. " << amount << ". New Balance: Rs. " << Balance << endl;
    }

    virtual void withdraw(double amount) {
        if (Balance >= amount) {
            Balance -= amount;
            cout << "Withdrew Rs. " << amount << ". New Balance: Rs. " << Balance << endl;
        } else {
            cout << "Insufficient Balance!" << endl;
        }
    }

    double checkBalance() {
        return Balance;
    }
};

class InterestAccount : virtual public Account {
protected:
    double Interest;

public:
    InterestAccount() : Account() {
        cout << "Enter interest rate (e.g., 0.30 for 30%): ";
        cin >> Interest;
    }

    InterestAccount(double b, double i = 0.30) : Account(b), Interest(i) {}

    void deposit(double amount) {
        double bonus = amount * Interest;
        Balance += (amount + bonus);
        cout << "Deposited Rs. " << amount << " with Rs. " << bonus << " interest. New Balance: Rs. " << Balance << endl;
    }
};
```

```

class ChargingAccount : virtual public Account {
protected:
    double fee;

    void withdraw(double amount) {
        if (Balance >= (amount + fee)) {
            Balance -= (amount + fee);
            cout << "Withdrew Rs. " << amount << " with Rs. " << fee << " fee. New Balance: Rs. " << Balance << endl;
        } else {
            cout << "Insufficient Balance for withdrawal and fee!" << endl;
        }
    }

public:
    ChargingAccount() : Account() {
        cout << "Enter withdrawal fee (e.g., 25): ";
        cin >> fee;
    }

    ChargingAccount(double b, double f = 25.0) : Account(b), fee(f) {}
};

class ACI : public InterestAccount, public ChargingAccount {
public:
    ACI() : Account(), InterestAccount(), ChargingAccount() {}

    ACI(double b, double i = 0.30, double f = 25.0) : Account(b), InterestAccount(b, i), ChargingAccount(b, f) {}

    void deposit(double amount) {
        InterestAccount::deposit(amount);
    }

    void withdraw(double amount) {
        ChargingAccount::withdraw(amount);
    }

    void transfer(double amount, Account& acc) {
        cout << "Transferring Rs. " << amount << " to standard Account..." << endl;
        withdraw(amount);
        acc.deposit(amount);
    }

    void transfer(double amount, InterestAccount& acc) {
        cout << "Transferring Rs. " << amount << " to InterestAccount..." << endl;
        withdraw(amount);
        acc.deposit(amount);
    }

    void transfer(double amount, ChargingAccount& acc) {
        cout << "Transferring Rs. " << amount << " to ChargingAccount..." << endl;
        withdraw(amount);
        acc.deposit(amount);
    }
};

```

```
int main(void) {
    cout << "--- Initializing ACI Account ---" << endl;
    ACI myACI(1000.0, 0.30, 25.0);

    cout << "\n--- Initializing Standard Account ---" << endl;
    Account targetAcc(500.0);

    cout << "\n--- Initializing Interest Account ---" << endl;
    InterestAccount targetIntAcc(500.0, 0.30);

    cout << "\n--- Testing ACI Deposit (Should add 30% interest) ---" << endl;
    myACI.deposit(100.0);

    cout << "\n--- Testing ACI Withdraw (Should charge Rs 25 fee) ---" << endl;
    myACI.withdraw(200.0);

    cout << "\n--- Testing Transfer to Standard Account ---" << endl;
    myACI.transfer(100.0, targetAcc);

    cout << "\n--- Testing Transfer to Interest Account ---" << endl;
    myACI.transfer(100.0, targetIntAcc);

    cout << "\n--- Final Balances ---" << endl;
    cout << "ACI Account Balance: Rs. " << myACI.checkBalance() << endl;
    cout << "Standard Account Balance: Rs. " << targetAcc.checkBalance() << endl;
    cout << "Interest Account Balance: Rs. " << targetIntAcc.checkBalance() << endl;

    return 0;
}
```

Output:

```
--- Initializing ACI Account ---  
  
--- Initializing Standard Account ---  
  
--- Initializing Interest Account ---  
  
--- Testing ACI Deposit (Should add 30% interest) ---  
Deposited Rs. 100 with Rs. 30 interest. New Balance: Rs. 1130  
  
--- Testing ACI Withdraw (Should charge Rs 25 fee) ---  
Withdrew Rs. 200 with Rs. 25 fee. New Balance: Rs. 905  
  
--- Testing Transfer to Standard Account ---  
Transferring Rs. 100 to standard Account...  
Withdrew Rs. 100 with Rs. 25 fee. New Balance: Rs. 780  
Deposited Rs. 100. New Balance: Rs. 600  
  
--- Testing Transfer to Interest Account ---  
Transferring Rs. 100 to InterestAccount...  
Withdrew Rs. 100 with Rs. 25 fee. New Balance: Rs. 655  
Deposited Rs. 100 with Rs. 30 interest. New Balance: Rs. 630  
  
--- Final Balances ---  
ACI Account Balance: Rs. 655  
Standard Account Balance: Rs. 600  
Interest Account Balance: Rs. 630  
  
-----  
Process exited after 0.2859 seconds with return value 0  
Press any key to continue . . .
```

**Question 04:****Source Code:**

```
#include <iostream>
#include <string>
using namespace std;

class Media {
protected:
    string title;
    bool isBorrowed;

public:
    Media(string t) : title(t), isBorrowed(false) {}

    void borrowMedia() {
        if (!isBorrowed) {
            isBorrowed = true;
            cout << title << " has been successfully borrowed." << endl;
        } else {
            cout << "Sorry, " << title << " is currently out of stock/borrowed." << endl;
        }
    }

    void returnMedia() {
        if (isBorrowed) {
            isBorrowed = false;
            cout << title << " has been returned successfully." << endl;
        } else {
            cout << title << " was not borrowed." << endl;
        }
    }

    virtual void display() {
        cout << "Title: " << title << endl;
        cout << "Status: " << (isBorrowed ? "Borrowed" : "Available") << endl;
    }
};

class Author {
protected:
    string authorName;

public:
    Author(string a) : authorName(a) {}

    string getAuthorName(){
        return authorName;
    }
};

class IssueDetails {
protected:
    int issueNumber;

public:
    IssueDetails(int issue) : issueNumber(issue) {}

    int getIssueNumber(){
        return issueNumber;
    }
};
```

```

class Director {
protected:
    string directorName;

public:
    Director(string d) : directorName(d) {}

    string getDirectorName(){
        return directorName;
    }
};

class Book : public Media, public Author {
public:
    Book(string t, string a) : Media(t), Author(a) {}

    void display() {
        cout << "==== Book Info ====" << endl;
        Media::display();
        cout << "Author: " << authorName << endl;
        cout << "===== " << endl;
    }
};

class Magazine : public Media, public IssueDetails {
public:
    Magazine(string t, int issue) : Media(t), IssueDetails(issue) {}

    void display() {
        cout << "==== Magazine Info ====" << endl;
        Media::display();
        cout << "Issue Number: " << issueNumber << endl;
        cout << "===== " << endl;
    }
};

class DVD : public Media, public Director {
public:
    DVD(string t, string d) : Media(t), Director(d) {}

    void display() {
        cout << "==== DVD Info ====" << endl;
        Media::display();
        cout << "Director: " << directorName << endl;
        cout << "===== " << endl;
    }
};

```

```
int main(void) {
    Book myBook("The Great Gatsby", "F. Scott Fitzgerald");
    Magazine myMagazine("National Geographic", 205);
    DVD myDVD("Inception", "Christopher Nolan");

    cout << "\n--- Displaying Initial States ---" << endl;
    myBook.display();
    myMagazine.display();
    myDVD.display();

    cout << "\n--- Testing Borrow Operations ---" << endl;
    myBook.borrowMedia();
    myDVD.borrowMedia();

    cout << "\n--- Testing Double Borrow (Should Fail) ---" << endl;
    myBook.borrowMedia();

    cout << "\n--- Displaying States After Borrowing ---" << endl;
    myBook.display();
    myDVD.display();

    cout << "\n--- Testing Return Operations ---" << endl;
    myBook.returnMedia();

    cout << "\n--- Testing DVD Return ---" << endl;
    myDVD.returnMedia();

    cout << "\n--- Displaying Final State ---" << endl;
    myBook.display();

    return 0;
}
```

Output:

```
Sorry, The Great Gatsby is currently out of stock/borrowed.
```

```
--- Displaying States After Borrowing ---
```

```
==== Book Info ====
```

```
Title: The Great Gatsby
```

```
Status: Borrowed
```

```
Author: F. Scott Fitzgerald
```

```
=====
```

```
==== DVD Info ====
```

```
Title: Inception
```

```
Status: Borrowed
```

```
Director: Christopher Nolan
```

```
=====
```

```
--- Testing Return Operations ---
```

```
The Great Gatsby has been returned successfully.
```

```
--- Testing DVD Return ---
```

```
Inception has been returned successfully.
```

```
--- Displaying Final State ---
```

```
==== Book Info ====
```

```
Title: The Great Gatsby
```

```
Status: Available
```

```
Author: F. Scott Fitzgerald
```

```
=====
```

```
-----
```

```
Process exited after 0.5022 seconds with return value 0
```

```
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include <iostream>
using namespace std;

class A{
public:
    int a;
};

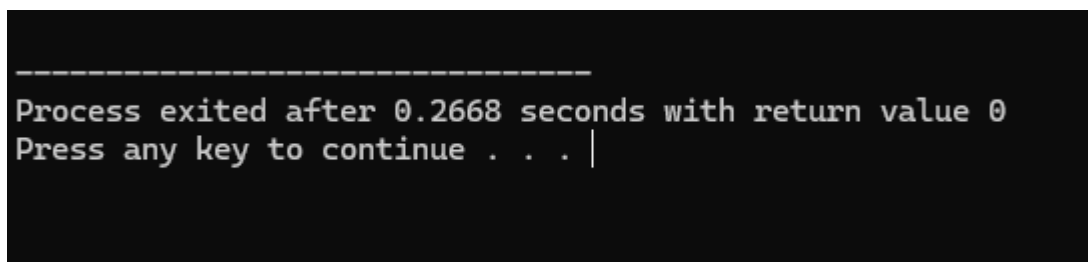
class B : virtual public A{
public:
    int b;
};

class C : virtual public A{
public:
    int c;
};

class D : public B, public C{
public:
    int d;
};

int main(){
    D obj;
    obj.a = 200;

    return 0;
}
```

**Output:**

```
-----
Process exited after 0.2668 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:**

**Source Code:**

```
#include <iostream>
using namespace std;

class PoweredDevice{
public:
    PoweredDevice(int power){
        cout << "PoweredDevice: " << power << '\n';
    }
};

class Scanner : virtual public PoweredDevice{
public:
    Scanner(int scanner, int power) : PoweredDevice(power){
        cout << "Scanner: " << scanner << '\n';
    }
};

class Printer : virtual public PoweredDevice{
public:
    Printer(int printer, int power) : PoweredDevice(power){
        cout << "Printer: " << printer << '\n';
    }
};

class Copier : public Scanner, public Printer{
public:
    Copier(int scanner, int printer, int power) : PoweredDevice(power), Scanner(scanner, power), Printer(printer, power) { }
};

int main(){
    Copier copier(1, 2, 3);

    return 0;
}
```

**Output:**

```
PoweredDevice: 3
Scanner: 1
Printer: 2

-----
Process exited after 0.283 seconds with return value 0
Press any key to continue . . .
```

# LAB 09

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include <iostream>
using namespace std;

class ArrayMultiplier {
public:
    virtual void calculate() = 0;
    virtual ~ArrayMultiplier(){}
};

class ArrayMultiplier1D : public ArrayMultiplier {
protected:
    int* arr;
    int size;

public:
    ArrayMultiplier1D(int* a, int s) : arr(a), size(s) {}

    void calculate() {
        long long product = 1;
        for (int i = 0; i < size; i++) {
            product *= arr[i];
        }
        cout << "Product of 1D Array elements: " << product << endl;
    }
};

class ArrayMultiplier2D : public ArrayMultiplier {
protected:
    int** arr;
    int rows;
    int cols;

public:
    ArrayMultiplier2D(int** a, int r, int c) : arr(a), rows(r), cols(c) {}

    void calculate() {
        long long product = 1;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                product *= arr[i][j];
            }
        }
        cout << "Product of 2D Array elements: " << product << endl;
    }
};
```

```
int main(void) {
    int array1[] = {2, 3, 4, 5};
    ArrayMultiplier1D mult1D(array1, 4);

    mult1D.calculate();

    int r = 2;
    int c = 3;

    int** array2 = new int*[r];
    for (int i = 0; i < r; i++) {
        array2[i] = new int[c];

    }

    int value = 1;
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            array2[i][j] = value++;
        }
    }

    ArrayMultiplier2D mult2D(array2, r, c);

    mult2D.calculate();

    for (int i = 0; i < r; i++) {
        delete[] array2[i];
    }
    delete[] array2;

    return 0;
}
```

**Output:**

```
Product of 1D Array elements: 120
Product of 2D Array elements: 720

-----
Process exited after 0.3438 seconds with return value 0
Press any key to continue . . . |
```

**Question 02:****Source Code:**

```
#include <iostream>
using namespace std;

class Store {
protected:
    double total_bill;
public:
    Store(double bill) : total_bill(bill) {}
    virtual void calculateFinalBill() = 0;
    virtual ~Store() {}
};

class ImtiazStore : public Store {
public:
    ImtiazStore(double bill) : Store(bill) {}

    void calculateFinalBill() {
        double discount = total_bill * 0.07;
        double final_bill = total_bill - discount;
        cout << "--- Imtiaz Store ---" << endl;
        cout << "Original Bill: Rs. " << total_bill << endl;
        cout << "Discount (7%): Rs. " << discount << endl;
        cout << "Final Bill: Rs. " << final_bill << endl << endl;
    }
};

class BinHashimStore : public Store {
public:
    BinHashimStore(double bill) : Store(bill) {}

    void calculateFinalBill() {
        double discount = total_bill * 0.05;
        double final_bill = total_bill - discount;
        cout << "--- Bin Hashim Store ---" << endl;
        cout << "Original Bill: Rs. " << total_bill << endl;
        cout << "Discount (5%): Rs. " << discount << endl;
        cout << "Final Bill: Rs. " << final_bill << endl << endl;
    }
};

int main(void) {
    double billAmount = 5000.0;

    ImtiazStore imtiaz(billAmount);
    BinHashimStore binHashim(billAmount);

    imtiaz.calculateFinalBill();
    binHashim.calculateFinalBill();

    return 0;
}
```

Output:

```
--- Imtiaz Store ---  
Original Bill: Rs. 5000  
Discount (7%): Rs. 350  
Final Bill: Rs. 4650  
  
--- Bin Hashim Store ---  
Original Bill: Rs. 5000  
Discount (5%): Rs. 250  
Final Bill: Rs. 4750  
  
-----  
Process exited after 0.3466 seconds with return value 0  
Press any key to continue . . .
```

**Question 03:****Source Code:**

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    int carId;
    string brand;
    string model;

public:
    Vehicle(int id, string b, string m) : carId(id), brand(b), model(m) {}
    virtual bool isAvailable() = 0;
    virtual void rent() = 0;
    virtual ~Vehicle() {}

    void display() {
        cout << brand << " " << model << " (ID: " << carId << ")";
    }
};

class Car : public Vehicle {
protected:
    bool available;

public:
    Car(int id, string b, string m) : Vehicle(id, b, m), available(true) {}

    ~Car() {}

    bool isAvailable() {
        return available;
    }

    void rent() {
        available = false;
    }

    void returnVehicle() {
        available = true;
    }
};
```

```
class RentalSystem {
public:
    void rentVehicle(Vehicle* v) {
        if (v->isAvailable()) {
            v->rent();
            cout << "RentalSystem: Vehicle successfully rented." << endl;
        }
        else {
            cout << "RentalSystem: Sorry, vehicle is currently not available." << endl;
        }
    }

    void returnVehicle(Vehicle* v) {
        Car* c = dynamic_cast<Car*>(v);
        if (c != NULL) {
            c->returnVehicle();
            cout << "RentalSystem: Vehicle successfully returned." << endl;
        }
        else {
            cout << "RentalSystem: Error returning vehicle." << endl;
        }
    }
};

class Customer {
protected:
    string name;

public:
    Customer(string n) : name(n) {}

    void rentVehicle(Vehicle* v, RentalSystem* rs) {
        cout << name << " is attempting to rent: ";
        v->display();
        cout << endl;
        rs->rentVehicle(v);
    }

    void returnVehicle(Vehicle* v, RentalSystem* rs) {
        cout << name << " is returning: ";
        v->display();
        cout << endl;
        rs->returnVehicle(v);
    }
};
```

```
int main(void) {
    RentalSystem sys;
    Customer cust1("Alice");
    Customer cust2("Bob");

    Vehicle* fleet[3];

    fleet[0] = new Car(101, "Toyota", "Corolla");
    fleet[1] = new Car(102, "Honda", "Civic");
    fleet[2] = new Car(103, "Ford", "Mustang");

    cout << "--- Initial Rental Process ---" << endl;
    cust1.rentVehicle(fleet[0], &sys);

    cout << "\n--- Trying to rent an already rented car ---" << endl;
    cust2.rentVehicle(fleet[0], &sys);

    cout << "\n--- Renting a different available car ---" << endl;
    cust2.rentVehicle(fleet[2], &sys);

    cout << "\n--- Returning the first car ---" << endl;
    cust1.returnVehicle(fleet[0], &sys);

    cout << "\n--- Renting the newly returned car ---" << endl;
    cust2.rentVehicle(fleet[0], &sys);

    for (int i = 0; i < 3; i++) {
        delete fleet[i];
    }

    return 0;
}
```

Output:

```
--- Initial Rental Process ---  
Alice is attempting to rent: Toyota Corolla (ID: 101)  
RentalSystem: Vehicle successfully rented.  
  
--- Trying to rent an already rented car ---  
Bob is attempting to rent: Toyota Corolla (ID: 101)  
RentalSystem: Sorry, vehicle is currently not available.  
  
--- Renting a different available car ---  
Bob is attempting to rent: Ford Mustang (ID: 103)  
RentalSystem: Vehicle successfully rented.  
  
--- Returning the first car ---  
Alice is returning: Toyota Corolla (ID: 101)  
RentalSystem: Vehicle successfully returned.  
  
--- Renting the newly returned car ---  
Bob is attempting to rent: Toyota Corolla (ID: 101)  
RentalSystem: Vehicle successfully rented.  
  
-----  
Process exited after 0.3477 seconds with return value 0  
Press any key to continue . . . |
```

**Question 04:****Source Code:**

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

class EncryptionTechnique {
public:
    virtual ~EncryptionTechnique() {}

    virtual void encrypt(string msg) = 0;
};

class EncryptionTechnique1 : public EncryptionTechnique {
public:
    void encrypt(string msg) {
        cout << "Encrypted using Technique 1: ";
        for (int i = 0; i < msg.length(); i++) {
            char c = toupper(msg[i]);
            cout << (int)c;
        }
        cout << endl;
    }
};

class EncryptionTechnique2 : public EncryptionTechnique {
public:
    void encrypt(string msg) {
        cout << "Encrypted using Technique 2: ";
        for (int i = 0; i < msg.length(); i++) {
            char c = toupper(msg[i]);
            cout << (int)c + 2;
        }
        cout << endl;
    }
};

int main(void) {
    string message;

    cout << "Enter a message to encrypt: ";
    cin >> message;

    cout << "\nOriginal Message: " << message << endl;

    EncryptionTechnique* tech1 = new EncryptionTechnique1();
    EncryptionTechnique* tech2 = new EncryptionTechnique2();

    tech1->encrypt(message);
    tech2->encrypt(message);

    delete tech1;
    delete tech2;

    return 0;
}
```

Output:

```
Enter a message to encrypt: HELLO

Original Message: HELLO
Encrypted using Technique 1: 7269767679
Encrypted using Technique 2: 7471787881

-----
Process exited after 3.789 seconds with return value 0
Press any key to continue . . . |
```

**Question 05:****Source Code:**

```

#include <iostream>
#include <string>
#include <cctype>
using namespace std;

class EncryptionTechnique {
public:
    virtual ~EncryptionTechnique() {}
    virtual string encrypt(string msg) = 0;
};

class EncryptionTechnique1 : public EncryptionTechnique {
public:
    string encrypt(string msg) {
        string result = "";
        for (int i = 0; i < msg.length(); i++) {
            char c = toupper(msg[i]);
            result += to_string((int)c);
        }
        return result;
    }
};

class EncryptionTechnique2 : public EncryptionTechnique {
public:
    string encrypt(string msg) {
        string result = "";
        for (int i = 0; i < msg.length(); i++) {
            char c = toupper(msg[i]);
            result += to_string((int)c + 2);
        }
        return result;
    }
};

class DecryptionTechnique1 : public DecryptionTechnique {
public:
    string decrypt(string encryptedMsg) {
        string result = "";
        for (int i = 0; i < encryptedMsg.length(); i += 2) {
            string code = encryptedMsg.substr(i, 2);
            int asciiVal = stoi(code);
            result += (char)asciiVal;
        }
        return result;
    }
};

class DecryptionTechnique2 : public DecryptionTechnique {
public:
    string decrypt(string encryptedMsg) {
        string result = "";
        for (int i = 0; i < encryptedMsg.length(); i += 2) {
            string code = encryptedMsg.substr(i, 2);
            int asciiVal = stoi(code) - 2;
            result += (char)asciiVal;
        }
        return result;
    }
};

```

```
int main(void) {
    string message;

    cout << "Enter a message to encrypt and decrypt: ";
    cin >> message;
    cout << "\nOriginal Message: " << message << endl;

    EncryptionTechnique* enc1 = new EncryptionTechnique1();
    EncryptionTechnique* enc2 = new EncryptionTechnique2();

    string encrypted1 = enc1->encrypt(message);
    string encrypted2 = enc2->encrypt(message);

    cout << "\nEncrypted using Technique 1: " << encrypted1 << endl;
    cout << "Encrypted using Technique 2: " << encrypted2 << endl;

    DecryptionTechnique* dec1 = new DecryptionTechnique1();
    DecryptionTechnique* dec2 = new DecryptionTechnique2();

    string decrypted1 = dec1->decrypt(encrypted1);
    string decrypted2 = dec2->decrypt(encrypted2);

    cout << "\nDecrypted using Technique 1: " << decrypted1 << endl;
    cout << "Decrypted using Technique 2: " << decrypted2 << endl;

    delete enc1;
    delete enc2;
    delete dec1;
    delete dec2;

    return 0;
}
```

**Output:**

```
Enter a message to encrypt and decrypt: Hello

Original Message: Hello

Encrypted using Technique 1: 7269767679
Encrypted using Technique 2: 7471787881

Decrypted using Technique 1: HELLO
Decrypted using Technique 2: HELLO

-----
Process exited after 5.985 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include <iostream>
using namespace std;

class Base {
    int x;

    public:
        virtual void fun() = 0;
        int getX() { return x; }
};

class Derived : public Base {
    int y;

    public:
        void fun() { cout << "fun() called"; }
};

int main(void) {
    Derived d;
    d.fun();

    return 0;
}
```

**Output:**

```
fun() called
-----
Process exited after 0.3687 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:****Source Code:**

```
#include <iostream>
using namespace std;
class Base {
protected:
    int x;

public:
    Base(int i) { x = i; }
    virtual void fun() = 0;
};

class Derived : public Base {
protected:
    int y;

public:
    Derived(int i, int j) : Base(i) { y = j; }
    void fun() { cout << "x = " << x << ", y = " << y; }
};

int main(void) {
    Derived d(4, 5);
    d.fun();

    return 0;
}
```

**Output:**

```
x = 4, y = 5
-----
Process exited after 0.3344 seconds with return value 0
Press any key to continue . . .
```

**Example 03:****Source Code:**

```
#include <iostream>
using namespace std;

class Shape {
protected:
    float dimension;

public:
    void getDimension() {
        cin >> dimension;
    }

private:
    virtual float calculateArea() = 0;
};

class Square : public Shape {
public:
    float calculateArea() {
        return dimension * dimension;
    }
};

class Circle : public Shape {
public:
    float calculateArea() {
        return 3.14 * dimension * dimension;
    }
};

int main() {
    Square square;
    Circle circle;

    cout << "Enter the length of the square: ";
    square.getDimension();
    cout << "Area of square: " << square.calculateArea() << endl;

    cout << "\nEnter radius of the circle: ";
    circle.getDimension();
    cout << "Area of circle: " << circle.calculateArea() << endl;

    return 0;
}
```

Output:

```
Enter the length of the square: 4
Area of square: 16

Enter radius of the circle: 5
Area of circle: 78.5

-----
Process exited after 13.98 seconds with return value 0
Press any key to continue . . .
```

# **LAB 10**

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include <iostream>
using namespace std;

template <class T1, class T2>
class Calculator {
    T1 num1;
    T2 num2;

public:
    Calculator(T1 n1, T2 n2) {
        num1 = n1;
        num2 = n2;
    }

    auto add() {
        return num1 + num2;
    }

    auto subtract() {
        return num1 - num2;
    }

    auto multiply() {
        return num1 * num2;
    }

    auto divide() {
        if (num2 == 0) {
            return (double)0;
        }
        return (double)num1 / num2;
    }

    void display() {
        cout << "Addition: " << add() << endl;
        cout << "Subtraction: " << subtract() << endl;
        cout << "Multiplication: " << multiply() << endl;
        cout << "Division: " << divide() << endl;
    }
};
```

```
int main() {
    cout << "--- Same Types (int, int) ---" << endl;
    Calculator<int, int> calc1(20, 10);
    calc1.display();

    cout << "\n--- Different Types (int, double) ---" << endl;
    Calculator<int, double> calc2(10, 2.5);
    calc2.display();

    cout << "\n--- Different Types (float, int) ---" << endl;
    Calculator<float, int> calc3(5.5f, 2);
    calc3.display();

    return 0;
}
```

**Output:**

```
--- Same Types (int, int) ---
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2

--- Different Types (int, double) ---
Addition: 12.5
Subtraction: 7.5
Multiplication: 25
Division: 4

--- Different Types (float, int) ---
Addition: 7.5
Subtraction: 3.5
Multiplication: 11
Division: 2.75

-----
Process exited after 0.3755 seconds with return value 0
Press any key to continue . . .
```

**Question 02:**

Source Code:

```
#include <iostream>
using namespace std;

template <typename T1, typename T2>
void swapData(T1 &a, T2 &b) {
    T1 temp = a;
    a = (T1)b;
    b = (T2)temp;
}

int main() {
    int x = 10;
    int y = 20;
    cout << "Before swap (int, int): x = " << x << ", y = " << y << endl;
    swapData(x, y);
    cout << "After swap (int, int): x = " << x << ", y = " << y << endl;

    int m = 5;
    double n = 9.5;
    cout << "\nBefore swap (int, double): m = " << m << ", n = " << n << endl;
    swapData(m, n);
    cout << "After swap (int, double): m = " << m << ", n = " << n << endl;

    char c = 'A';
    int d = 70;
    cout << "\nBefore swap (char, int): c = " << c << ", d = " << d << endl;
    swapData(c, d);
    cout << "After swap (char, int): c = " << c << ", d = " << d << endl;

    return 0;
}
```

Output:

```
Before swap (int, int): x = 10, y = 20
After swap (int, int): x = 20, y = 10

Before swap (int, double): m = 5, n = 9.5
After swap (int, double): m = 9, n = 5

Before swap (char, int): c = A, d = 70
After swap (char, int): c = F, d = 65

-----
Process exited after 0.3472 seconds with return value 0
Press any key to continue . . .
```

Question 03:

Source Code:

```
#include <iostream>
using namespace std;

template <class T>
class mycontainer {
    T element;
public:
    mycontainer(T arg) {
        element = arg;
    }
    T increase() {
        return ++element;
    }
};

template <>
class mycontainer<char> {
    char element;
public:
    mycontainer(char arg) {
        element = arg;
    }
    char uppercase() {
        if ((element >= 'a') && (element <= 'z')) {
            element += ('A' - 'a');
        }
        return element;
    }
};

int main() {
    mycontainer<int> myint(7);
    mycontainer<char> mychar('j');

    cout << "Increased int: " << myint.increase() << endl;
    cout << "Uppercased char: " << mychar.uppercase() << endl;

    return 0;
}
```

Output:

```
Increased int: 8
Uppercased char: J

-----
Process exited after 0.3398 seconds with return value 0
Press any key to continue . . . |
```

Question 04:

Source Code:

```

#include <iostream>
using namespace std;

template <class T>
class AbstractArray {
protected:
    T* data;
    int capacity;
    int currentSize;

public:
    AbstractArray(int cap) {
        capacity = cap;
        currentSize = 0;
        data = new T[capacity];
    }

    virtual ~AbstractArray() {
        delete[] data;
    }

    virtual void resize() = 0;
    virtual bool isEmpty() = 0;
    virtual bool isFull() = 0;
};

template <class T>
class Queue : public AbstractArray<T> {
private:
    int frontIdx;
    int rearIdx;

public:
    Queue(int cap) : AbstractArray<T>(cap) {
        frontIdx = 0;
        rearIdx = -1;
    }

    bool isEmpty() {
        return this->currentSize == 0;
    }

    bool isFull() {
        return this->currentSize == this->capacity;
    }

    int size() {
        return this->currentSize;
    }

    void resize() {
        int newCapacity = this->capacity * 2;
        T* newData = new T[newCapacity];

        for (int i = 0; i < this->currentSize; i++) {
            newData[i] = this->data[(frontIdx + i) % this->capacity];
        }
    }
};

```

```

        delete[] this->data;
        this->data = newData;
        this->capacity = newCapacity;
        frontIdx = 0;
        rearIdx = this->currentSize - 1;
    }

    void enqueue(T val) {
        if (isFull()) {
            resize();
        }
        rearIdx = (rearIdx + 1) % this->capacity;
        this->data[rearIdx] = val;
        this->currentSize++;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue Empty" << endl;
            return;
        }
        frontIdx = (frontIdx + 1) % this->capacity;
        this->currentSize--;
    }

    T Front() {
        if (!isEmpty()) return this->data[frontIdx];
        return T();
    }

    T Rear() {
        if (!isEmpty()) return this->data[rearIdx];
        return T();
    }
};

int main() {
    Queue<int> q(3);

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Size after 3 enqueues: " << q.size() << endl;
    cout << "Front: " << q.Front() << " | Rear: " << q.Rear() << endl;

    q.enqueue(40);
    cout << "Size after resize (4th enqueue): " << q.size() << endl;

    q.dequeue();
    cout << "Front after dequeue: " << q.Front() << endl;

    return 0;
}

```

Output:

```
Size after 3 enqueues: 3
Front: 10 | Rear: 30
Size after resize (4th enqueue): 4
Front after dequeue: 20

-----
Process exited after 0.3805 seconds with return value 0
Press any key to continue . . .
```

Question 05:Source Code:

```
#include <iostream>
#include <string>
using namespace std;

template <class T>
class AbstractArray {
protected:
    T* data;
    int capacity;
    int currentSize;

public:
    AbstractArray(int cap) {
        capacity = cap;
        currentSize = 0;
        data = new T[capacity];
    }

    virtual ~AbstractArray() {
        delete[] data;
    }

    virtual void resize() = 0;
    virtual bool isEmpty() = 0;
    virtual bool isFull() = 0;
};
```

```

template <class T>
class Queue : public AbstractArray<T> {
private:
    int frontIdx;
    int rearIdx;

public:
    Queue(int cap) : AbstractArray<T>(cap) {
        frontIdx = 0;
        rearIdx = -1;
    }

    bool isEmpty() {
        return this->currentSize == 0;
    }

    bool isFull() {
        return this->currentSize == this->capacity;
    }

    int size() {
        return this->currentSize;
    }

    void resize() {
        int newCapacity = this->capacity * 2;
        T* newData = new T[newCapacity];
        for (int i = 0; i < this->currentSize; i++) {
            newData[i] = this->data[(frontIdx + i) % this->capacity];
        }
        delete[] this->data;
        this->data = newData;
        this->capacity = newCapacity;
        frontIdx = 0;
        rearIdx = this->currentSize - 1;
    }

    void enqueue(T val) {
        if (isFull()) {
            resize();
        }
        rearIdx = (rearIdx + 1) % this->capacity;
        this->data[rearIdx] = val;
        this->currentSize++;
    }

    void dequeue() {
        if (isEmpty()) return;
        frontIdx = (frontIdx + 1) % this->capacity;
        this->currentSize--;
    }

    T Front() {
        if (!isEmpty()) return this->data[frontIdx];
        return T();
    }
};

```

```

int main() {
    Queue<string> printerQueue(5);

    cout << "--- Receiving Print Jobs ---" << endl;
    printerQueue.enqueue("Assignment_Final.pdf");
    printerQueue.enqueue("Resume_2026.docx");
    printerQueue.enqueue("Concert_Tickets.png");
    printerQueue.enqueue("Budget_Report.xlsx");

    cout << "Total jobs in queue: " << printerQueue.size() << endl;

    cout << "\n--- Processing Printer Queue ---" << endl;
    while (!printerQueue.isEmpty()) {
        string currentJob = printerQueue.Front();
        cout << "Printer: Now printing [" << currentJob << "]" << endl;

        printerQueue.dequeue();

        if (!printerQueue.isEmpty()) {
            cout << "Status: Job completed. Next in line: " << printerQueue.Front() << endl;
        } else {
            cout << "Status: All jobs completed. Printer is now idle." << endl;
        }
        cout << "-----" << endl;
    }

    return 0;
}

```

**Output:**

```

--- Receiving Print Jobs ---
Total jobs in queue: 4

--- Processing Printer Queue ---
Printer: Now printing [Assignment_Final.pdf]
Status: Job completed. Next in line: Resume_2026.docx
-----
Printer: Now printing [Resume_2026.docx]
Status: Job completed. Next in line: Concert_Tickets.png
-----
Printer: Now printing [Concert_Tickets.png]
Status: Job completed. Next in line: Budget_Report.xlsx
-----
Printer: Now printing [Budget_Report.xlsx]
Status: All jobs completed. Printer is now idle.
-----

-----
Process exited after 0.2848 seconds with return value 0
Press any key to continue . . .

```

**Example 01:****Source Code:**

```
#include <iostream>
using namespace std;

int Max(int a, int b) {
    return a < b ? b : a;
}

double Max(double a, double b) {
    return a < b ? b : a;
}

int main() {
    cout << Max(10, 20) << endl;
    cout << Max(1.0, 1.5) << endl;

    return 0;
}
```

**Output:**

```
20
1.5

-----
Process exited after 0.2881 seconds with return value 0
Press any key to continue . . .
```

**Example 02:**

**Source Code:**

```
#include <iostream>
using namespace std;

template <typename T1>
T1 Max(T1 a, T1 b) {
    return a < b ? b : a;
}

int main() {
    cout << "Max integer are: " << Max(22, 2) << endl;
    cout << "Max float are: " << Max(3.9, 22.8) << endl;

    return 0;
}
```

**Output:**

```
Max integer are: 22
Max float are: 22.8

-----
Process exited after 0.3625 seconds with return value 0
Press any key to continue . . .
```

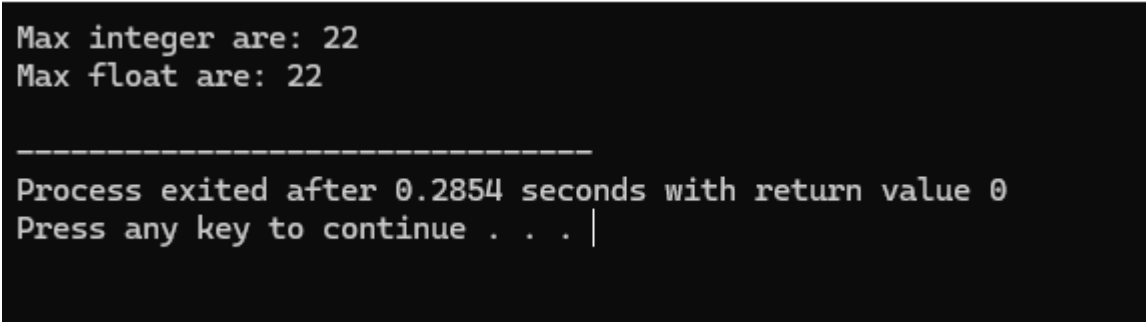
**Example 03:****Source Code:**

```
#include <iostream>
using namespace std;

template <typename T1, typename T2>
T1 Max(T1 a, T2 b) {
    return a < b ? b : a;
}

template <class T1, class T2>
void someFunc(T1 var1, T2 var2) {
}

int main() {
    cout << "Max integer are: " << Max(22, 2.77) << endl;
    cout << "Max float are: " << Max(3.9, 22) << endl;
    return 0;
}
```

**Output:**A screenshot of a terminal window with a black background and white text. The output shows two lines: "Max integer are: 22" and "Max float are: 22". Below these is a horizontal line of dashes. At the bottom, it says "Process exited after 0.2854 seconds with return value 0" and "Press any key to continue . . . |".

```
Max integer are: 22
Max float are: 22

-----
Process exited after 0.2854 seconds with return value 0
Press any key to continue . . . |
```

**Example 04:****Source Code:**

```
#include <iostream>
using namespace std;

template <class T>
class mypair {
    T a, b;

public:
    mypair(T first, T second) {
        a = first;
        b = second;
    }

    T getmax();
};

template <class T>
T mypair<T>::getmax() {
    T retval;
    retval = a < b ? b : a;
    return retval;
}

int main() {
    mypair<int> myobject(100, 75);
    cout << myobject.getmax() << endl;

    return 0;
}
```

**Output:**

```
100

-----
Process exited after 0.3446 seconds with return value 0
Press any key to continue . . .
```

**Example 05:****Source Code:**

```
#include <iostream>
using namespace std;

template <class t1, class t2>
class sample {
    t1 a;
    t2 b;

public:
    void getdata() {
        cout << "Enter value for a and b" << endl;
        cin >> a >> b;
    }

    void display() {
        cout << "Value of a: " << a << endl;
        cout << "Value of b: " << b << endl;
    }
};

int main() {
    sample <int, int> s1;
    sample <float, float> s2;

    cout << "Two integer data" << endl;
    s1.getdata();
    s1.display();

    cout << "\nTwo float data" << endl;
    s2.getdata();
    s2.display();

    return 0;
}
```

**Output:**

```
Two integer data
Enter value for a and b
45 67
Value of a: 45
Value of b: 67

Two float data
Enter value for a and b
45.6 8.9
Value of a: 45.6
Value of b: 8.9

-----
Process exited after 17.13 seconds with return value 0
Press any key to continue . . .
```

# **LAB 11**

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include <iostream>
#include <string>
#include <stdexcept>
#include <cctype>

using namespace std;

class PasswordException : public exception {
private:
    string message;
public:
    PasswordException(string msg) : message(msg) {}

    const char* what() const noexcept override {
        return message.c_str();
    }
};

void validatePassword(string password) {
    bool hasDigit = false;

    if (password.length() < 6) {
        throw PasswordException("Error: Password must be at least 6 characters long!");
    }

    for (char ch : password) {
        if (isdigit(ch)) {
            hasDigit = true;
            break;
        }
    }

    if (!hasDigit) {
        throw PasswordException("Error: Password must contain at least one digit (0-9)!");
    }
}
```

```
int main() {
    string username, password;

    cout << "--- Login System ---" << endl;
    cout << "Enter Username: ";
    getline(cin, username);

    cout << "Enter Password: ";
    getline(cin, password);

    try {
        validatePassword(password);
        cout << "Login successful! Welcome, " << username << "." << endl;
    }
    catch (const PasswordException& e) {
        cout << "Login Failed: " << e.what() << endl;
    }
    catch (const exception& e) {
        cout << "An unexpected error occurred: " << e.what() << endl;
    }

    return 0;
}
```

**Output 01:**

```
--- Login System ---
Enter Username: Areefa
Enter Password: abcd1
Login Failed: Error: Password must be at least 6 characters long!

-----
Process exited after 8.023 seconds with return value 0
Press any key to continue . . .
```

**Output 02:**

```
--- Login System ---
Enter Username: Areefa
Enter Password: areefa123
Login successful! Welcome, Areefa.

-----
Process exited after 5.716 seconds with return value 0
Press any key to continue . . .
```

**Question 02:****Source Code:**

```

#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

class DynamicStack {
private:
    vector<char> stackData;

public:
    void push(char val) {
        stackData.push_back(val);
    }

    void pop() {
        if (!isEmpty()) {
            stackData.pop_back();
        }
    }

    char peek() const {
        if (!isEmpty()) {
            return stackData.back();
        }
        return '\0';
    }

    bool isEmpty() const {
        return stackData.empty();
    }

    string reverseWords(string sentence) {
        string result = "";
        stringstream ss(sentence);
        string word;

        while (ss >> word) {
            for (char ch : word) {
                push(ch);
            }

            while (!isEmpty()) {
                result += peek();
                pop();
            }

            result += " ";
        }

        if (!result.empty()) {
            result.pop_back();
        }

        return result;
    }
};

```

```
int main() {
    DynamicStack myStack;
    string input;

    cout << "Enter a sentence: ";
    getline(cin, input);

    string reversed = myStack.reverseWords(input);

    cout << "Processed Sentence: " << reversed << endl;

    return 0;
}
```

### Output:

```
Enter a sentence: My name is Areefa Samar
Processed Sentence: yM eman si afeerA ramaS

-----
Process exited after 17.48 seconds with return value 0
Press any key to continue . . . |
```

### Question 03:

#### Source Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>

using namespace std;
using namespace std::chrono;

void Sort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter the number of integers (N): ";
    cin >> n;
```

```

vector<int> original;
cout << "Enter " << n << " integers:" << endl;
for (int i = 0; i < n; i++) {
    int val;
    cin >> val;
    original.push_back(val);
}

const int iterations = 1000;

auto start1 = high_resolution_clock::now();
for (int i = 0; i < iterations; i++) {
    vector<int> temp = original;
    Sort(temp);
}
auto stop1 = high_resolution_clock::now();
auto total_ns1 = duration_cast<nanoseconds>(stop1 - start1).count();

auto start2 = high_resolution_clock::now();
for (int i = 0; i < iterations; i++) {
    vector<int> temp = original;
    sort(temp.begin(), temp.end());
}
auto stop2 = high_resolution_clock::now();
auto total_ns2 = duration_cast<nanoseconds>(stop2 - start2).count();

cout << "\n--- Timing Results ---" << endl;
cout << "Custom Sort Time: " << total_ns1 / iterations << " nanoseconds" << endl;
cout << "STL Sort Time: " << total_ns2 / iterations << " nanoseconds" << endl;

vector<int> finalSorted = original;
sort(finalSorted.begin(), finalSorted.end());

cout << "\nSorted Order: ";
for (int x : finalSorted) {
    cout << x << " ";
}
cout << endl;

return 0;
}

```

**Output:**

```

Enter the number of integers (N): 10
Enter 10 integers:
45 12 3 90 10 6 78 8 9 2

--- Timing Results ---
Custom Sort Time: 1894 nanoseconds
STL Sort Time:    1103 nanoseconds

Sorted Order: 2 3 6 8 9 10 12 45 78 90

```

```

-----
Process exited after 28.2 seconds with return value 0
Press any key to continue . . . |

```

**Question 04:****Source Code:**

```
#include <map>
#include <string>
#include <limits>

using namespace std;

void displayAll(const map<string, double>& grades) {
    if (grades.empty()) {
        cout << "\nThe gradebook is currently empty." << endl;
        return;
    }
    cout << "\n--- Current Gradebook ---" << endl;
    for (auto const& [name, grade] : grades) {
        cout << "Student: " << name << " | Grade: " << grade << endl;
    }
}

void retrieveGrade(const map<string, double>& grades) {
    string name;
    cout << "Enter student name to find: ";
    cin.ignore();
    getline(cin, name);

    auto it = grades.find(name);
    if (it != grades.end()) {
        cout << "Grade for " << name << ": " << it->second << endl;
    } else {
        cout << "Error: Student '" << name << "' not found." << endl;
    }
}
```

```

void updateGrade(map<string, double>& grades) {
    string name;
    cout << "Enter student name to update: ";
    cin.ignore();
    getline(cin, name);

    if (grades.count(name)) {
        double newGrade;
        cout << "Enter new grade: ";
        cin >> newGrade;
        grades[name] = newGrade;
        cout << "Grade updated successfully." << endl;
    } else {
        cout << "Error: Student not found. Use 'Add' to create a new entry." << endl;
    }
}

void deleteStudent(map<string, double>& grades) {
    string name;
    cout << "Enter student name to delete: ";
    cin.ignore();
    getline(cin, name);

    if (grades.erase(name)) {
        cout << "Student '" << name << "' removed from records." << endl;
    } else {
        cout << "Error: Student not found." << endl;
    }
}

int main() {
    map<string, double> gradebook;
    int choice;

    cout << "Welcome, Teacher! Student Grade Management System" << endl;

    while (true) {
        cout << "\n1. Add Student\n2. Retrieve Grade\n3. Update Grade\n4. Delete Student\n5. Display All\n6. Exit" << endl;
        cout << "Select an option: ";
        cin >> choice;

        if (choice == 6) break;

        switch (choice) {
            case 1: {
                string name;
                double grade;
                cout << "Enter Student Name: ";
                cin.ignore();
                getline(cin, name);
                cout << "Enter Grade: ";
                cin >> grade;
                gradebook[name] = grade;
                break;
            }
            case 2: retrieveGrade(gradebook); break;
            case 3: updateGrade(gradebook); break;
            case 4: deleteStudent(gradebook); break;
            case 5: displayAll(gradebook); break;
            default: cout << "Invalid choice. Try again." << endl;
        }
    }

    return 0;
}

```

Output:

```
Welcome, Teacher! Student Grade Management System
```

1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit

```
Select an option: 1
```

```
Enter Student Name: Areefa Samar
```

```
Enter Grade: 98
```

1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit

```
Select an option: 1
```

```
Enter Student Name: Ayesha Irfan
```

```
Enter Grade: 78
```

1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit

```
Select an option: 5
```

```
--- Current Gradebook ---
```

```
Student: Areefa Samar | Grade: 98
```

```
Student: Ayesha Irfan | Grade: 78
```

```
1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 2
Enter student name to find: Areefa Samar
Grade for Areefa Samar: 98

1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 3
Enter student name to update: Ayesha Irfan
Enter new grade: 89
Grade updated successfully.

1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 5

--- Current Gradebook ---
Student: Areefa Samar | Grade: 98
Student: Ayesha Irfan | Grade: 89
```

```
1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 4
Enter student name to delete: Ayesha Irfan
Student 'Ayesha Irfan' removed from records.
```

```
1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 5
```

```
--- Current Gradebook ---
Student: Areefa Samar | Grade: 98
```

```
1. Add Student
2. Retrieve Grade
3. Update Grade
4. Delete Student
5. Display All
6. Exit
Select an option: 6
```

```
-----
Process exited after 93.95 seconds with return value 0
Press any key to continue . . .
```

**Question 05:****Source Code:**

```
#include <iostream>
#include <set>
#include <string>

using namespace std;

int main() {
    set<string> guestList;
    string name;
    char choice;

    cout << "--- Party Guest List Manager ---" << endl;

    do {
        cout << "\nEnter guest name: ";
        getline(cin >> ws, name);

        auto result = guestList.insert(name);

        if (result.second) {
            cout << "Success: " << name << " added to the list." << endl;
        } else {
            cout << "Note: " << name << " is already on the list (duplicate ignored)." << endl;
        }

        cout << "Add another guest? (y/n): ";
        cin >> choice;

    } while (choice == 'y' || choice == 'Y');

    cout << "\n--- Final Unique Guest List (Alphabetical) ---" << endl;
    if (guestList.empty()) {
        cout << "The list is empty." << endl;
    } else {
        for (const string& guest : guestList) {
            cout << "- " << guest << endl;
        }

        cout << "\nTotal number of unique guests: " << guestList.size() << endl;
    }

    cout << "\nThank you for using the Guest Manager!" << endl;

    return 0;
}
```

Output:

```
--- Party Guest List Manager ---

Enter guest name: Areefa
Success: Areefa added to the list.
Add another guest? (y/n): y

Enter guest name: Waqar
Success: Waqar added to the list.
Add another guest? (y/n): y

Enter guest name: Areefa
Note: Areefa is already on the list (duplicate ignored).
Add another guest? (y/n): n

--- Final Unique Guest List (Alphabetical) ---
- Areefa
- Waqar

Total number of unique guests: 2

Thank you for using the Guest Manager!

-----
Process exited after 16.82 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include <iostream>
using namespace std;

int main() {
    double numerator, denominator, answer;

    cout << "Enter numerator: ";
    cin >> numerator;
    cout << "Enter denominator: ";
    cin >> denominator;

    try {
        if (denominator == 0) {
            // Throwing a double to match the denominator's type
            throw denominator;
        }

        answer = numerator / denominator;
        cout << numerator << " / " << denominator << " = " << answer << endl;
    }
    catch (double num_exception) {
        cout << "Error: Cannot divide by " << num_exception << endl;
    }

    return 0;
}
```

**Output 01:**

```
Enter numerator: 8
Enter denominator: 2
8 / 2 = 4

-----
Process exited after 6.723 seconds with return value 0
Press any key to continue . . .
```

**Output 02:**

```
Enter numerator: 32
Enter denominator: 0
Error: Cannot divide by 0

-----
Process exited after 6.774 seconds with return value 0
Press any key to continue . . .
```

**Example 02:**

**Source Code:**

```
#include <iostream>
using namespace std;

int main() {
    try {
        throw 10;
    }
    catch (char excp) {
        cout << "Caught char: " << excp << endl;
    }
    catch (...) {
        cout << "Default Exception " << endl;
    }

    return 0;
}
```

**Output:**

```
Default Exception
-----
Process exited after 0.3329 seconds with return value 0
Press any key to continue . . .
```

**Example 03:**

Source Code:

```
#include <iostream>
using namespace std;
int main() {
    double numerator, denominator, arr[4] = {0.0, 0.0, 0.0, 0.0};
    int index;

    cout << "Enter array index: ";
    cin >> index;

    try {
        if (index >= 4 || index < 0) {
            throw "Error: Array out of bounds!";
        }
        cout << "Enter numerator: ";
        cin >> numerator;
        cout << "Enter denominator: ";
        cin >> denominator;

        if (denominator == 0) {
            throw denominator;
        }

        arr[index] = numerator / denominator;
        cout << "Result stored at index " << index << ": " << arr[index] << endl;
    }
    catch (const char* msg) {
        cout << msg << endl;
    }
    catch (double num) {
        cout << "Error: Cannot divide by " << num << endl;
    }
    catch (...) {
        cout << "Unexpected exception!" << endl;
    }

    return 0;
}
```

Output 01:

```
Enter array index: 2
Enter numerator: 34
Enter denominator: 23
Result stored at index 2: 1.47826

-----
Process exited after 134.7 seconds with return value 0
Press any key to continue . . .
```

Output 02:

```
Enter array index: 3
Enter numerator: 23
Enter denominator: 0
Error: Cannot divide by 0

-----
Process exited after 4.981 seconds with return value 0
Press any key to continue . . .
```

#### Example 04:

##### Source Code:

```
#include <iostream>
#include <stdexcept>
using namespace std;

void divide(int numerator, int denominator) {
    try {
        if (denominator == 0) {
            throw runtime_error("Division by zero error");
        }

        cout << "The result is: " << float(numerator) / denominator << endl;
    }
    catch (const exception& ex) {
        cout << "Exception caught: " << ex.what() << endl;
    }
}

int main() {
    divide(10, 0);
    divide(10, 20);

    return 0;
}
```

##### Output:

```
Exception caught: Division by zero error
The result is: 0.5

-----
Process exited after 0.2978 seconds with return value 0
Press any key to continue . . . |
```

#### Example 05:

##### Source Code:

```
#include <iostream>
using namespace std;

class MyException {
public:
    const char * what() {
        return "Attempted to divide by zero!\n";
    }
};

int main() {
    try {
        int x, y;
        cout << "Enter the two numbers : \n";
        cin >> x >> y;

        if (y == 0) {
            MyException z;
            throw z;
        } else {
            cout << "x / y = " << x / y << endl;
        }
    }
    catch (MyException& e) {
        cout << e.what();
    }

    return 0;
}
```

**Output:**

```
Enter the two numbers :
2 0
Attempted to divide by zero!

-----
Process exited after 1.679 seconds with return value 0
Press any key to continue . . . |
```

**Example 06:****Source Code:**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> num {1, 2, 3, 4, 5};

    cout << "Initial Vector: ";
    for (const int& i : num) {
        cout << i << " ";
    }

    num.push_back(6);
    num.push_back(7);

    cout << "\nUpdated Vector: ";
    for (const int& i : num) {
        cout << i << " ";
    }

    return 0;
}
```

**Output:**

```
Initial Vector: 1 2 3 4 5
Updated Vector: 1 2 3 4 5 6 7
-----
Process exited after 0.4242 seconds with return value 0
Press any key to continue . . .
```

**Example 07:****Source Code:**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> num {1, 2, 3, 4, 5};

    cout << "Element at Index 0: " << num.at(0) << endl;
    cout << "Element at Index 2: " << num.at(2) << endl;
    cout << "Element at Index 4: " << num.at(4) << endl;

    return 0;
}
```

**Output:**

```
Element at Index 0: 1
Element at Index 2: 3
Element at Index 4: 5

-----
Process exited after 0.3626 seconds with return value 0
Press any key to continue . . .
```

**Example 08:****Source Code:**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> num {1, 2, 3, 4, 5};

    cout << "Initial Vector: ";
    for (const int& i : num) {
        cout << i << " ";
    }

    num.at(1) = 9;
    num.at(4) = 7;

    cout << "\nUpdated Vector: ";
    for (const int& i : num) {
        cout << i << " ";
    }

    return 0;
}
```

**Output:**

```
Initial Vector: 1 2 3 4 5
Updated Vector: 1 9 3 4 7
-----
Process exited after 0.4072 seconds with return value 0
Press any key to continue . . .
```

**Example 09:****Source Code:**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> prime_numbers{2, 3, 5, 7};

    cout << "Initial Vector: ";
    for (int i : prime_numbers) {
        cout << i << " ";
    }

    prime_numbers.pop_back();

    cout << "\nUpdated Vector: ";
    for (int i : prime_numbers) {
        cout << i << " ";
    }

    return 0;
}
```

**Output:**

```
Initial Vector: 2 3 5 7
Updated Vector: 2 3 5
-----
Process exited after 0.3616 seconds with return value 0
Press any key to continue . . .
```

**Example 10:****Source Code:**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> numbers {1, 2, 3, 4, 5};

    vector<int>::iterator itr = numbers.begin();
    cout << "First Element: " << *itr << " " << endl;

    itr = numbers.end() - 1;
    cout << "Last Element: " << *itr << endl;

    return 0;
}
```

**Output:**

```
First Element: 1
Last Element: 5

-----
Process exited after 0.3812 seconds with return value 0
Press any key to continue . . .
```

**Example 11:****Source Code:**

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> m;

    m["one"] = 1;
    m["two"] = 2;
    m["three"] = 3;

    map<string, int>::iterator it = m.begin();

    while (it != m.end()) {
        cout << "Key: " << it->first << ", Value: " << it->second << endl;
        ++it;
    }

    return 0;
}
```

**Output:**

```
Key: one, Value: 1
Key: three, Value: 3
Key: two, Value: 2

-----
Process exited after 0.4184 seconds with return value 0
Press any key to continue . . .
```

**Example 12:****Source Code:**

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<char> a;
    set<char>::iterator p;

    a.insert('G');
    a.insert('F');
    a.insert('G');

    p = a.begin();
    while (p != a.end()) {
        cout << *p << ' ';
        p++;
    }
    cout << '\n';

    return 0;
}
```

**Output:**

F G

-----  
Process exited after 0.3488 seconds with return value 0  
Press any key to continue . . .

# **LAB 12**

## OBJECT ORIENTED PROGRAMMING (CT-260)

### Question 01:

#### Source Code:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
int main() {
    char str[100];
    cout << "Enter a string: ";
    cin.getline(str, 100);

    int len = strlen(str);
    cout << "Length: " << len << endl;

    // Write to file
    ofstream fout("string.txt");
    fout << str;
    fout.close();

    // Read from file
    char ch;
    ifstream fin("string.txt");
    cout << "Stored characters: ";
    while (fin.get(ch)) {
        cout << ch;
    }
    fin.close();

    return 0;
}
```

#### Output:

```
Enter a string: Areefa
Length: 6
Stored characters: Areefa
-----
Process exited after 3.278 seconds with return value 0
Press any key to continue . . .
```

**Question 02:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream fout("source.txt");
    fout << "This is the source file content.";
    fout.close();

    ifstream fin("source.txt");
    ofstream fcopy("destination.txt");

    char ch;
    while (fin.get(ch)) {
        fcopy.put(ch);
    }

    fin.close();
    fcopy.close();

    cout << "File copied successfully!" << endl;
    return 0;
}
```

**Output:**

```
File copied successfully!
```

```
-----
Process exited after 0.3171 seconds with return value 0
Press any key to continue . . .
```

**Question 03:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

class Person {
public:
    char name[50];
    int age;

    // Change constructor to take const char*:
    Person(const char* n, int a) {
        strcpy(name, n);
        age = a;
    }
};

int main() {
    Person person1("Areefa", 18);

    // Write to binary file
    ofstream fout("person.bin", ios::binary);
    fout.write((char*)&person1, sizeof(person1));
    fout.close();

    // Read from binary file
    Person temp("", 0);
    ifstream fin("person.bin", ios::binary);
    fin.read((char*)&temp, sizeof(temp));
    fin.close();

    cout << "Name: " << temp.name << endl;
    cout << "Age: " << temp.age << endl;

    return 0;
}
```

Output:

```
Name: Areefa
Age: 18

-----
Process exited after 0.3425 seconds with return value 0
Press any key to continue . . . |
```

Question 04:Source Code:

```
#include <iostream>
#include <fstream>
using namespace std;

class Participant {
public:
    int id, score;
    char name[50];

    void Input() {
        cout << "Enter ID: "; cin >> id;
        cout << "Enter Name: "; cin >> name;
        cout << "Enter Score: "; cin >> score;

        ofstream fout("participant.dat", ios::binary | ios::app);
        fout.write((char*)this, sizeof(*this));
        fout.close();
    }

    void Output() {
        int searchId;
        cout << "Enter ID to search: "; cin >> searchId;

        Participant temp;
        ifstream fin("participant.dat", ios::binary);
        bool found = false;
        while (fin.read((char*)&temp, sizeof(temp))) {
            if (temp.id == searchId) {
                cout << "Name: " << temp.name << ", Score: " << temp.score << endl;
                found = true;
                break;
            }
        }
        fin.close();
        if (!found) cout << "ID not found." << endl;
    }
}
```

```

void Max() {
    Participant temp;
    ifstream fin("participant.dat", ios::binary);
    int maxScore = -1;
    char maxName[50];

    while (fin.read((char*)&temp, sizeof(temp))) {
        if (temp.score > maxScore) {
            maxScore = temp.score;
            strcpy(maxName, temp.name);
        }
    }
    fin.close();
    cout << "Highest Score: " << maxName << " with " << maxScore << endl;
}

};

int main() {
    Participant p;
    int choice;
    cout << "1. Input  2. Output  3. Max\nChoice: ";
    cin >> choice;

    if (choice == 1) p.Input();
    else if (choice == 2) p.Output();
    else p.Max();

    return 0;
}

```

**Output:**

```

1. Input  2. Output  3. Max
Choice: 1
Enter ID: 1
Enter Name: Areefa
Enter Score: 90

-----
Process exited after 11.03 seconds with return value 0
Press any key to continue . . . |

```

```

1. Input  2. Output  3. Max
Choice: 2
Enter ID to search: 1
Name: Areefa, Score: 90

-----
Process exited after 3.041 seconds with return value 0
Press any key to continue . . . |

```

```
1. Input  2. Output  3. Max
Choice: 3
Highest Score: Areefa with 90

-----
Process exited after 3.062 seconds with return value 0
Press any key to continue . . .
```

### Question 05:

#### Source Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Create the file first (for testing)
    ofstream fout("STORY.TXT");
    fout << "The rose is red.\n";
    fout << "A girl is playing there.\n";
    fout << "There is a playground.\n";
    fout << "An airplane is in the sky.\n";
    fout << "Numbers are not allowed in the password.\n";
    fout.close();

    // Count Lines not starting with 'A'
    ifstream fin("STORY.TXT");
    string line;
    int count = 0;

    while (getline(fin, line)) {
        if (!line.empty() && line[0] != 'A') {
            count++;
        }
    }
    fin.close();

    cout << "Lines not starting with 'A': " << count << endl;
    // Output: 3
    return 0;
}
```

#### Output:

```
Lines not starting with 'A': 3

-----
Process exited after 0.3133 seconds with return value 0
Press any key to continue . . .
```

**Example 01:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream myfile("file1.txt");
    if (!myfile) {
        cout << "File not created!";
    } else {
        cout << "File created successfully!";
        myfile.close();
    }
    return 0;
}
```

**Output:**

```
File created successfully!
-----
Process exited after 0.349 seconds with return value 0
Press any key to continue . . . |
```

**Example 02:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream my_file;
    my_file.open("file2.txt", ios::out);
    if (!my_file) {
        cout << "File not created!";
    } else {
        cout << "File created successfully!";
        my_file.close();
    }
    return 0;
}
```

**Output:**

```
File created successfully!
-----
Process exited after 0.272 seconds with return value 0
Press any key to continue . . . |
```

**Example 03:**

**Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

bool fileExists(const string& filePath) {
    ifstream file(filePath);
    return file.good();
}

bool canAccessFile(const string& filePath) {
    ifstream file(filePath);
    return file.is_open();
}

int main() {
    string filePath = "file1.txt";

    if (fileExists(filePath)) {
        if (canAccessFile(filePath)) {
            cout << "File exists and can be accessed." << endl;
            ifstream file(filePath);
            // perform operations...
        } else {
            cout << "Cannot access the file." << endl;
        }
    } else {
        cout << "File does not exist." << endl;
    }

    return 0;
}
```

**Output:**

```
File exists and can be accessed.

-----
Process exited after 0.3265 seconds with return value 0
Press any key to continue . . . |
```

**Example 04:**

**Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream my_file;
    my_file.open("file2.txt", ios::out);
    if (!my_file) {
        cout << "File not created!";
    } else {
        cout << "File created successfully!" << endl;
        my_file << "OOP BSAI - Section B";
        my_file.close();
    }
    return 0;
}
```

**Output:**

```
File created successfully!

-----
Process exited after 0.3375 seconds with return value 0
Press any key to continue . . . |
```

**Example 05:**

**Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream my_file;
    my_file.open("file2.txt", ios::in);
    if (!my_file) {
        cout << "No such file";
    } else {
        char ch;
        while (1) {
            my_file >> ch;
            if (my_file.eof())
                break;
            cout << ch;
        }
    }
    my_file.close();
    return 0;
}
```

**Output:**

OOPBSAI-SectionB

-----  
Process exited after 0.3123 seconds with return value 0  
Press any key to continue . . . |

**Example 06:**

**Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream my_file;
    my_file.open("file2.txt", ios::in);

    if (!my_file) {
        cout << "No such file";
    } else {
        string line;
        while (!my_file.eof()) {
            getline(my_file, line);
            cout << line << endl;
        }
        my_file.close();
    }

    return 0;
}
```

**Output:**

OOP BSAI – Section B

-----  
Process exited after 0.3326 seconds with return value 0  
Press any key to continue . . . |

**Example 07:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream openFile("file2.txt");
    char ch;

    if (!openFile) {
        cout << "No such file";
    } else {
        while (!openFile.eof()) {
            openFile.get(ch);
            cout << ch;
        }
    }

    openFile.close();
    return 0;
}
```

**Output:**

```
OOP BSAI - Section BB
-----
Process exited after 0.3709 seconds with return value 0
Press any key to continue . . . |
```

**Example 08:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
    char data[100];
    // Write to file
    ofstream outfile;
    outfile.open("array.dat");

    cout << "Writing to the file" << endl;
    cout << "Enter your name: ";
    cin.getline(data, 100);
    outfile << data << endl;

    cout << "Enter your age: ";
    cin >> data;
    cin.ignore();
    outfile << data << endl;

    outfile.close();

    // Read from file
    ifstream infile;
    infile.open("array.dat");

    cout << endl << "Reading data from file" << endl;
    infile >> data;
    cout << "Name: " << data << endl;

    infile >> data;
    cout << "Age: " << data << endl;

    infile.close();

    return 0;
}
```

**Output:**

```
Writing to the file
Enter your name: Areefa
Enter your age: 18

Reading data from file
Name: Areefa
Age: 18

-----
Process exited after 4.161 seconds with return value 0
Press any key to continue . . .
```

**Example 09:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;
class student {
    int roll;
    char name[25];
    float marks;
public:
    void getdata() {
        cout << "Enter roll no: ";
        cin >> roll;
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter marks: ";
        cin >> marks;
    }

    void addRecord() {
        fstream f;
        student s;
        f.open("student.dat", ios::app | ios::binary);
        s.getdata();
        f.write((char*)&s, sizeof(s));
        f.close();
    }
};
int main() {
    student s;
    char c = 'n';
    do {
        s.addRecord();
        cout << "Do you want to add another record? Press Y to continue: ";
        cin >> c;
    } while (c == 'y' || c == 'Y');

    cout << endl << "Data written successfully";
    return 0;
}
```

**Output:**

```
Enter roll no: 62
Enter name: Areefa
Enter marks: 90
Do you want to add another record? Press Y to continue: Y
Enter roll no: 60
Enter name: Laiba
Enter marks: 89
Do you want to add another record? Press Y to continue: N

Data written successfully
-----
Process exited after 24.99 seconds with return value 0
Press any key to continue . . .
```

**Example 10:****Source Code:**

```
#include <iostream>
#include <fstream>
using namespace std;

class student {
    int roll;
    char name[25];
    float marks;
public:
    void displayStudent() {
        cout << "Roll no: " << roll << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }

    void Readdata() {
        fstream f;
        student s;
        f.open("student.dat", ios::in | ios::binary);
        if (f.read((char*)&s, sizeof(s))) {
            cout << endl << endl;
            s.displayStudent();
        } else {
            cout << "Error in reading data from file...\n";
        }
    }
};

int main() {
    student s;
    s.Readdata();
    return 0;
}
```

**Output:**

```
Roll no: 62
Name: Areefa
Marks: 90
```

```
-----
Process exited after 0.3022 seconds with return value 0
Press any key to continue . . . |
```

# CCP REPORT

**NED UNIVERSITY OF ENGINEERING & TECHNOLOGY**

Department of Computer Science &amp; Information Technology

CT-260 | Object Oriented Programming | Spring 2026

**COMPLEX COMPUTING PROBLEM (CCP) REPORT**

# Password Security Suite

*A Console-Based C++ Application**Demonstrating Core OOP Principles Applied to Real-World Cybersecurity*

|                 |                                      |
|-----------------|--------------------------------------|
| Course          | CT-260 — Object Oriented Programming |
| Semester        | 2nd Semester   Spring 2026           |
| Section         | B                                    |
| Submitted To    | Dr. Murk Marvi / Mr. Ahmed Zaki      |
| Group Members   | Areefa Samar   Arbish                |
| Roll Numbers    | CT-25062   CT-25053                  |
| Submission Date | 20-May-2026                          |

## TABLE OF CONTENTS

---

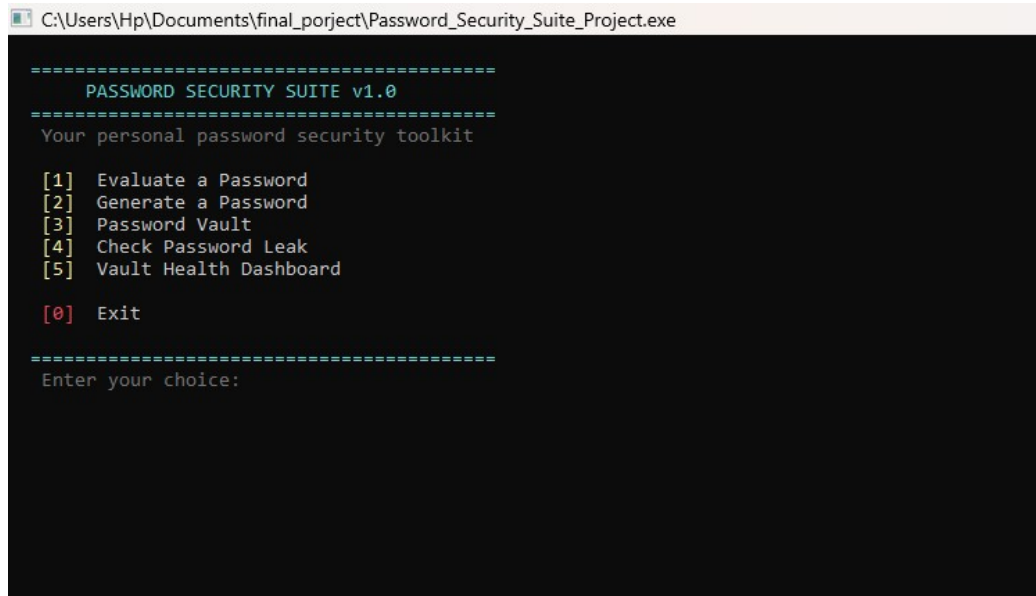
|                                      |    |
|--------------------------------------|----|
| 1. Problem Statement                 | 3  |
| 2. OOP Principles Identification     | 5  |
| 3. System Design & UML Class Diagram | 7  |
| 4. Solution Definition               | 10 |
| 5. Design Pattern Application        | 18 |
| 6. Limitations & Future Work         | 19 |
| 7. Team Contribution Table           | 20 |
| 8. Conclusion                        | 22 |
| 9. References                        | 23 |
| 10. Appendix A (Source Code)         | 24 |

---

# 1. Problem Statement

## 1.1 Background

Most users still rely on weak, reused, or leaked passwords directly enabling identity theft and data breaches. Commercial password managers are effective but often require subscriptions or cloud access. A local, console-based tool that handles password generation, evaluation, breach detection, and secure storage fills this gap without requiring any internet connectivity or paid services.



```
=====
PASSWORD SECURITY SUITE v1.0
=====
Your personal password security toolkit

[1] Evaluate a Password
[2] Generate a Password
[3] Password Vault
[4] Check Password Leak
[5] Vault Health Dashboard

[0] Exit

=====
Enter your choice:
```

Figure 1 — Password Security Suite main menu

## 1.2 Computing Challenge

The system must perform several non-trivial computing tasks simultaneously:

- Heuristic strength analysis covering length, character variety, common patterns, and dictionary matching.
- Password generation via two algorithms random character selection and word-phrase composition.
- Encrypted local vault with master-password authentication and brute-force lockout after three failures.
- Offline breach detection against a local dataset of known compromised passwords.
- Health reporting by combining all subsystems into one consolidated analytical dashboard.

### Why It Matters

Password-related breaches account for over 80% of hacking incidents globally. Password Security Suite is a fully offline, free alternative to commercial password managers built entirely from first principles using C++ and OOP.

### 1.3 OOP Relevance

Every feature maps naturally to a class with clear, single responsibilities. Inheritance, composition, and aggregation arise from the problem itself not forced in to satisfy a checklist. This makes the project a genuine demonstration of OOP applied to a real engineering problem rather than an academic exercise.

## 2. OOP Principles Identification

All four pillars of OOP appear naturally in this project. Each is justified below with the specific classes and design decisions that demonstrate it.

### 2.1 Encapsulation

Encapsulation means bundling data and the methods that operate on it inside a class, and restricting external access to internal state.

In PasswordEvaluator, the private fields (password, score, level) and six private checking methods (checkLength(), checkUppercase(), checkDigits(), checkSymbols(), checkCommonPatterns(), checkLeakedList()) are completely hidden. The public interface only exposes setPassword(), evaluate(), displayReport(), and read-only getters preventing external code from bypassing the scoring logic or reading a stale score.

In AuthGuard, all security-critical fields (failCount, isLocked, masterHash, lockTime) are private. External code can only call verifymasterPassword() and getLockStatus() it cannot reset the fail counter or clear the lock directly, which would defeat the brute-force protection.

### 2.2 Inheritance

Inheritance allows a derived class to acquire the properties and behaviours of its base class, promoting code reuse and a natural IS-A hierarchy.

PasswordGenerator is the abstract base class, declaring the pure virtual generate() and holding shared configuration (length, useUppercase, useDigits, useSymbols, avoidLookAlikes) with validated setters. Both RandomGenerator and PhraseGenerator inherit from it reusing all the setter/getter infrastructure without reimplementing it. Each subclass IS-A PasswordGenerator and must provide its own generate() implementation.

### 2.3 Polymorphism

Polymorphism allows one interface to have multiple implementations, resolved at runtime based on the actual type of the object.

Both RandomGenerator and PhraseGenerator override generate(). When invoked through a PasswordGenerator pointer or reference, C++ dispatches to the correct subclass at runtime. RandomGenerator produces a character-pool password; PhraseGenerator produces a word-based passphrase. Same call different output depending entirely on the object's runtime type.

### 2.4 Abstraction

Abstraction means hiding complex implementation details behind a clean, simple interface.

PasswordGenerator cannot be instantiated directly because generate() is pure virtual. Users must work through a concrete subclass, but always via the common PasswordGenerator interface they never need to know how the character pool is built or how words are selected. Similarly, PasswordEvaluator callers simply call evaluate() without knowing how the score is computed or how crack time is estimated.

| OOP Principle | Where It Appears                               | Specific Classes                                     |
|---------------|------------------------------------------------|------------------------------------------------------|
| Encapsulation | Private data + restricted public interface     | PasswordEvaluator, AuthGuard, PasswordVault          |
| Inheritance   | Base class + concrete subclasses sharing state | PasswordGenerator → RandomGenerator, PhraseGenerator |
| Polymorphism  | Virtual generate() dispatched at runtime       | RandomGenerator, PhraseGenerator override generate() |
| Abstraction   | Pure virtual base; implementation hidden       | PasswordGenerator (abstract), PasswordEvaluator      |

Table 1 — OOP principles mapping — where each pillar appears in the system

## 3. System Design & UML Class Diagram

### 3.1 Architecture Overview

The system splits into two independent subsystems the Input & Analysis Engine (Partner A) and the Storage & Reporting Engine (Partner B) that communicate through pre-agreed public interfaces. The overall design is layered: user interaction on top, feature-level classes in the middle, and file I/O at the base.

### 3.2 Class Responsibilities

| Class                     | Role                | Responsibilities                                                                                                                                            |
|---------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PasswordGenerator</b>  | Abstract Base       | Declares pure virtual generate(). Stores shared config: length, character-type flags, look-alike avoidance. Cannot be instantiated.                         |
| <b>RandomGenerator</b>    | Concrete Subclass   | Inherits from PasswordGenerator. Overrides generate() to build a random character-pool password. Private buildCharPool() assembles the allowed set.         |
| <b>PhraseGenerator</b>    | Concrete Subclass   | Inherits from PasswordGenerator. Overrides generate() to produce a word-based passphrase (e.g. Blue\$Tiger\$Runs49). Configurable word count and separator. |
| <b>PasswordEvaluator</b>  | Standalone Class    | Evaluates strength across six criteria. Computes 0-100 score and maps to StrengthLevel enum. Provides displayReport() for formatted output.                 |
| <b>PasswordEntry</b>      | Value Object        | Stores one vault record: label, encrypted password, date added. Validates date format on construction.                                                      |
| <b>AuthGuard</b>          | Security Controller | Verifies master password. Tracks failed attempts; enforces 30-second lockout after three failures. All state is private.                                    |
| <b>MasterPasswordFile</b> | Utility (Static)    | Manages the master_passwords.txt file. Static methods to check existence, save credentials, and verify a supplied password.                                 |
| <b>PasswordVault</b>      | Aggregate / Manager | Owens vector<PasswordEntry> and composes AuthGuard. Handles create, open, save, delete, and search. XOR-encrypts passwords before file write.               |
| <b>LeakChecker</b>        | Standalone Checker  | Loads a local leaked-password file into a vector<string> at construction. Exposes isLeaked() for case-insensitive lookup.                                   |
| <b>HealthDashboard</b>    | Report Aggregator   | Combines PasswordVault, LeakChecker, and PasswordEvaluator to scan all vault entries, tally weak/compromised passwords, and display a health report.        |

Table 2 — Class responsibilities — all ten classes with roles and descriptions

### 3.3 Class Relationships

| From               | Relationship        | To                   | Why                                               |
|--------------------|---------------------|----------------------|---------------------------------------------------|
| RandomGenerator    | Inheritance         | PasswordGenerator    | IS-A PasswordGenerator                            |
| PhraseGenerator    | Inheritance         | PasswordGenerator    | IS-A PasswordGenerator                            |
| PasswordVault      | Composition (owns)  | AuthGuard            | AuthGuard lifecycle tied to Vault                 |
| PasswordVault      | Composition (owns)  | PasswordEntry[ ]     | Entries created & destroyed within Vault          |
| HealthDashboard    | Aggregation (uses)  | PasswordVault        | Dashboard uses Vault but does not own it          |
| HealthDashboard    | Aggregation (uses)  | LeakChecker          | Dashboard uses LeakChecker without owning it      |
| HealthDashboard    | Aggregation (uses)  | PasswordEvaluator    | Dashboard uses Evaluator without owning it        |
| PasswordEvaluator  | Association (reads) | leaked_passwords.txt | Evaluator reads file via checkLeakedList()        |
| LeakChecker        | Association (reads) | leaked_passwords.txt | LeakChecker loads the file at construction        |
| MasterPasswordFile | Association (r/w)   | master_passwords.txt | Static class manages vault credential persistence |

Table 3 — Class relationships — inheritance, composition, aggregation, and association

### 3.4 UML Notation Key

| Symbol              | Meaning                        | Example in this Project                    |
|---------------------|--------------------------------|--------------------------------------------|
| ▲ (hollow arrow up) | Inheritance / Generalisation   | RandomGenerator → PasswordGenerator        |
| ◆ (filled diamond)  | Composition (strong ownership) | PasswordVault ◆ → AuthGuard                |
| ◇ (hollow diamond)  | Aggregation (weak ownership)   | HealthDashboard ◇ → PasswordVault          |
| —► (solid arrow)    | Association (uses / reads)     | PasswordEvaluator → → leaked_passwords.txt |

|            |                           |                              |
|------------|---------------------------|------------------------------|
| «abstract» | Abstract class stereotype | PasswordGenerator            |
| «static»   | Static method stereotype  | MasterPasswordFile methods   |
| # prefix   | Protected access modifier | PasswordGenerator attributes |
| - prefix   | Private access modifier   | AuthGuard::failCount         |
| + prefix   | Public access modifier    | All public methods           |

Table 4 — UML notation key — symbols used in the class diagram

### 3.5 UML Class Diagram

The diagram below illustrates the full class hierarchy, relationships, and key attributes/methods for each class in the system. Standard UML notation is used throughout.

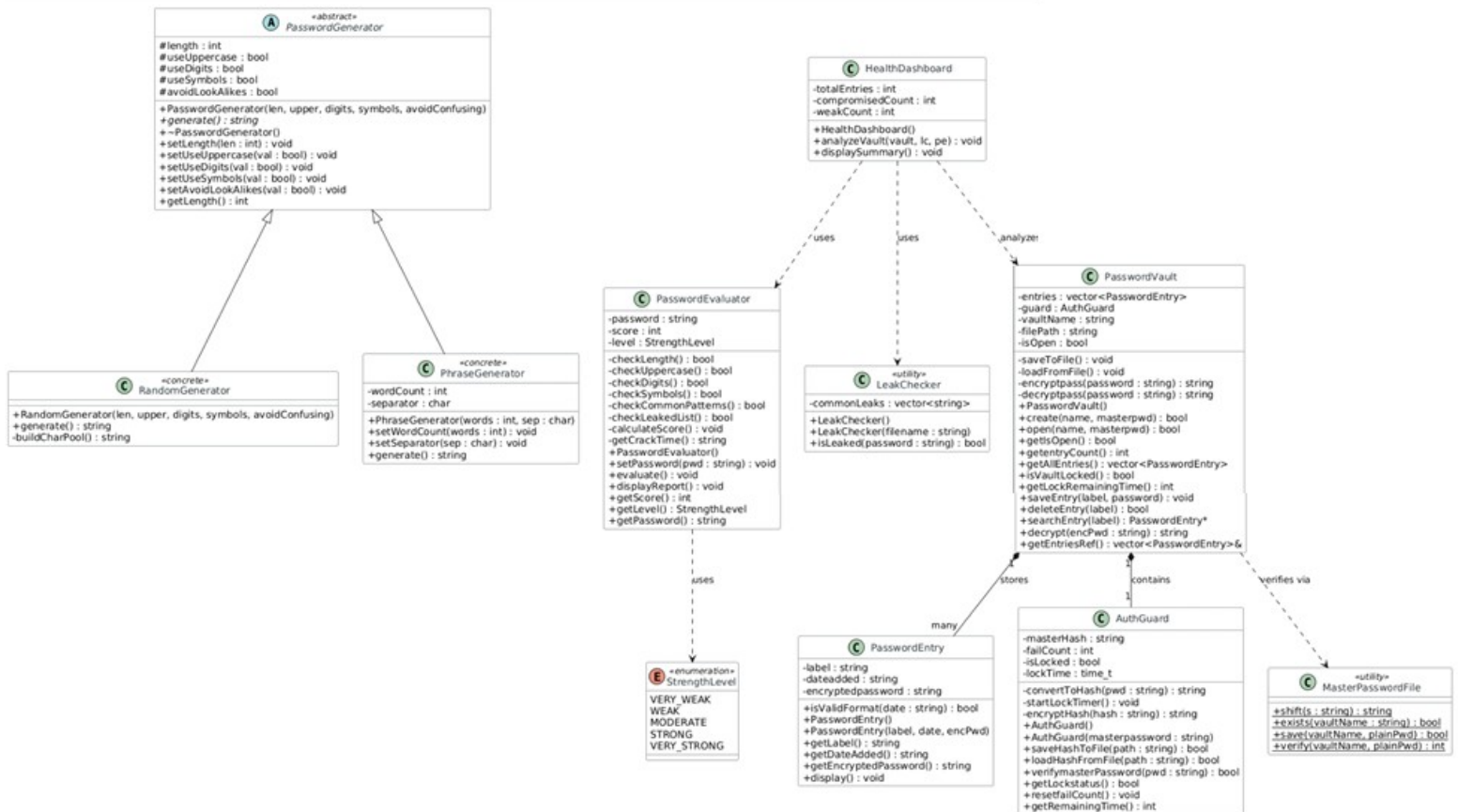


Figure 2 — UML Class Diagram — full system hierarchy

## 4. Solution Definition

### 4.1 Feature 1 — Password Evaluator

PasswordEvaluator receives a plaintext password via setPassword(). Calling evaluate() computes a 0-100 score using six private checks, each encapsulated inside the class and invisible to external code.

| Criterion            | Points     | Logic                                                        |
|----------------------|------------|--------------------------------------------------------------|
| Length               | +5 to +30  | >=8: +12   >=12: +20   >=16: +25   >=20: +30                 |
| Lowercase present    | +10        | At least one lowercase character                             |
| Uppercase present    | +15        | At least one uppercase character                             |
| Digits present       | +15        | At least one numeric digit                                   |
| Symbols present      | +20        | At least one symbol from the defined set                     |
| Common pattern found | -20        | Deducted if patterns like '123', 'abc', or 'qwerty' detected |
| Found in leaked list | Score -> 0 | Score forced to 0; password is known compromised             |

Table 5 — Password Evaluator scoring criteria — points awarded per criterion

Score mapping to StrengthLevel: VERY\_WEAK (0-19), WEAK (20-39), MODERATE (40-59), STRONG (60-79), VERY\_STRONG (80-100). displayReport() shows the score, an ASCII progress bar, a crack-time estimate, per-criterion pass/fail marks, and targeted improvement suggestions.

```

C:\Users\Hp\Documents\final_porject\Password_Security_Suite_Project.exe

=====
PASSWORD STRENGTH EVALUATOR
=====

Enter password to evaluate: arbish789

=====
PASSWORD STRENGTH REPORT
=====
Password   : arbish789
Score      : 37 / 100
Strength   : [#####-----] WEAK
Crack Time : A few minutes

---- Criteria Breakdown ----
[PASS] Length is 8 or more characters
[FAIL] Contains uppercase letters (A-Z)
[PASS] Contains numbers (0-9)
[FAIL] Contains symbols (@#$!...)
[PASS] No common weak patterns (123, abc, qwerty)
[PASS] Not found in known leaked passwords list

---- Suggestions ----
-> Add at least one uppercase letter (e.g. A, B, C).
-> Add a symbol like @, #, $, or ! to boost strength.
-> Consider making it 12+ characters for better protection.
=====

Evaluate another password? (y/n):

```

Figure 3 — Password Evaluator output for a weak password

```

=====
PASSWORD STRENGTH REPORT
=====
Password   : A%&u09*1
Score      : 72 / 100
Strength   : [#####-----] STRONG
Crack Time : Several years

---- Criteria Breakdown ----
[PASS] Length is 8 or more characters
[PASS] Contains uppercase letters (A-Z)
[PASS] Contains numbers (0-9)
[PASS] Contains symbols (@#$!...)
[PASS] No common weak patterns (123, abc, qwerty)
[PASS] Not found in known leaked passwords list

---- Suggestions ----
-> Consider making it 12+ characters for better protection.
=====

Evaluate another password? (y/n):

```

Figure 4 — Password Evaluator output for a strong password

## 4.2 Feature 2 — Password Generator

The user selects Phrase Mode (PhraseGenerator) or Random Mode (RandomGenerator). The program asks a sequence of questions tailored to the selected mode before generating.

|                                      |                                                                                                                                                                          |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Random Mode — buildCharPool()</b> | pool <- lowercase alphabet if useUppercase -> add A-Z if useDigits -> add 0-9 if useSymbols -> add @#\$!%&* for i in 0 to length: password += pool[rand() % pool.size()] |
| <b>Phrase Mode — generate()</b>      | for i in 0 to wordCount: pick random word from 56-word list if i < wordCount-1: append separator append random 2-digit number (10-99)<br>Example: Blue\$Tiger\$Runs49    |

Table 6 — Password Generator algorithm pseudocode — Random Mode and Phrase Mode

```

C:\Users\Hp\Documents\final_porject\Password_Security_Suite_Project.exe

=====
PASSWORD GENERATOR
=====

Do you need to remember this password, or will it be saved in the vault?
[1] I need to remember it (Phrase mode)
[2] I will save it (Random mode)

Enter choice: 2

How strong does it need to be?
[1] Basic (good for low-risk accounts)
[2] Strong (recommended for personal accounts)
[3] Maximum (banking, work, email)
Enter choice: 3

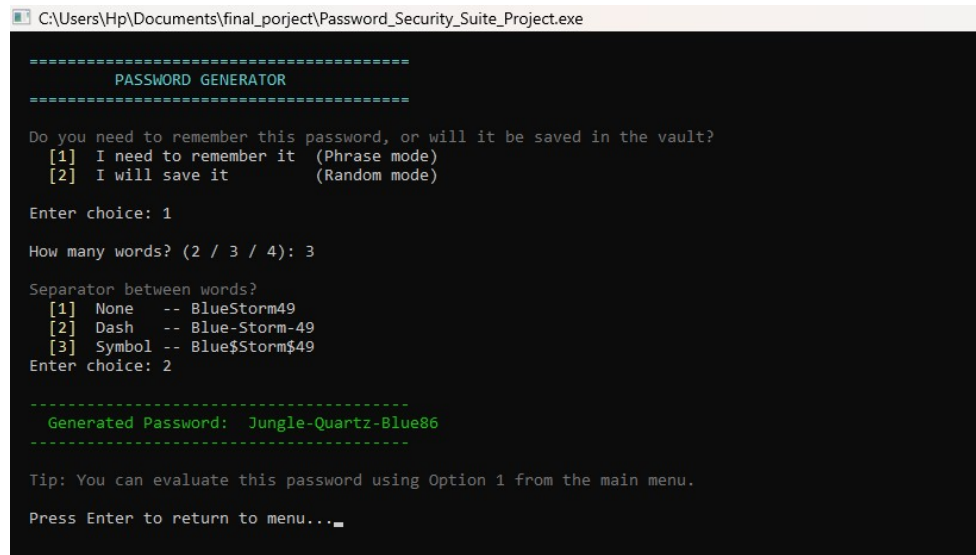
Maximum mode: All character types enabled. Length set to 20.

-----
Generated Password: akSwXvgVs2SpgW7W9Jb8
-----

Would you like to save this password to your vault? (y/n):

```

Figure 5 — Password Generator in Random Mode



```

C:\Users\Hp\Documents\final_project\Password_Security_Suite_Project.exe

=====
PASSWORD GENERATOR
=====

Do you need to remember this password, or will it be saved in the vault?
[1] I need to remember it (Phrase mode)
[2] I will save it (Random mode)

Enter choice: 1

How many words? (2 / 3 / 4): 3

Separator between words?
[1] None -- BlueStorm49
[2] Dash -- Blue-Storm-49
[3] Symbol -- Blue$Storm$49
Enter choice: 2

-----
Generated Password: Jungle-Quartz-Blue86
-----

Tip: You can evaluate this password using Option 1 from the main menu.

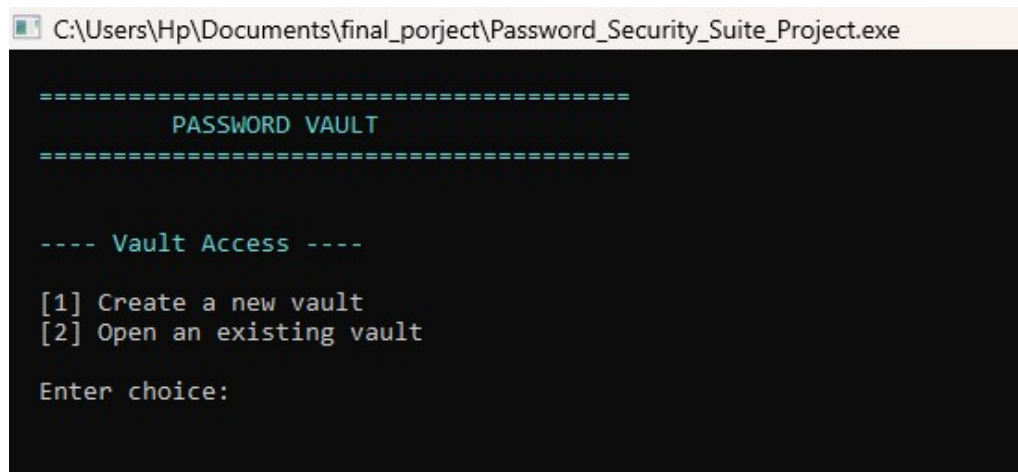
Press Enter to return to menu...

```

Figure 6 — Password Generator in Phrase Mode

### 4.3 Feature 3 — Password Vault

PasswordVault has two lifecycle phases: creation and access. On creation, the master password is hashed and saved to `master_passwords.txt` via `MasterPasswordFile::save()`. On access, `MasterPasswordFile::verify()` compares the stored entry. AuthGuard tracks failures and enforces a 30-second lockout after three wrong attempts. Each stored password is XOR-encrypted with key 9 before being written to a per-vault CSV file. The same XOR operation reverses the encryption on read.



```

C:\Users\Hp\Documents\final_project\Password_Security_Suite_Project.exe

=====
PASSWORD VAULT
=====

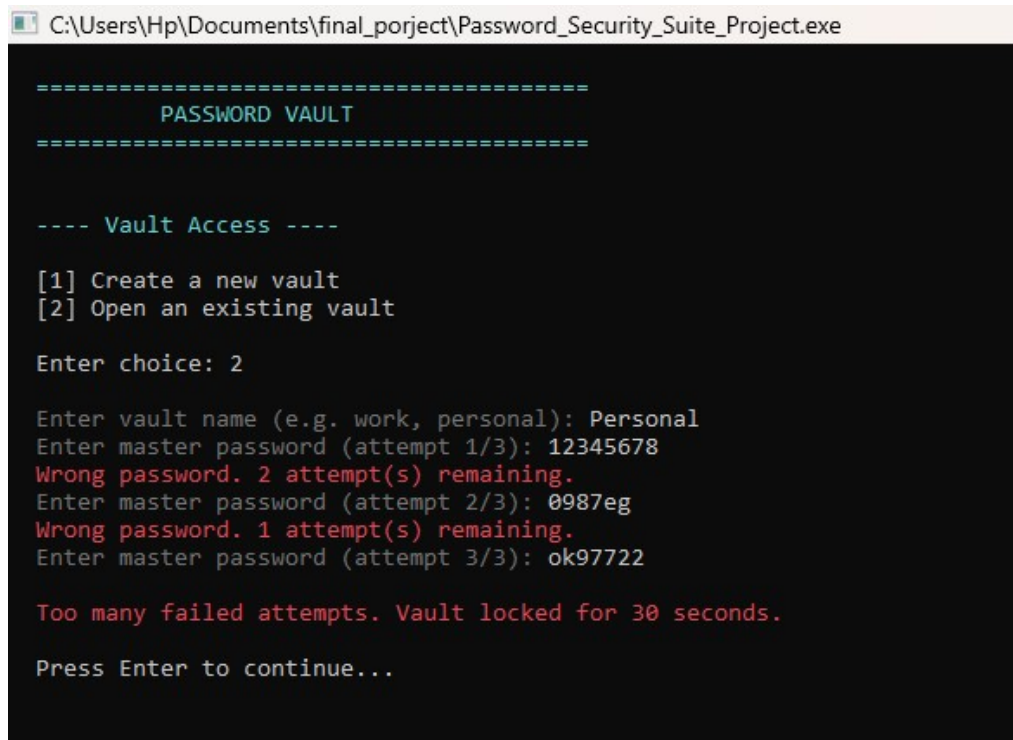
---- Vault Access ----

[1] Create a new vault
[2] Open an existing vault

Enter choice:

```

Figure 7 — Password Vault — vault access menu



```
C:\Users\Hp\Documents\final_porject>Password_Security_Suite_Project.exe

=====
          PASSWORD VAULT
=====

---- Vault Access ----

[1] Create a new vault
[2] Open an existing vault

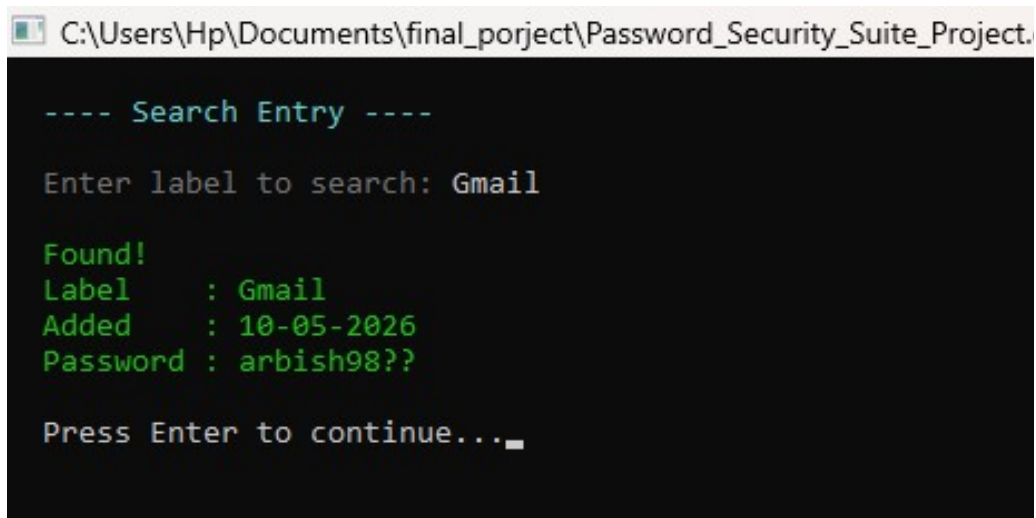
Enter choice: 2

Enter vault name (e.g. work, personal): Personal
Enter master password (attempt 1/3): 12345678
Wrong password. 2 attempt(s) remaining.
Enter master password (attempt 2/3): 0987eg
Wrong password. 1 attempt(s) remaining.
Enter master password (attempt 3/3): ok97722

Too many failed attempts. Vault locked for 30 seconds.

Press Enter to continue...
```

Figure 8 — Password Vault — brute-force lockout after three failed attempts



```
C:\Users\Hp\Documents\final_porject>Password_Security_Suite_Project.

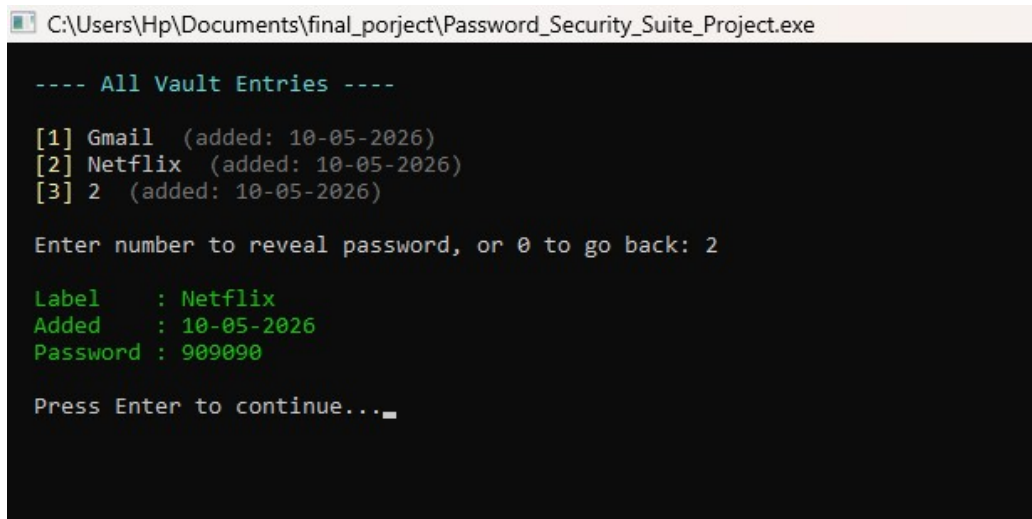
---- Search Entry ----

Enter label to search: Gmail

Found!
Label      : Gmail
Added      : 10-05-2026
Password   : arbish98??

Press Enter to continue..._
```

Figure 9 — Password Vault — search entry by label



```
C:\Users\Hp\Documents\final_porject\Password_Suite_Project.exe

---- All Vault Entries ----

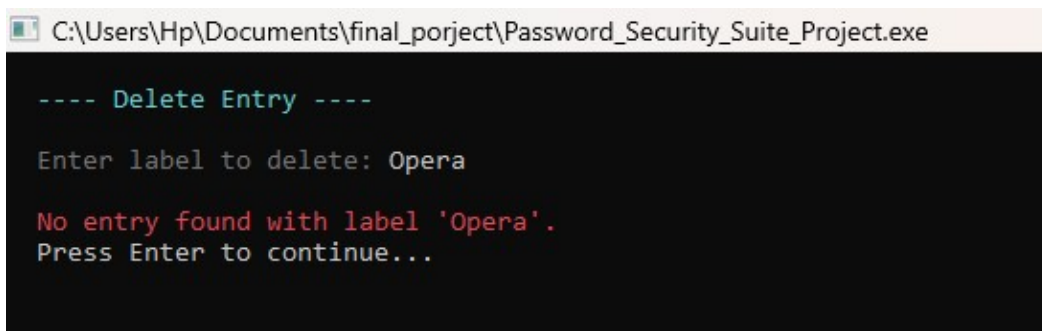
[1] Gmail (added: 10-05-2026)
[2] Netflix (added: 10-05-2026)
[3] 2 (added: 10-05-2026)

Enter number to reveal password, or 0 to go back: 2

Label      : Netflix
Added      : 10-05-2026
Password   : 909090

Press Enter to continue..._
```

Figure 10 — Password Vault — view all entries and reveal password



```
C:\Users\Hp\Documents\final_porject\Password_Suite_Project.exe

---- Delete Entry ----

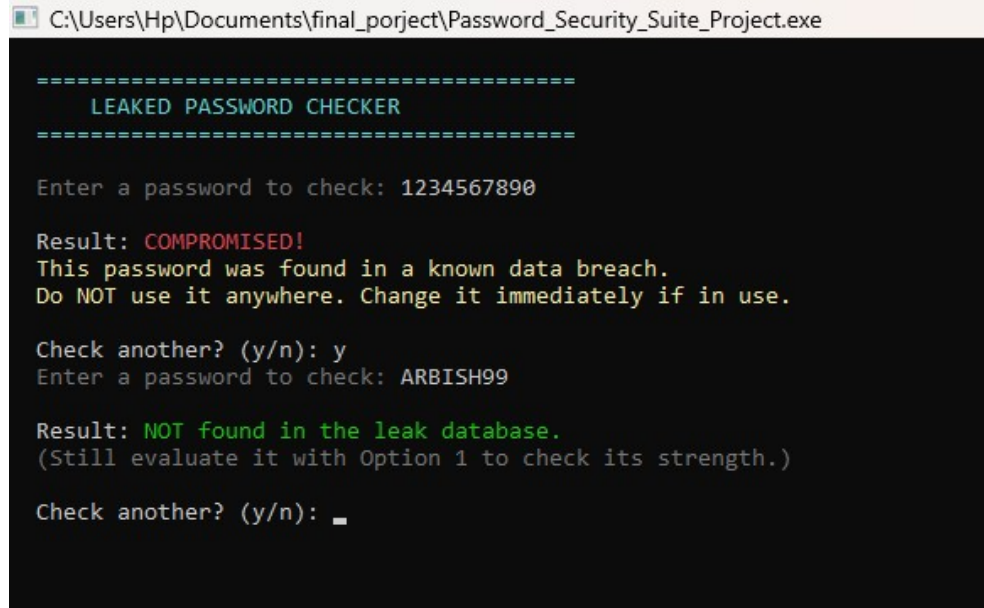
Enter label to delete: Opera

No entry found with label 'Opera'.
Press Enter to continue...
```

Figure 11 — Password Vault — delete entry

#### 4.4 Feature 4 — Leaked Password Checker

LeakChecker is constructed with a filename. It reads the file line by line, normalises each entry to lowercase, and stores the list in a private vector<string>. isLeaked() converts the supplied password to lowercase and performs a linear search. The feature works entirely offline with a local breach dataset of 100+ known compromised passwords.



```
C:\Users\Hp\Documents\final_porject\Password_Security_Suite_Project.exe

=====
LEAKED PASSWORD CHECKER
=====

Enter a password to check: 1234567890

Result: COMPROMISED!
This password was found in a known data breach.
Do NOT use it anywhere. Change it immediately if in use.

Check another? (y/n): y
Enter a password to check: ARBISH99

Result: NOT found in the leak database.
(Still evaluate it with Option 1 to check its strength.)

Check another? (y/n): _
```

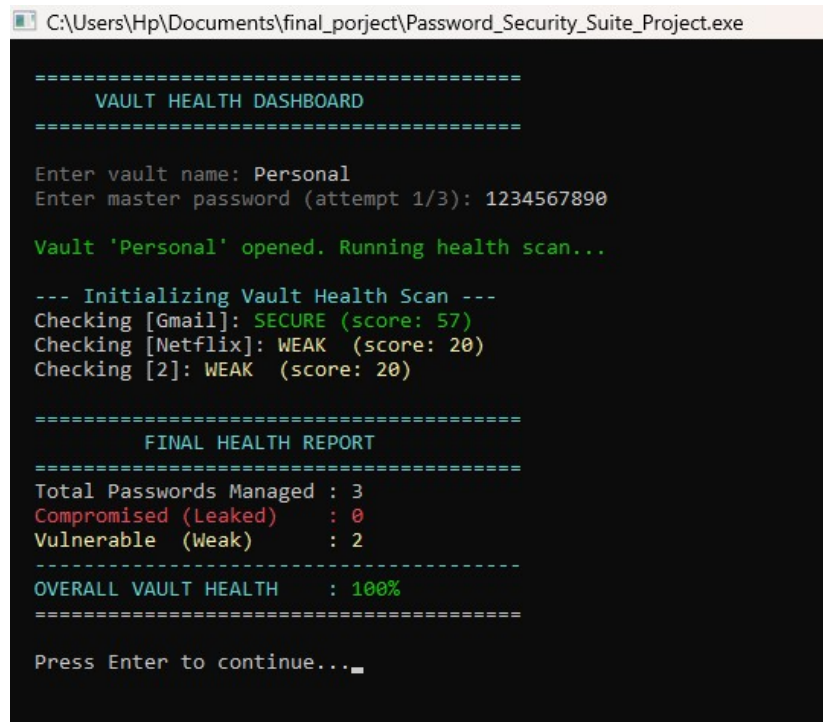
Figure 12 — Leaked Password Checker — compromised and safe results

## 4.5 Feature 5 — Health Dashboard

HealthDashboard::analyzeVault() takes references to a PasswordVault, LeakChecker, and PasswordEvaluator. It loops over every vault entry, decrypts the stored password, calls isLeaked() and evaluate(), and maintains two independent counters compromisedCount for confirmed leaked passwords and weakCount for low-scoring ones. The overall health percentage reported by displaySummary() is calculated from compromised entries only; weak passwords are surfaced as a separate warning, giving users a clear distinction between accounts requiring immediate action and those advisable to strengthen.

### Integration Point

HealthDashboard demonstrates OOP composition in action it talks to PasswordVault, LeakChecker, and PasswordEvaluator purely through their public interfaces, with no knowledge of internal implementation details. This is the Facade pattern applied directly.



```
=====
      VAULT HEALTH DASHBOARD
=====

Enter vault name: Personal
Enter master password (attempt 1/3): 1234567890

Vault 'Personal' opened. Running health scan...

--- Initializing Vault Health Scan ---
Checking [Gmail]: SECURE (score: 57)
Checking [Netflix]: WEAK (score: 20)
Checking [2]: WEAK (score: 20)

=====
      FINAL HEALTH REPORT
=====
Total Passwords Managed : 3
Compromised (Leaked)    : 0
Vulnerable (Weak)      : 2
-----
OVERALL VAULT HEALTH    : 100%
=====

Press Enter to continue..._
```

Figure 13 — Password Health Dashboard — final vault health report

## 5. Design Pattern Application

### 5.1 Template Method Pattern

Defines an algorithm's skeleton in a base class and defers specific steps to subclasses. PasswordGenerator declares generate() as pure virtual the skeleton. The concrete steps (how the character pool is built vs. how words are selected) are left entirely to RandomGenerator and PhraseGenerator. Callers invoke generate() on a PasswordGenerator reference without caring which subclass executes.

### 5.2 Strategy Pattern

Defines a family of interchangeable algorithms and selects one at runtime. RandomGenerator and PhraseGenerator are two interchangeable strategies. The user selects Phrase vs. Random at runtime; from that point, the rest of the program uses the common PasswordGenerator interface. Adding a new strategy (e.g. PINGenerator) requires zero changes to calling code.

### 5.3 Facade Pattern

Provides a simple interface over a complex subsystem. HealthDashboard hides the complexity of iterating entries, decrypting passwords, running evaluations, and checking leak databases behind just two calls analyzeVault() and displaySummary(). Users of HealthDashboard need no knowledge of the three subsystems it orchestrates.

### 5.4 Guard / Sentinel Pattern

All access to the vault is mediated through a single AuthGuard object that maintains fail count, lock status, and lock timer in one place. This prevents authentication logic from being scattered across the codebase and ensures the lockout cannot be bypassed by calling a different code path.

| Pattern                 | Classes Involved                                       | Benefit                                                                               | Type         |
|-------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|--------------|
| <b>Template Method</b>  | PasswordGenerator, RandomGenerator, PhraseGenerator    | Enforces consistent algorithm skeleton; defers implementation details to subclasses   | Formal (GoF) |
| <b>Strategy</b>         | RandomGenerator, PhraseGenerator via PasswordGenerator | Interchangeable algorithms selectable at runtime without changing calling code        | Formal (GoF) |
| <b>Facade</b>           | HealthDashboard over Vault, Evaluator, LeakChecker     | Simplifies multi-subsystem orchestration behind a two-method interface                | Formal (GoF) |
| <b>Guard / Sentinel</b> | AuthGuard inside PasswordVault                         | Centralises and enforces access-control policy; prevents bypass via alternative paths | Informal     |

Table 7 — Design pattern summary — pattern, classes involved, benefit, and type

## 6. Limitations & Future Work

### 6.1 Current Limitations

| Limitation                         | Detail & Impact                                                                                                                                          |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>XOR Encryption Weakness</b>     | Simple XOR with a fixed key is not cryptographically secure. A production system would use AES-256 or bcrypt.                                            |
| <b>Plain-text Leaked Database</b>  | The leaked-password file is plaintext easily inspected or modified. A production system would use a Bloom filter or hashed dataset.                      |
| <b>No Salt on Master Password</b>  | convertToHash() produces a deterministic checksum. Without a random salt, identical passwords produce identical hashes, enabling precomputation attacks. |
| <b>Linear Leak Search</b>          | isLeaked() performs a linear scan. For millions of entries this becomes too slow std::unordered_set would give O(1) average-case lookup.                 |
| <b>Console-Only Interface</b>      | Non-technical users must navigate text menus. No GUI or web interface is provided.                                                                       |
| <b>Single-User, Single-Machine</b> | All data is in local files. No sync, no multi-device access, and no backup mechanism.                                                                    |
| <b>No Password Age Tracking</b>    | The vault stores a date-added field but does not currently alert users when a password is stale (e.g. older than 90 days).                               |
| <b>Static Word List</b>            | PhraseGenerator uses a hardcoded 56-word list. A larger external dictionary would produce significantly more entropy.                                    |

Table 8 — Current limitations — constraint, detail, and impact

### 6.2 Proposed Enhancements

- Replace XOR encryption with AES-256 via OpenSSL or a header-only C++ library.
- Replace the master password checksum with bcrypt or PBKDF2 with a random per-vault salt.
- Replace the linear leak search with std::unordered\_set for O(1) average-case lookup.
- Add password expiry alerts flag vault entries in the Health Dashboard when date added exceeds a configurable threshold (e.g. 90 days).
- Implement a GUI using Qt, or a web-based interface, to improve usability for non-technical users.
- Add cloud-sync with end-to-end encryption so the vault is accessible across devices.
- Load the phrase word list from a configurable external file instead of a hardcoded array.
- Add categories and tags on PasswordEntry objects (Work, Banking, Social, etc.) with filter support in the vault menu.
- Add import/export for standard password-manager formats such as CSV and KeePass XML.

## 7. Team Contribution Table

| Member Name  | Roll Number | Section / Theme                        | Specific Contributions                                                                                                                                                                                                                                                                                                |
|--------------|-------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Areefa Samar | CT-25062    | Partner A — Input & Analysis Engine    | <b>Feature 1: PasswordEvaluator</b><br>All six criteria, score calculation, crack time estimate, displayReport()<br><b>Feature 2: Password Generator</b><br>Abstract base class PasswordGenerator, RandomGenerator, PhraseGenerator, runPasswordGenerator()                                                           |
| Arbish       | CT-25053    | Partner B — Storage & Reporting Engine | <b>Feature 3: Password Vault</b><br>PasswordVault, AuthGuard, MasterPasswordFile, PasswordEntry — encryption, file I/O, logout<br><b>Feature 4: LeakChecker</b><br>Offline breach detection against local dataset<br><b>Feature 5: HealthDashboard</b><br>Multi-class aggregation and full security report generation |

Table 9 — Team contribution table — member names, roll numbers, and contributions

### 7.1 Individual Class Assignment

| Class              | Partner   | OOP Concept                   | Files                      |
|--------------------|-----------|-------------------------------|----------------------------|
| PasswordGenerator  | Partner A | Abstraction, Inheritance      | PasswordGenerator.h / .cpp |
| RandomGenerator    | Partner A | Inheritance, Polymorphism     | PasswordGenerator.h / .cpp |
| PhraseGenerator    | Partner A | Inheritance, Polymorphism     | PasswordGenerator.h / .cpp |
| PasswordEvaluator  | Partner A | Encapsulation, Abstraction    | PasswordEvaluator.h / .cpp |
| PasswordEntry      | Partner B | Encapsulation, Validation     | PasswordEntry.h / .cpp     |
| AuthGuard          | Partner B | Encapsulation, Security Logic | AuthGuard.h / .cpp         |
| MasterPasswordFile | Partner B | File I/O, Static Methods      | PasswordVault.h / .cpp     |
| PasswordVault      | Partner B | Composition, File I/O,        | PasswordVault.h / .cpp     |

|                        |           |                                |                          |
|------------------------|-----------|--------------------------------|--------------------------|
|                        |           | Arrays                         |                          |
| <b>LeakChecker</b>     | Partner B | Encapsulation, File I/O        | LeakChecker.h / .cpp     |
| <b>HealthDashboard</b> | Partner B | Aggregation, Facade<br>Pattern | HealthDashboard.h / .cpp |

*Table 10 — Individual class assignment — class, partner, OOP concept, and files*

## 8. Conclusion

Password Security Suite demonstrates that Object Oriented Programming is not simply a theoretical framework it is a practical engineering discipline that produces cleaner, safer, and more maintainable software when applied thoughtfully to real problems.

All four OOP pillars appear in this project not because the rubric requires them, but because the problem demands them. Encapsulation protects security-critical data from external tampering. Inheritance eliminates code duplication across two generator strategies. Polymorphism allows the main menu to call generate() without knowing or caring which mode the user selected. Abstraction prevents callers from depending on internal scoring or encryption logic that may change in future iterations.

Beyond the OOP principles, the project demonstrates three formal design patterns Template Method, Strategy, and Façade each justified by the structure of the problem rather than chosen arbitrarily. The AuthGuard class implements an informal Guard pattern that centralises all authentication logic and prevents it from being scattered across the codebase.

The five features password evaluation, generation, secure vault storage, breach detection, and health reporting form a coherent, end-to-end security tool. Each feature is independently testable, independently maintainable, and connected to the others only through clean public interfaces. This is precisely the kind of modular, extensible architecture that OOP is designed to produce.

The limitations acknowledged in Section 6 XOR encryption, linear search, no GUI are honest reflections of the project's scope as a second-semester console application. The proposed enhancements chart a clear path from this educational prototype toward a production-grade password manager, demonstrating awareness of real-world software engineering constraints beyond what a first implementation can address.

### Summary

Password Security Suite is a fully offline, console-based C++ application that solves a genuine real-world problem using all four OOP pillars, three formal design patterns, robust input and exception handling, and a modular class architecture that could realistically be extended into a production tool.

## 9. References

- [1] Eckel, B. (2000). *Thinking in C++, Volume 1: Introduction to standard C++* (2nd ed.). Prentice Hall.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- [3] Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ primer* (5th ed.). Addison-Wesley.
- [4] National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63b>
- [5] Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.
- [6] Verizon. (2024). *2024 data breach investigations report*. Verizon Communications. <https://www.verizon.com/business/resources/reports/dbir/>
- [7] OWASP Foundation. (2023). Password storage cheat sheet. Open Web Application Security Project. [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- [8] Hunt, T. (2019). Pwned passwords. HaveIBeenPwned. <https://haveibeenpwned.com/Passwords>

## Appendix A — Source Code

All source code is presented below with comments removed for clarity. Files are grouped by header (.h) followed by implementation (.cpp).

### A.1 Compatibility & Utilities

#### WinCompat.h

```
#ifndef WINCOMPAT_H
#define WINCOMPAT_H

#ifdef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#ifdef NOMINMAX
#define NOMINMAX
#endif
#ifdef NOGDI
#define NOGDI
#endif

#include <windows.h>
#undef byte

#endif
```

*Listing 1 — WinCompat.h*

### A.2 AuthGuard

#### AuthGuard.h

```
#ifndef AuthGuard_h
#define AuthGuard_h
#include <string>
#include <ctime>

class AuthGuard {
private:
    std::string masterHash;
    int failCount;
    bool isLocked;
    time_t lockTime;

    std::string convertToHash(std::string masterpassword);
    void startLockTimer();

    std::string encryptHash(std::string hash) const;
public:
    AuthGuard();
    AuthGuard(std::string masterpassword);

    bool saveHashToFile(const std::string& keyFilePath) const;

    bool loadHashFromFile(const std::string& keyFilePath);

    bool verifymasterPassword(std::string masterpassword);
    bool getLockstatus() const;
    void resetfailCount();
    int getRemainingTime();
};
```

```
};
#endif
```

Listing 2 — AuthGuard.h

## AuthGuard.cpp

```
#include "AuthGuard.h"
#include <string>
#include <fstream>
#include <iostream>
using namespace std;

string AuthGuard::convertToHash(string masterpassword) {
    if (masterpassword.size() < 8 || masterpassword.size() > 20)
        return "N/A";

    int sum = 0;
    for (int i = 0; i < (int)masterpassword.size(); i++)
        sum += masterpassword[i] * (i + 1);

    return to_string(sum);
}

string AuthGuard::encryptHash(string hash) const {
    const char KEY1 = 0x5A;
    for (int i = 0; i < (int)hash.size(); i++)
        hash[i] ^= KEY1;

    for (int i = 0; i < (int)hash.size(); i++)
        hash[i] ^= (char)(i + 1);

    int l = 0, r = (int)hash.size() - 1;
    while (l < r) { char tmp = hash[l]; hash[l] = hash[r]; hash[r] = tmp; l++; r--; }

    return hash;
}

bool AuthGuard::saveHashToFile(const string& keyFilePath) const {
    if (masterHash == "N/A") return false;

    ofstream file(keyFilePath, ios::binary);
    if (!file.is_open()) return false;

    string encrypted = encryptHash(masterHash);
    file.write(encrypted.c_str(), (int)encrypted.size());
    file.close();
    return true;
}

bool AuthGuard::loadHashFromFile(const string& keyFilePath) {
    ifstream file(keyFilePath, ios::binary);
    if (!file.is_open()) return false;

    string encrypted((istreambuf_iterator<char>(file)),
                    istreambuf_iterator<char>());
    file.close();

    if (encrypted.empty()) return false;

    masterHash = encryptHash(encrypted);
    return true;
}

AuthGuard::AuthGuard()
```

```

        : masterHash("N/A"), failCount(0), isLocked(false), lockTime(0) {}

AuthGuard::AuthGuard(string masterpassword)
    : masterHash(convertToHash(masterpassword)),
      failCount(0), isLocked(false), lockTime(0) {}

bool AuthGuard::verifymasterPassword(string masterPassword) {
    if (getLockstatus()) return false;

    if (convertToHash(masterPassword) == masterHash) {
        resetfailCount();
        return true;
    }

    failCount++;
    if (failCount == 3) {
        isLocked = true;
        startLockTimer();
    }
    return false;
}

bool AuthGuard::getLockstatus() const { return isLocked; }
void AuthGuard::resetfailCount()      { failCount = 0; }

void AuthGuard::startLockTimer() {
    lockTime = time(0);
}

int AuthGuard::getRemainingTime() {
    if (!isLocked) return 0;
    double elapsed = difftime(time(0), lockTime);
    int remaining = 30 - (int)elapsed;
    if (remaining <= 0) {
        isLocked = false;
        failCount = 0;
        return 0;
    }
    return remaining;
}

```

Listing 3 — AuthGuard.cpp

## A.3 PasswordEntry

### PasswordEntry.h

```

#ifndef PasswordEntry_h
#define PasswordEntry_h
#include<string>

class PasswordEntry{
private:
    std::string label;
    std::string dateadded;
    std::string encryptedpassword;

public:
    bool isValidFormat(const std::string& dateadded) const;

    PasswordEntry();

    PasswordEntry(std::string label , std::string datadded , std::string
encryptedpassword);

```

```

        std::string getLabel() const;
        std::string getDateAdded() const;
        std::string getEncryptedPassword() const;

        void display() const;
};
#endif

```

Listing 4 — PasswordEntry.h

## PasswordEntry.cpp

```

#include "PasswordEntry.h"
#include<string>
#include <stdexcept>
#include <cctype>
#include<iostream>
using namespace std;

bool PasswordEntry::isValidFormat(const string& date) const {
    if(date.empty()){
        return false;
    }
    if (date.size() != 10) {
        return false; }
    if (date[2] != '-' || date[5] != '-') {
        return false; }
    int digitPositions[] = {0, 1, 3, 4, 6, 7, 8, 9};
    for (int i : digitPositions) {
        if (!isdigit(date[i])) {
            return false;
        }
    }
    return true;
}

PasswordEntry::PasswordEntry() : label("N/A") , dateadded("N/A") ,
    encryptedpassword("N/A"){

PasswordEntry::PasswordEntry(string label, string dateadded, string encryptedpassword)
: label(!label.empty()? label : "N/A") ,
    dateadded("N/A") ,
    encryptedpassword(!encryptedpassword.empty() ? encryptedpassword : "N/A") {

    if(isValidFormat(dateadded)){
        try{
            int day = stoi(dateadded.substr(0,2));
            int month = stoi(dateadded.substr(3,2));
            int year = stoi(dateadded.substr(6,4));

            if((day >= 1 && day <= 31) &&
                (month >= 1 && month <= 12) &&
                (year >= 1 && year <= 2026)){
                this->dateadded = dateadded;
            }
            else {
                this->dateadded = "N/A";
            }
        }
        catch(invalid_argument& e){
            this->dateadded = "N/A";
        }
        catch(out_of_range& e){
            this->dateadded = "N/A";
        }
    }
}

```

```

    }
    else{
        this->dateadded = "N/A";
    }
}

string PasswordEntry::getLabel() const{
    return label;
}

string PasswordEntry::getDateAdded() const{
    return dateadded;
}

string PasswordEntry::getEncryptedPassword() const{
    return encryptedpassword;
}

void PasswordEntry::display() const{
    cout<<"Label : " << getLabel() <<endl;
    cout<<"DateAdded : " << getDateAdded() <<endl;
    cout<<"EncryptedPassword : " << getEncryptedPassword() <<endl;
}

```

Listing 5 — PasswordEntry.cpp

## A.4 PasswordEvaluator

### PasswordEvaluator.h

```

#ifndef PASSWORDEVALUATOR_H
#define PASSWORDEVALUATOR_H

#include <string>
using namespace std;

enum StrengthLevel {
    VERY_WEAK,
    WEAK,
    MODERATE,
    STRONG,
    VERY_STRONG
};

class PasswordEvaluator {
private:
    string password;
    int score;
    StrengthLevel level;

    bool checkLength() const;
    bool checkUppercase() const;
    bool checkDigits() const;
    bool checkSymbols() const;
    bool checkCommonPatterns() const;
    bool checkLeakedList() const;

    void calculateScore();
    string getCrackTime() const;
public:
    PasswordEvaluator();

    void setPassword(const string& pwd);

```

```

    void evaluate();

    void displayReport() const;

    int          getScore()    const;
    StrengthLevel getLevel()   const;
    string        getPassword() const;
};

#endif

```

Listing 6 — PasswordEvaluator.h

## PasswordEvaluator.cpp

```

#include "PasswordEvaluator.h"
#include "WinCompat.h"
#include <iostream>
#include <fstream>
#include <stdexcept>
#include <cctype>
using namespace std;

static void evalSetColor(int code) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), code);
}
static void evalResetColor() { evalSetColor(7); }

PasswordEvaluator::PasswordEvaluator()
    : password(""), score(0), level(VERY_WEAK) {}

void PasswordEvaluator::setPassword(const string& pwd) {
    if (pwd.empty()) {
        throw invalid_argument("Password cannot be empty.");
    }
    if (pwd.length() > 128) {
        throw invalid_argument("Password is too long (max 128 characters).");
    }
    for (char c : pwd) {
        if (c == ' ') {
            throw invalid_argument("Password should not contain spaces.");
        }
    }
    password = pwd;
    score    = 0;
    level    = VERY_WEAK;
}

bool PasswordEvaluator::checkLength() const {
    return password.length() >= 8;
}

bool PasswordEvaluator::checkUppercase() const {
    for (char c : password) {
        if (isupper(c)) return true;
    }
    return false;
}

bool PasswordEvaluator::checkDigits() const {
    for (char c : password) {
        if (isdigit(c)) return true;
    }
    return false;
}

bool PasswordEvaluator::checkSymbols() const {

```

```

    string symbols = "@#$!%&*-_+=?";
    for (char c : password) {
        if (symbols.find(c) != string::npos) return true;
    }
    return false;
}

bool PasswordEvaluator::checkCommonPatterns() const {
    string patterns[] = {
        "123", "1234", "12345", "123456",
        "abc", "abcd", "abcde",
        "qwerty", "qwert", "asdf",
        "password", "pass", "1111", "0000",
        "aaa", "aaaa", "admin", "login",
        "letmein", "welcome", "monkey", "dragon"
    };
    int patternCount = 22;

    string lower = "";
    for (char c : password) {
        lower += tolower(c);
    }

    for (int i = 0; i < patternCount; i++) {
        if (lower.find(patterns[i]) != string::npos) {
            return true;
        }
    }
    return false;
}

bool PasswordEvaluator::checkLeakedList() const {
    ifstream file("leaked_passwords.txt");
    if (!file.is_open()) {
        return false;
    }

    string line;
    string lower = "";
    for (char c : password) lower += tolower(c);

    while (getline(file, line)) {
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }
        string lineLower = "";
        for (char c : line) lineLower += tolower(c);

        if (lineLower == lower) {
            file.close();
            return true;
        }
    }

    file.close();
    return false;
}

void PasswordEvaluator::calculateScore() {
    score = 0;

    int len = password.length();
    if (len >= 20) score += 30;
    else if (len >= 16) score += 25;
    else if (len >= 12) score += 20;
    else if (len >= 8) score += 12;
    else score += 5;

    if (checkUppercase()) score += 15;
    if (checkDigits()) score += 15;
}

```

```

    if (checkSymbols())    score += 20;

    bool hasLower = false;
    for (char c : password) {
        if (islower(c)) { hasLower = true; break; }
    }
    if (hasLower) score += 10;

    if (checkCommonPatterns()) score -= 20;
    if (checkLeakedList())     score = 0;

    if (score < 0)    score = 0;
    if (score > 100) score = 100;

    if      (score >= 80) level = VERY_STRONG;
    else if (score >= 60) level = STRONG;
    else if (score >= 40) level = MODERATE;
    else if (score >= 20) level = WEAK;
    else          level = VERY_WEAK;
}

string PasswordEvaluator::getCrackTime() const {
    if      (score >= 80) return "Centuries (practically uncrackable)";
    else if (score >= 60) return "Several years";
    else if (score >= 40) return "A few hours to days";
    else if (score >= 20) return "A few minutes";
    else          return "Less than 1 second";
}

void PasswordEvaluator::evaluate() {
    if (password.empty()) {
        throw runtime_error("No password set. Call setPassword() first.");
    }
    calculateScore();
}

void PasswordEvaluator::displayReport() const {

    cout << "\n";
    evalSetColor(11);
    cout << " =====\n";
    cout << "          PASSWORD STRENGTH REPORT\n";
    cout << " =====\n";
    evalResetColor();

    evalSetColor(8); cout << " Password : "; evalResetColor();
    cout << password << "\n";

    evalSetColor(8); cout << " Score      : "; evalResetColor();

    if      (score >= 80) evalSetColor(10);
    else if (score >= 60) evalSetColor(14);
    else if (score >= 40) evalSetColor(14);
    else          evalSetColor(12);
    cout << score << " / 100";
    evalResetColor();
    cout << "\n";

    evalSetColor(8); cout << " Strength : "; evalResetColor();
    cout << "[";
    int filled = score / 5;
    for (int i = 0; i < 20; i++) {
        if (i < filled) {
            if      (score >= 80) evalSetColor(10);
            else if (score >= 40) evalSetColor(14);
            else          evalSetColor(12);
            cout << "#";
            evalResetColor();
        } else {
            evalSetColor(8); cout << "-"; evalResetColor();

```

```

    }
}
cout << "]" ";

if (level == VERY_STRONG) { evalSetColor(10); cout << "VERY STRONG\n"; }
else if (level == STRONG) { evalSetColor(10); cout << "STRONG\n"; }
else if (level == MODERATE) { evalSetColor(14); cout << "MODERATE\n"; }
else if (level == WEAK) { evalSetColor(12); cout << "WEAK\n"; }
else { evalSetColor(12); cout << "VERY WEAK\n"; }
evalResetColor();

evalSetColor(8); cout << " Crack Time: "; evalResetColor();
if (score >= 80) evalSetColor(10);
else if (score >= 40) evalSetColor(14);
else evalSetColor(12);
cout << getCrackTime() << "\n";
evalResetColor();

evalSetColor(11);
cout << "\n ---- Criteria Breakdown ----\n";
evalResetColor();

auto printCriteria = [](bool passed, const string& msg) {
    if (passed) { evalSetColor(10); cout << " [PASS] "; }
    else { evalSetColor(12); cout << " [FAIL] "; }
    evalResetColor();
    cout << msg << "\n";
};

printCriteria(checkLength(), "Length is 8 or more characters");
printCriteria(checkUppercase(), "Contains uppercase letters (A-Z)");
printCriteria(checkDigits(), "Contains numbers (0-9)");
printCriteria(checkSymbols(), "Contains symbols (@#$!...)");
printCriteria(!checkCommonPatterns(), "No common weak patterns (123, abc, qwerty)");
printCriteria(!checkLeakedList(), "Not found in known leaked passwords list");

evalSetColor(11);
cout << "\n ---- Suggestions ----\n";
evalResetColor();

bool allGood = true;

if (!checkLength()) {
    evalSetColor(14); cout << " -> "; evalResetColor();
    cout << "Make your password at least 8 characters long.\n";
    allGood = false;
}
if (!checkUppercase()) {
    evalSetColor(14); cout << " -> "; evalResetColor();
    cout << "Add at least one uppercase letter (e.g. A, B, C).\n";
    allGood = false;
}
if (!checkDigits()) {
    evalSetColor(14); cout << " -> "; evalResetColor();
    cout << "Add at least one number (e.g. 3, 7, 9).\n";
    allGood = false;
}
if (!checkSymbols()) {
    evalSetColor(14); cout << " -> "; evalResetColor();
    cout << "Add a symbol like @, #, $, or ! to boost strength.\n";
    allGood = false;
}
if (checkCommonPatterns()) {
    evalSetColor(12); cout << " -> "; evalResetColor();
    cout << "Remove common patterns like '123', 'abc', or 'qwerty'.\n";
    allGood = false;
}
if (checkLeakedList()) {
    evalSetColor(12); cout << " -> "; evalResetColor();
    cout << "This password was found in a known data breach list!\n";
}

```

```

        evalSetColor(12); cout << "  -> "; evalResetColor();
        cout << "You must NOT use this password anywhere.\n";
        allGood = false;
    }
    if (password.length() < 12 && level != VERY_STRONG) {
        evalSetColor(14); cout << "  -> "; evalResetColor();
        cout << "Consider making it 12+ characters for better protection.\n";
        allGood = false;
    }

    if (allGood) {
        evalSetColor(10);
        cout << " Great job! Your password meets all the criteria.\n";
        evalResetColor();
    }

    evalSetColor(11);
    cout << " =====\n\n";
    evalResetColor();
}

int      PasswordEvaluator::getScore()    const { return score;    }
StrengthLevel PasswordEvaluator::getLevel() const { return level;  }
string   PasswordEvaluator::getPassword() const { return password; }

```

Listing 7 — PasswordEvaluator.cpp

## A.5 PasswordGenerator

### PasswordGenerator.h

```

#ifndef PASSWORDGENERATOR_H
#define PASSWORDGENERATOR_H

#include <string>
using namespace std;

class PasswordVault;

class PasswordGenerator {
protected:
    int length;
    bool useUppercase;
    bool useDigits;
    bool useSymbols;
    bool avoidLookAlikes;

public:
    PasswordGenerator(int len = 12,
                      bool upper = true,
                      bool digits = true,
                      bool symbols = true,
                      bool avoidConfusing = false);

    virtual string generate() = 0;

    virtual ~PasswordGenerator() {}

    void setLength(int len);
    void setUseUppercase(bool val);
    void setUseDigits(bool val);
    void setUseSymbols(bool val);
    void setAvoidLookAlikes(bool val);

    int getLength() const;

```

```

};

class RandomGenerator : public PasswordGenerator {
public:
    RandomGenerator(int len = 12,
                    bool upper = true,
                    bool digits = true,
                    bool symbols = true,
                    bool avoidConfusing = false);

    string generate() override;

private:
    string buildCharPool();
};

class PhraseGenerator : public PasswordGenerator {
private:
    int wordCount;
    char separator;

public:
    PhraseGenerator(int words = 3, char sep = '$');

    void setWordCount(int words);
    void setSeparator(char sep);

    string generate() override;
};

bool askYesNo(const string& question);
int askInt(const string& question, int minVal, int maxVal);

#endif

```

Listing 8 — PasswordGenerator.h

## PasswordGenerator.cpp

```

#include "PasswordGenerator.h"
#include "PasswordVault.h"
#include "WinCompat.h"
#include <iostream>
#include <stdexcept>
#include <cstdlib>
#include <ctime>
using namespace std;

static void genSetColor(int code) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), code);
}

static void genResetColor() { genSetColor(7); }

PasswordGenerator::PasswordGenerator(int len, bool upper, bool digits,
                                     bool symbols, bool avoidConfusing)
    : length(len), useUppercase(upper), useDigits(digits),
      useSymbols(symbols), avoidLookAlikes(avoidConfusing) {}

void PasswordGenerator::setLength(int len) {
    if (len < 4 || len > 64) {
        throw invalid_argument("Length must be between 4 and 64.");
    }
    length = len;
}

void PasswordGenerator::setUseUppercase(bool val)    { useUppercase    = val; }
void PasswordGenerator::setUseDigits(bool val)      { useDigits      = val; }

```

```

void PasswordGenerator::setUseSymbols(bool val)      { useSymbols      = val; }
void PasswordGenerator::setAvoidLookAlikes(bool val) { avoidLookAlikes = val; }
int PasswordGenerator::getLength() const            { return length; }

RandomGenerator::RandomGenerator(int len, bool upper, bool digits,
                                   bool symbols, bool avoidConfusing)
    : PasswordGenerator(len, upper, digits, symbols, avoidConfusing) {}

string RandomGenerator::buildCharPool() {
    string pool = "abcdefghijklmnopqrstuvwxyz";

    if (avoidLookAlikes) {
        string cleaned = "";
        for (char c : pool) {
            if (c != 'o') cleaned += c;
        }
        pool = cleaned;
    }

    if (useUppercase) {
        if (avoidLookAlikes) {
            pool += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        } else {
            pool += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        }
    }

    if (useDigits) {
        pool += avoidLookAlikes ? "23456789" : "0123456789";
    }

    if (useSymbols) {
        pool += "@#$!%&*";
    }

    return pool;
}

string RandomGenerator::generate() {
    string pool = buildCharPool();

    if (pool.empty()) {
        throw runtime_error("Character pool is empty. Enable at least one character type.");
    }

    static bool seeded = false;
    if (!seeded) { srand((unsigned int)time(0)); seeded = true; }

    string password = "";
    for (int i = 0; i < length; i++) {
        int index = rand() % pool.size();
        password += pool[index];
    }

    return password;
}

PhraseGenerator::PhraseGenerator(int words, char sep)
    : PasswordGenerator(12, false, false, false, false),
      wordCount(words), separator(sep) {}

void PhraseGenerator::setWordCount(int words) {
    if (words < 2 || words > 4) {
        throw invalid_argument("Word count must be 2, 3, or 4.");
    }
    wordCount = words;
}

void PhraseGenerator::setSeparator(char sep) {
    separator = sep;
}

```

```

}

string PhraseGenerator::generate() {
    static bool seeded = false;
    if (!seeded) { srand((unsigned int)time(0)); seeded = true; }

    string words[] = {
        "Blue", "Tiger", "Storm", "Cloud", "River", "Eagle", "Stone", "Flame",
        "Swift", "Night", "Brave", "Solar", "Frost", "Cedar", "Maple", "Ocean",
        "Pixel", "Lunar", "Amber", "Steel", "Crisp", "Vivid", "Blaze", "Delta",
        "Flint", "Shadow", "Glacier", "Summit", "Zenith", "Orbit", "Pulse", "Canyon",
        "Prism", "Echo", "Falcon", "Gully", "Harbor", "Jungle", "Kite", "Legend",
        "Mist", "Nomad", "Onyx", "Panda", "Quartz", "Radar", "Siren", "Tulip",
        "Under", "Vortex", "Willow", "Xenon", "Yacht", "Zebra", "Alpha", "Beta"
    };
    int wordListSize = 56;

    string passphrase = "";

    for (int i = 0; i < wordCount; i++) {
        int index = rand() % wordListSize;
        passphrase += words[index];

        if (i < wordCount - 1) {
            passphrase += separator;
        }
    }

    int num = 10 + rand() % 90;
    passphrase += to_string(num);

    return passphrase;
}

bool askYesNo(const string& question) {
    string input;
    while (true) {
        cout << question << " (y/n): ";
        cin >> input;
        cin.ignore(1000, '\n');

        if (input == "y" || input == "Y" || input == "yes" || input == "YES") return true;
        if (input == "n" || input == "N" || input == "no" || input == "NO") return false;

        cout << " Invalid input. Please enter y or n.\n";
    }
}

int askInt(const string& question, int minVal, int maxVal) {
    string input;
    while (true) {
        cout << question;
        cin >> input;
        cin.ignore(1000, '\n');

        bool valid = !input.empty();
        for (char c : input) {
            if (!isdigit(c)) { valid = false; break; }
        }

        if (valid) {
            int value = stoi(input);
            if (value >= minVal && value <= maxVal) return value;
        }

        cout << " Invalid input. Please enter a number between "
              << minVal << " and " << maxVal << ".\n";
    }
}

```

```

string runPasswordGenerator() {
    system("cls");
    genSetColor(11);
    cout << "\n =====\n";
    cout << "          PASSWORD GENERATOR\n";
    cout << " =====\n\n";
    genResetColor();

    genSetColor(8);
    cout << " Do you need to remember this password, or will it be saved in the vault?\n";
    genResetColor();
    genSetColor(14); cout << " [1]"; genResetColor(); cout << " I need to remember it
    (Phrase mode)\n";
    genSetColor(14); cout << " [2]"; genResetColor(); cout << " I will save it
    (Random mode)\n\n";

    int modeChoice = askInt(" Enter choice: ", 1, 2);

    string generatedPassword = "";

    if (modeChoice == 1) {
        PhraseGenerator pg;

        int words = askInt("\n How many words? (2 / 3 / 4): ", 2, 4);
        pg.setWordCount(words);

        genSetColor(8);
        cout << "\n Separator between words?\n";
        genResetColor();
        genSetColor(14); cout << " [1]"; genResetColor(); cout << " None --
        BlueStorm49\n";
        genSetColor(14); cout << " [2]"; genResetColor(); cout << " Dash -- Blue-Storm-
        49\n";
        genSetColor(14); cout << " [3]"; genResetColor(); cout << " Symbol --
        Blue$Storm$49\n";
        int sepChoice = askInt(" Enter choice: ", 1, 3);

        char sep = '\\0';
        if (sepChoice == 2) sep = '-';
        if (sepChoice == 3) sep = '$';
        pg.setSeparator(sep);

        try {
            generatedPassword = pg.generate();
        } catch (exception& e) {
            genSetColor(12);
            cout << "\n Error generating password: " << e.what() << "\n";
            genResetColor();
            return "";
        }
    }

    else {
        genSetColor(8);
        cout << "\n How strong does it need to be?\n";
        genResetColor();
        genSetColor(14); cout << " [1]"; genResetColor(); cout << " Basic (good for
        low-risk accounts)\n";
        genSetColor(14); cout << " [2]"; genResetColor(); cout << " Strong (recommended
        for personal accounts)\n";
        genSetColor(14); cout << " [3]"; genResetColor(); cout << " Maximum (banking,
        work, email)\n";
        int strength = askInt(" Enter choice: ", 1, 3);

        bool upper = true;
        bool digits = true;
        bool symbols = true;
        bool avoid = false;
        int len = 12;
    }
}

```

```

    if (strength == 1) {
        len = askInt("\n Password length (8 to 64): ", 8, 64);
        upper = askYesNo(" Include uppercase letters? (A-Z)");
        digits = askYesNo(" Include numbers? (0-9)");
        symbols = askYesNo(" Include symbols? (@#$!)");
        avoid = askYesNo(" Avoid look-alike characters? (0/0, 1/1)");

    } else if (strength == 2) {
        len = askInt("\n Password length (12 to 64, recommended 16): ", 12, 64);

    } else {
        len = 20;
        genSetColor(14);
        cout << "\n Maximum mode: All character types enabled. Length set to 20.\n";
        genResetColor();
    }

    if (!upper && !digits && !symbols) {
        genSetColor(14);
        cout << "\n Note: Only lowercase letters will be used.\n";
        genResetColor();
    }

    RandomGenerator rg(len, upper, digits, symbols, avoid);

    try {
        generatedPassword = rg.generate();
    } catch (exception& e) {
        genSetColor(12);
        cout << "\n Error generating password: " << e.what() << "\n";
        genResetColor();
        return "";
    }
}

genSetColor(10);
cout << "\n ----- \n";
cout << " Generated Password: " << generatedPassword << "\n";
cout << " ----- \n";
genResetColor();

if (modeChoice == 2) {
    genSetColor(8);
    cout << "\n Would you like to save this password to your vault? (y/n): ";
    genResetColor();
    string saveChoice;
    cin >> saveChoice;
    cin.ignore(1000, '\n');

    if (saveChoice == "y" || saveChoice == "Y") {
        string vaultName;
        genSetColor(8); cout << " Enter vault name (e.g., work, personal): ";
        genResetColor();
        getline(cin, vaultName);

        if (vaultName.empty()) {
            genSetColor(12); cout << " Invalid vault name.\n"; genResetColor();
        } else {
            string masterPass;
            genSetColor(8); cout << " Enter your master password: "; genResetColor();
            cin >> masterPass;
            cin.ignore(1000, '\n');

            PasswordVault vault;
            if (vault.open(vaultName, masterPass)) {
                string label;
                genSetColor(8); cout << "\n Enter a label for this password (e.g.,
                Gmail, Bank): "; genResetColor();
                getline(cin, label);
            }
        }
    }
}

```

```

        if (!label.empty()) {
            vault.saveEntry(label, generatedPassword);
            genSetColor(10);
            cout << "\n [OK] Password saved to vault successfully!\n";
            genResetColor();
        } else {
            genSetColor(12); cout << " Invalid label. Password not saved.\n";
            genResetColor();
        }
    } else {
        if (vault.isVaultLocked()) {
            genSetColor(12);
            cout << "\n Too many failed attempts. Vault locked for 30 seconds.\n";
            genResetColor();
        } else {
            genSetColor(12);
            cout << "\n [ERROR] Wrong master password or vault not found.\n";
            genResetColor();
        }
    }
}

}

}

genSetColor(8);
cout << "\n Tip: You can evaluate this password using Option 1 from the main menu.\n";
genResetColor();
cout << "\n Press Enter to return to menu...";
cin.ignore(1000, '\n');

return generatedPassword;
}

```

Listing 9 — PasswordGenerator.cpp

## A.6 PasswordVault & MasterPasswordFile

### PasswordVault.h

```

#ifndef PASSWORDVAULT_H
#define PASSWORDVAULT_H

#include <vector>
#include <string>
#include <fstream>
#include "PasswordEntry.h"
#include "AuthGuard.h"

class MasterPasswordFile {
public:

    static std::string shift(const std::string& s);

    static bool exists(const std::string& vaultName);

    static bool save(const std::string& vaultName,
                     const std::string& plainPassword);

    static int verify(const std::string& vaultName,
                     const std::string& plainPassword);
};

class PasswordVault {
private:
    std::vector<PasswordEntry> entries;

```

```

    AuthGuard    guard;
    std::string vaultName;
    std::string filePath;
    bool         isOpen;

    void saveToFile();
    void loadFromFile();
    std::string encryptpass(std::string password);
    std::string decryptpass(std::string password);

public:
    PasswordVault();

    bool create(const std::string& name, const std::string& masterpassword);

    bool open(const std::string& name, const std::string& masterpassword);

    bool getIsOpen() const;
    int  getentryCount() const;
    std::vector<PasswordEntry> getAllEntries() const;

    bool isVaultLocked();
    int  getLockRemainingTime();

    void          saveEntry(std::string label, std::string password);
    bool          deleteEntry(std::string label);
    const PasswordEntry* searchEntry(std::string label);

    std::string decrypt(std::string encryptedPassword);
    const std::vector<PasswordEntry>& getEntriesRef() const;
};

#endif

```

Listing 10 — PasswordVault.h

## PasswordVault.cpp

```

#include "PasswordVault.h"
#include <string>
#include <ctime>
#include <iostream>
using namespace std;

static const string MASTER_FILE = "master_passwords.txt";

string MasterPasswordFile::shift(const string& s) {
    string out = s;
    for (int i = 0; i < (int)out.size(); i++) {
        out[i] = (char)(out[i] ^ 77);
    }
    return out;
}

bool MasterPasswordFile::exists(const string& vaultName) {
    ifstream file(MASTER_FILE);
    if (!file.is_open()) return false;

    string line;
    while (getline(file, line)) {

        if (!line.empty() && line.back() == '\r') line.pop_back();
        if (line.empty()) continue;

        int colon = -1;
        for (int i = 0; i < (int)line.size(); i++) {
            if (line[i] == ':') { colon = i; break; }
        }
    }
}

```

```

        }
        if (colon == -1) continue;

        string name = line.substr(0, colon);
        if (name == vaultName) {
            file.close();
            return true;
        }
    }
    file.close();
    return false;
}

bool MasterPasswordFile::save(const string& vaultName, const string& plainPassword) {
    ofstream file(MASTER_FILE, ios::app);
    if (!file.is_open()) return false;

    file << vaultName << ":" << shift(plainPassword) << "\n";
    file.close();
    return true;
}

int MasterPasswordFile::verify(const string& vaultName, const string& plainPassword) {
    ifstream file(MASTER_FILE);
    if (!file.is_open()) return -1;

    string line;
    while (getline(file, line)) {
        if (!line.empty() && line.back() == '\r') line.pop_back();
        if (line.empty()) continue;

        int colon = -1;
        for (int i = 0; i < (int)line.size(); i++) {
            if (line[i] == ':') { colon = i; break; }
        }
        if (colon == -1) continue;

        string name      = line.substr(0, colon);
        string stored     = line.substr(colon + 1);

        if (name == vaultName) {
            file.close();

            string decrypted = shift(stored);
            if (decrypted == plainPassword) return 1;
            else return 0;
        }
    }
    file.close();
    return -1;
}

PasswordVault::PasswordVault()
    : vaultName(""), filePath(""), isOpen(false) {}

bool PasswordVault::create(const string& name, const string& masterpassword) {
    if (masterpassword.size() < 8 || masterpassword.size() > 20)
        return false;

    if (MasterPasswordFile::exists(name))
        return false;

    if (!MasterPasswordFile::save(name, masterpassword))
        return false;

    vaultName = name;
    filePath  = name + ".csv";
    guard     = AuthGuard(masterpassword);

    loadFromFile();
}

```

```

        isOpen = true;
        return true;
    }

    bool PasswordVault::open(const string& name, const string& masterpassword) {
        if (guard.getLockstatus()) return false;

        int result = MasterPasswordFile::verify(name, masterpassword);

        if (result == 1) {
            vaultName = name;
            filePath = name + ".csv";
            guard.resetfailCount();
            loadFromFile();
            isOpen = true;
            return true;
        }

        guard.verifyMasterPassword(masterpassword);
        return false;
    }

    bool PasswordVault::getIsOpen() const { return isOpen; }
    int PasswordVault::getentryCount() const { return (int)entries.size(); }
    bool PasswordVault::isVaultLocked() { return guard.getLockstatus(); }
    int PasswordVault::getLockRemainingTime() { return guard.getRemainingTime(); }
    string PasswordVault::decrypt(string enc) { return decryptpass(enc); }

    vector<PasswordEntry> PasswordVault::getAllEntries() const {
        if (isOpen) return entries;
        return {};
    }

    const vector<PasswordEntry>& PasswordVault::getEntriesRef() const {
        return entries;
    }

    void PasswordVault::saveEntry(string label, string password) {
        if (!isOpen) return;

        for (int i = 0; i < (int)label.size(); i++)
            if (label[i] == ',') label[i] = ' ';

        if (searchEntry(label) != nullptr) {
            cout << " Error: Label '" << label << "' already exists!\n";
            return;
        }

        time_t t = time(0);
        tm* now = localtime(&t);
        string d = (now->tm_mday < 10 ? "0" : "") + to_string(now->tm_mday);
        string m = (now->tm_mon+1 < 10 ? "0" : "") + to_string(now->tm_mon+1);
        string y = to_string(now->tm_year + 1900);
        string formattedDate = d + "-" + m + "-" + y;

        entries.push_back(PasswordEntry(label, formattedDate, encryptpass(password)));
        saveToFile();
    }

    bool PasswordVault::deleteEntry(string label) {
        if (!isOpen) return false;
        for (auto it = entries.begin(); it != entries.end(); ++it) {
            if (it->getLabel() == label) {
                entries.erase(it);
                saveToFile();
                return true;
            }
        }
    }

```

```

    return false;
}

const PasswordEntry* PasswordVault::searchEntry(string label) {
    if (!isOpen) return nullptr;
    for (auto it = entries.begin(); it != entries.end(); ++it)
        if (it->getLabel() == label) return &(*it);
    return nullptr;
}

string PasswordVault::encryptpass(string password) {
    char key = 9;
    for (int i = 0; i < (int)password.size(); i++) password[i] ^= key;
    return password;
}

string PasswordVault::decryptpass(string password) {
    char key = 9;
    for (int i = 0; i < (int)password.size(); i++) password[i] ^= key;
    return password;
}

void PasswordVault::saveToFile() {
    if (filePath.empty()) return;
    ofstream file(filePath);
    if (!file.is_open()) return;
    for (const auto& entry : entries) {
        file << entry.getLabel() << ", "
              << entry.getDateAdded() << ", "
              << entry.getEncryptedPassword() << "\n";
    }
    file.close();
}

void PasswordVault::loadFromFile() {
    entries.clear();
    if (filePath.empty()) return;
    ifstream file(filePath);
    if (!file.is_open()) return;
    string label, dateadded, encryptedpassword;
    while (getline(file, label, ',') &&
           getline(file, dateadded, ',') &&
           getline(file, encryptedpassword)) {
        if (!encryptedpassword.empty() && encryptedpassword.back() == '\r')
            encryptedpassword.pop_back();
        entries.push_back(PasswordEntry(label, dateadded, encryptedpassword));
    }
    file.close();
}

```

Listing 11 — PasswordVault.cpp

## A.7 LeakChecker

### LeakChecker.h

```

#ifndef LeakChecker_h
#define LeakChecker_h
#include<vector>
#include<string>

class LeakChecker{
private:
    std::vector<std::string> commonLeaks;

public:

```

```

        LeakChecker();
        LeakChecker(std::string filename);
        bool isLeaked(std::string password);
};
#endif

```

Listing 12 — LeakChecker.h

## LeakChecker.cpp

```

#include "LeakChecker.h"
#include <fstream>
#include <iostream>
#include <cctype>
using namespace std;

LeakChecker::LeakChecker() {}

LeakChecker::LeakChecker(string filename) {
    ifstream file(filename);
    string line;

    if (file.is_open()) {
        while (getline(file, line)) {
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (!line.empty()) {
                string lower = "";
                for (char c : line) lower += (char)tolower(c);
                commonLeaks.push_back(lower);
            }
        }
        file.close();
    } else {
        cout << "Warning: Leaks file (" << filename << ") not found!" << endl;
    }
}

bool LeakChecker::isLeaked(string password) {
    string lower = "";
    for (char c : password) lower += (char)tolower(c);

    for (const string& leakedPass : commonLeaks) {
        if (lower == leakedPass) {
            return true;
        }
    }
    return false;
}

```

Listing 13 — LeakChecker.cpp

## A.8 HealthDashboard

### HealthDashboard.h

```

#ifndef HEALTHDASHBOARD_H
#define HEALTHDASHBOARD_H

#include <vector>
#include <string>

```

```

#include "PasswordEntry.h"
#include "LeakChecker.h"
#include "PasswordEvaluator.h"
#include "PasswordVault.h"

class HealthDashboard {
private:
    int totalEntries;
    int compromisedCount;
    int weakCount;

public:
    HealthDashboard();

    void analyzeVault(const PasswordVault& vault,
                     LeakChecker& lc,
                     PasswordEvaluator& pe);

    void displaySummary() const;
};

#endif

```

*Listing 14 — HealthDashboard.h***HealthDashboard.cpp**

```

#include "HealthDashboard.h"
#include <iostream>
#include "WinCompat.h"
using namespace std;

static void setDashColor(int colorCode) {
    HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleTextAttribute(hConsole, colorCode);
}

static void resetDashColor() { setDashColor(7); }

static string decryptForCheck(string p) {
    for (int i = 0; i < (int)p.size(); i++) p[i] ^= 9;
    return p;
}

HealthDashboard::HealthDashboard()
    : totalEntries(0), compromisedCount(0), weakCount(0) {}

void HealthDashboard::analyzeVault(const PasswordVault& vault,
                                   LeakChecker& lc,
                                   PasswordEvaluator& pe) {

    totalEntries    = vault.getentryCount();
    compromisedCount = 0;
    weakCount       = 0;

    if (totalEntries == 0) {
        setDashColor(14);
        cout << "\n Vault is empty – nothing to scan.\n";
        resetDashColor();
        return;
    }

    cout << "\n";
    setDashColor(11);
    cout << " --- Initializing Vault Health Scan ---\n";
    resetDashColor();
}

```

```

vector<PasswordEntry> entries = vault.getAllEntries();

for (int i = 0; i < (int)entries.size(); i++) {
    string plainPass = decryptForCheck(entries[i].getEncryptedPassword());

    bool leaked = lc.isLeaked(plainPass);

    int score = 0;
    try {
        pe.setPassword(plainPass);
        pe.evaluate();
        score = pe.getScore();
    } catch (...) {
        weakCount++;
        setDashColor(14);
        cout << " Checking [" << entries[i].getLabel() << "]: WEAK (invalid format)\n";
        resetDashColor();
        continue;
    }

    if (leaked)        compromisedCount++;
    if (score < 50) weakCount++;

    cout << " Checking [" << entries[i].getLabel() << "]: ";
    if (leaked) {
        setDashColor(12);
        cout << "CRITICAL (LEAKED)";
    } else if (score < 50) {
        setDashColor(14);
        cout << "WEAK (score: " << score << ")";
    } else {
        setDashColor(10);
        cout << "SECURE (score: " << score << ")";
    }
    resetDashColor();
    cout << "\n";
}

}

void HealthDashboard::displaySummary() const {
    setDashColor(11);
    cout << "\n =====\n";
    cout << "          FINAL HEALTH REPORT\n";
    cout << " =====\n";
    resetDashColor();

    cout << " Total Passwords Managed : " << totalEntries << "\n";

    setDashColor(12);
    cout << " Compromised (Leaked)      : " << compromisedCount << "\n";
    setDashColor(14);
    cout << " Vulnerable (Weak)          : " << weakCount << "\n";

    setDashColor(11);
    cout << " -----\n";

    float healthScore = (totalEntries > 0)
        ? ((float)(totalEntries - compromisedCount) / totalEntries) * 100.0f
        : 0.0f;

    cout << " OVERALL VAULT HEALTH      : ";
    if (healthScore >= 80) setDashColor(10);
    else if (healthScore >= 50) setDashColor(14);
    else setDashColor(12);

    cout << (int)healthScore << "%\n";
    resetDashColor();
    cout << " =====\n\n";
}

```

Listing 15 — HealthDashboard.cpp

## A.9 main.cpp

### main.cpp

```
#include "WinCompat.h"
#include <iostream>
#include <cstdlib>
#include "PasswordGenerator.h"
#include "PasswordEvaluator.h"
#include "PasswordVault.h"
#include "LeakChecker.h"
#include "HealthDashboard.h"
using namespace std;

string runPasswordGenerator();

static void setColor(int code) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), code);
}
static void resetColor() { setColor(7); }

PasswordVault openVaultSession() {
    PasswordVault vault;

    setColor(11);
    cout << "\n ---- Vault Access ----\n\n";
    resetColor();

    cout << " [1] Create a new vault\n";
    cout << " [2] Open an existing vault\n\n";

    int choice = askInt(" Enter choice: ", 1, 2);

    string vaultName;
    setColor(8); cout << "\n Enter vault name (e.g. work, personal): "; resetColor();
    cin >> vaultName;
    cin.ignore(1000, '\n');

    if (vaultName.empty()) {
        setColor(12); cout << "\n Vault name cannot be empty.\n"; resetColor();
        return vault;
    }

    if (choice == 1) {
        if (MasterPasswordFile::exists(vaultName)) {
            setColor(12);
            cout << "\n A vault named '" << vaultName << "' already exists.\n";
            cout << " Use option [2] to open it.\n";
            resetColor();
            return vault;
        }

        string masterPwd;
        setColor(8); cout << " Set a master password (8-20 characters): "; resetColor();
        cin >> masterPwd;
        cin.ignore(1000, '\n');

        if (masterPwd.size() < 8 || masterPwd.size() > 20) {
            setColor(12);
            cout << "\n Master password must be 8 to 20 characters.\n";
            resetColor();
            return vault;
        }

        if (!vault.create(vaultName, masterPwd)) {
```

```

        setColor(12);
        cout << "\n Failed to create vault. Check write permissions.\n";
        resetColor();
        return vault;
    }

    setColor(10);
    cout << "\n Vault '" << vaultName << "' created successfully!\n";
    cout << " Master password saved to registry (encrypted).\n";
    resetColor();
    Sleep(800);
    system("cls");
    return vault;
}

else {
    if (!MasterPasswordFile::exists(vaultName)) {
        setColor(12);
        cout << "\n No vault named '" << vaultName << "' found.\n";
        cout << " Use option [1] to create it first.\n";
        resetColor();
        return vault;
    }

    for (int attempt = 1; attempt <= 3; attempt++) {
        if (vault.isVaultLocked()) {
            while (true) {
                int remaining = vault.getLockRemainingTime();
                if (remaining <= 0) break;
                setColor(12);
                cout << "\r Vault locked. Please wait " << remaining
                    << " second(s)... ";
                resetColor();
                cout.flush();

                time_t start = time(0);
                while (difftime(time(0), start) < 1.0) {}
            }
            setColor(14);
            cout << "\n\n Lockout expired. You may try again.\n\n";
            resetColor();

            attempt = 1;
        }

        string masterPwd;
        setColor(8);
        cout << " Enter master password (attempt " << attempt << "/" << 3 << "): ";
        resetColor();
        cin >> masterPwd;
        cin.ignore(1000, '\n');

        if (vault.open(vaultName, masterPwd)) {
            setColor(10);
            cout << "\n Vault '" << vaultName << "' opened.\n";
            resetColor();

            Sleep(600);
            system("cls");
            return vault;
        }

        if (vault.isVaultLocked()) {
            continue;
        }

        int left = 3 - attempt;
    }
}

```

```

        setColor(12);
        if (left > 0)
            cout << " Wrong password. " << left << " attempt(s) remaining.\n";
        resetColor();
    }

    setColor(12);
    cout << "\n Too many failed attempts. Vault locked for 30 seconds.\n";
    resetColor();
    return vault;
}

}

void runPasswordEvaluator() {
    system("cls");
    setColor(11);
    cout << "\n =====\n";
    cout << "          PASSWORD STRENGTH EVALUATOR\n";
    cout << " =====\n";
    resetColor();

    PasswordEvaluator evaluator;
    char again = 'y';

    while (again == 'y' || again == 'Y') {
        string pwd = "";

        while (true) {
            setColor(8);
            cout << "\n Enter password to evaluate: ";
            resetColor();
            cin >> pwd;
            cin.ignore(1000, '\n');

            try {
                evaluator.setPassword(pwd);
                break;
            }
            catch (invalid_argument& e) {
                setColor(12);
                cout << " Error: " << e.what() << "\n";
                resetColor();
            }
        }

        try {
            evaluator.evaluate();
            evaluator.displayReport();
        }
        catch (runtime_error& e) {
            setColor(12);
            cout << " Error: " << e.what() << "\n";
            resetColor();
        }

        setColor(8);
        cout << " Evaluate another password? (y/n): ";
        resetColor();
        cin >> again;
        cin.ignore(1000, '\n');
    }
}

void runGeneratorMenu() {
    string generated = runPasswordGenerator();
    (void)generated;
}

void runVaultMenu() {
    system("cls");

```

```

setColor(11);
cout << "\n =====\n";
cout << "          PASSWORD VAULT\n";
cout << " =====\n\n";
resetColor();

PasswordVault vault = openVaultSession();

if (!vault.getIsOpen()) {
    cout << "\n Press Enter to continue...";
    cin.ignore(1000, '\n');
    return;
}

while (true) {
    system("cls");
    setColor(11);
    cout << "\n =====\n";
    cout << "          PASSWORD VAULT\n";
    cout << " =====\n";
    resetColor();
    setColor(10);
    cout << " Entries: " << vault.getEntryCount() << "\n\n";
    resetColor();

    setColor(14); cout << " [1]"; resetColor(); cout << " View all entries\n";
    setColor(14); cout << " [2]"; resetColor(); cout << " Add a new entry\n";
    setColor(14); cout << " [3]"; resetColor(); cout << " Search for an entry\n";
    setColor(14); cout << " [4]"; resetColor(); cout << " Delete an entry\n";
    setColor(12); cout << " [0]"; resetColor(); cout << " Back to main menu\n\n";

    int sub = askInt(" Enter choice: ", 0, 4);
    if (sub == 0) break;

    if (sub == 1) {
        system("cls");
        setColor(11);
        cout << "\n ---- All Vault Entries ----\n\n";
        resetColor();

        vector<PasswordEntry> entries = vault.getAllEntries();
        if (entries.empty()) {
            setColor(14);
            cout << " Vault is empty. Add some entries first!\n";
            resetColor();
        } else {
            for (int i = 0; i < (int)entries.size(); i++) {
                setColor(14);
                cout << " [" << (i+1) << "] ";
                resetColor();
                cout << entries[i].getLabel();
                setColor(8);
                cout << " (added: " << entries[i].getDateAdded() << ")\n";
                resetColor();
            }

            cout << "\n Enter number to reveal password, or 0 to go back: ";
            int sel = askInt("", 0, (int)entries.size());
            if (sel > 0) {
                string plain = vault.decrypt(entries[sel-1].getEncryptedPassword());
                cout << "\n";
                setColor(10);
                cout << " Label      : " << entries[sel-1].getLabel() << "\n";
                cout << " Added      : " << entries[sel-1].getDateAdded() << "\n";
                cout << " Password   : " << plain << "\n";
                resetColor();
            }
        }
        cout << "\n Press Enter to continue...";
        cin.ignore(1000, '\n');
    }
}

```

```

    }

    else if (sub == 2) {
        system("cls");
        setColor(11);
        cout << "\n ---- Add New Entry ----\n\n";
        resetColor();

        string label, password;
        setColor(8); cout << " Label (e.g. Gmail, Netflix): "; resetColor();
        getline(cin, label);

        if (label.empty()) {
            setColor(12); cout << " Label cannot be empty.\n"; resetColor();
            cout << " Press Enter to continue..."; cin.ignore(1000, '\n');
            continue;
        }

        setColor(8); cout << " Password to save: "; resetColor();
        getline(cin, password);

        if (password.empty()) {
            setColor(12); cout << " Password cannot be empty.\n"; resetColor();
            cout << " Press Enter to continue..."; cin.ignore(1000, '\n');
            continue;
        }

        vault.saveEntry(label, password);
        setColor(10);
        cout << "\n Entry '" << label << "' saved!\n";
        resetColor();
        cout << " Press Enter to continue...";
        cin.ignore(1000, '\n');
    }

    else if (sub == 3) {
        system("cls");
        setColor(11);
        cout << "\n ---- Search Entry ----\n\n";
        resetColor();

        string label;
        setColor(8); cout << " Enter label to search: "; resetColor();
        getline(cin, label);

        const PasswordEntry* found = vault.searchEntry(label);
        if (found) {
            string plain = vault.decrypt(found->getEncryptedPassword());
            setColor(10);
            cout << "\n Found!\n";
            cout << " Label      : " << found->getLabel() << "\n";
            cout << " Added       : " << found->getDateAdded() << "\n";
            cout << " Password    : " << plain << "\n";
            resetColor();
        } else {
            setColor(12);
            cout << "\n No entry found with label '" << label << "'.\n";
            resetColor();
        }
        cout << "\n Press Enter to continue...";
        cin.ignore(1000, '\n');
    }

    else if (sub == 4) {
        system("cls");
        setColor(11);
        cout << "\n ---- Delete Entry ----\n\n";
        resetColor();

        string label;

```

```

        setColor(8); cout << " Enter label to delete: "; resetColor();
        getline(cin, label);

        if (vault.deleteEntry(label)) {
            setColor(10);
            cout << "\n Entry '" << label << "' deleted.\n";
        } else {
            setColor(12);
            cout << "\n No entry found with label '" << label << "'.\n";
        }
        resetColor();
        cout << " Press Enter to continue...";
        cin.ignore(1000, '\n');
    }
}

void runLeakChecker() {
    system("cls");
    setColor(11);
    cout << "\n =====\n";
    cout << "          LEAKED PASSWORD CHECKER\n";
    cout << " =====\n\n";
    resetColor();

    LeakChecker lc("leaked_passwords.txt");

    char again = 'y';
    while (again == 'y' || again == 'Y') {
        string password;
        setColor(8);
        cout << " Enter a password to check: ";
        resetColor();
        cin >> password;
        cin.ignore(1000, '\n');

        if (password.empty()) {
            setColor(12); cout << " Password cannot be empty.\n"; resetColor();
        } else {
            bool leaked = lc.isLeaked(password);
            cout << "\n Result: ";
            if (leaked) {
                setColor(12);
                cout << "COMPROMISED!\n";
                resetColor();
                setColor(14);
                cout << " This password was found in a known data breach.\n";
                cout << " Do NOT use it anywhere. Change it immediately if in use.\n";
            } else {
                setColor(10);
                cout << "NOT found in the leak database.\n";
                resetColor();
                setColor(8);
                cout << " (Still evaluate it with Option 1 to check its strength.)\n";
            }
            resetColor();
        }

        cout << "\n Check another? (y/n): ";
        cin >> again;
        cin.ignore(1000, '\n');
    }
}

void runHealthDashboard() {
    system("cls");
    setColor(11);
    cout << "\n =====\n";
    cout << "          VAULT HEALTH DASHBOARD\n";
    cout << " =====\n\n";

```

```

resetColor();

string vaultName;
setColor(8); cout << " Enter vault name: "; resetColor();
cin >> vaultName;
cin.ignore(1000, '\n');

if (vaultName.empty()) {
    setColor(12); cout << "\n Vault name cannot be empty.\n"; resetColor();
    cout << "\n Press Enter to continue...";
    cin.ignore(1000, '\n');
    return;
}

if (!MasterPasswordFile::exists(vaultName)) {
    setColor(12);
    cout << "\n No vault named '" << vaultName << "' found.\n";
    cout << " Use the Password Vault option to create one first.\n";
    resetColor();
    cout << "\n Press Enter to continue...";
    cin.ignore(1000, '\n');
    return;
}

PasswordVault vault;
for (int attempt = 1; attempt <= 3; attempt++) {
    if (vault.isVaultLocked()) {
        while (true) {
            int remaining = vault.getLockRemainingTime();
            if (remaining <= 0) break;
            setColor(12);
            cout << "\r Vault locked. Please wait " << remaining << " second(s)... ";
            resetColor();
            cout.flush();
            time_t start = time(0);
            while (difftime(time(0), start) < 1.0) {}
        }
        setColor(14);
        cout << "\n\n Lockout expired. You may try again.\n\n";
        resetColor();
        attempt = 1;
    }

    string masterPwd;
    setColor(8);
    cout << " Enter master password (attempt " << attempt << "/" << 3): ";
    resetColor();
    cin >> masterPwd;
    cin.ignore(1000, '\n');

    if (vault.open(vaultName, masterPwd)) {
        setColor(10);
        cout << "\n Vault '" << vaultName << "' opened. Running health scan...\n";
        resetColor();
        Sleep(500);
        break;
    }

    if (vault.isVaultLocked()) continue;

    int left = 3 - attempt;
    setColor(12);
    if (left > 0)
        cout << " Wrong password. " << left << " attempt(s) remaining.\n";
    resetColor();
}

if (!vault.getIsOpen()) {
    setColor(12);
    cout << "\n Could not open vault. Returning to menu.\n";
}

```

```

        resetColor();
        cout << "\n Press Enter to continue...";
        cin.ignore(1000, '\n');
        return;
    }

    LeakChecker      lc("leaked_passwords.txt");
    PasswordEvaluator pe;
    HealthDashboard  hd;

    hd.analyzeVault(vault, lc, pe);
    hd.displaySummary();

    cout << " Press Enter to continue...";
    cin.ignore(1000, '\n');
}

void showMenu() {
    system("cls");
    setColor(11);
    cout << "\n =====\n";
    cout << "          PASSWORD SECURITY SUITE v1.0\n";
    cout << " =====\n";
    resetColor();

    setColor(8);
    cout << " Your personal password security toolkit\n\n";
    resetColor();

    setColor(14); cout << " [1]"; resetColor(); cout << " Evaluate a Password\n";
    setColor(14); cout << " [2]"; resetColor(); cout << " Generate a Password\n";
    setColor(14); cout << " [3]"; resetColor(); cout << " Password Vault\n";
    setColor(14); cout << " [4]"; resetColor(); cout << " Check Password Leak\n";
    setColor(14); cout << " [5]"; resetColor(); cout << " Vault Health Dashboard\n";

    cout << "\n";
    setColor(12); cout << " [0]"; resetColor(); cout << " Exit\n";

    setColor(11);
    cout << "\n =====\n";
    resetColor();
}

int main() {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD dwMode = 0;
    GetConsoleMode(hOut, &dwMode);
    SetConsoleMode(hOut, dwMode);

    string input = "";
    int choice = -1;

    while (true) {
        showMenu();

        setColor(8);
        cout << " Enter your choice: ";
        resetColor();

        cin >> input;
        cin.ignore(1000, '\n');

        bool valid = (input.length() == 1 && isdigit(input[0]));
        if (!valid) {
            setColor(12);
            cout << "\n Invalid input. Please enter a number from the menu.\n";
            resetColor();
            cout << " Press Enter to continue...";
            cin.ignore(1000, '\n');
            continue;
        }
    }
}

```

```
    }

    choice = stoi(input);

    if (choice == 0) {
        setColor(10);
        cout << "\n Goodbye! Stay secure.\n\n";
        resetColor();
        break;
    }

    switch (choice) {
        case 1: runPasswordEvaluator(); break;
        case 2: runGeneratorMenu();      break;
        case 3: runVaultMenu();          break;
        case 4: runLeakChecker();        break;
        case 5: runHealthDashboard();    break;
        default:
            setColor(12);
            cout << "\n Invalid choice.\n";
            resetColor();
            cout << " Press Enter to continue...";
            cin.ignore(1000, '\n');
            break;
    }

    return 0;
}
```

*Listing 16 — main.cpp*

— End of Report —